

Complément d'informatique

INFO0952

Pierre Geurts

Dernière mise à jour le 14 septembre 2026

E-mail : p.geurts@uliege.be
URL : <http://www.montefiore.ulg.ac.be/~geurts/>
Bureau : 1.134 (Montefiore)
Téléphone : 04.366.48.15

Contact

- Professeur :
 - ▶ Pierre Geurts, p.geurts@uliege.be, 1.134 Montefiore, 04/3664815
- Assistants :
 - ▶ Jérôme Pierre, jerome.pierre@uliege.be
 - ▶ Maxime Amodei, maxime.amodei@uliege.be
- Site web du cours :
 - ▶ Ecampus pour tout le matériel lié au cours :
<https://www.ecampus.ulg.ac.be>
 - ▶ Plate-forme de soumission de tp/projets : <https://gradescope.com>
(Code cours : X2D3XE)
- Comment nous contacter (par ordre de priorité) ?
 - ▶ **Lors du cours ou des séances de travaux dirigés**
 - ▶ Via les forums de discussion sur Ecampus
 - ▶ Par email : info0952@uliege.be (pas d'envoi de code à débbuger les jours de deadline)
 - ▶ A mon bureau (sur rendez-vous)

Les objectifs du cours

- Consolider et étendre vos connaissances d'un langage de programmation (le C)
- Vous apprendre à écrire des programmes pour résoudre des problèmes réalistes (de taille moyenne)
- Vous initier à l'algorithmique et à l'étude des structures de données
- Vous ouvrir à d'autres paradigmes et langages (le Python)
- Et, on ne sait jamais, vous donner le goût pour la programmation (et l'informatique)

L'informatique dans le bachelier ingénieur

Bloc 1 :

- (Obl) INFO2009 - Introduction à l'informatique
- (Obl) INFO0061 - Organisation des ordinateurs

Bloc 2 :

- **(Obl) INFO0952 - Complément d'informatique**
- INFO0902 - Structures de données et algorithmes
- INFO0062 - Object-oriented programming

Bloc 3 :

- INFO0012 - Computation structures
- INFO9012 - Parallel programming
- INFO0009 - Base de données
- INFO0054 - Programmation fonctionnelle
- INFO0010 - Introduction to computer networking
- INFO8006 - Introduction to artificial intelligence

Approche pédagogique

Apprentissage par la pratique :

- 7 séances de travaux dirigés sur ordinateur encadrées par des assistants et élèves-moniteurs
- Un projet de programmation à réaliser individuellement, lors de séances de travaux dirigés et à la maison.
- Deux projets de plus grande envergure à faire seul ou par groupe de deux, lors de séances de travaux dirigés et à la maison.

Cours “théorique” :

- Compléments de programmation en C (surtout) et en python (un peu), en particulier sur l'écriture et l'organisation de programmes
- Compléments d'algorithmique et structures de données
- Feedback général sur les projets

Organisation pratique

Cours théoriques :

- Le mardi de 13h45 à 15h45 maximum, pas toutes les semaines (8 séances maximum).
- Transparents disponibles sur Ecampus.

Travaux pratiques :

- Le mardi de 15h45 à 17h45 (7 séances, après le cours théorique) ou de 13h45 à 17h45 max (3 séances, dédiées aux projets).
- Instructeurs : 2 assistants, 4 élèves-moniteurs et le professeur.
- Exercices sur ordinateur ou sur feuille portant sur la matière théorique ou la réalisation des projets. Enoncés disponibles sur Ecampus avant la séance.

Projets :

- Trois projets au total, le premier à faire seul, les deux suivants, seul ou en binôme (idéalement).

En première session :

- Participation aux travaux pratiques : 16%
 - ▶ 4 exercices de TP à soumettre sur la plateforme (pour 4% chacun).
- 3 projets : 84% au total (28% par projet).

Critères d'évaluation :

- ▶ Exactitude du code : testée automatiquement, en partie lors de la soumission des projets sur la plateforme
 - ▶ Style et utilisation du langage : vérification rapide, menant à un cotation (presque) binaire : ok ou pas.
 - ▶ Rapport : précision et qualité des réponses
 - ▶ Une interrogation écrite par projet :
- Pas d'examen en session

Une interrogation écrite par projet :

- Obligatoire (0 au projet en cas d'absence), organisée le mardi suivant la date de remise, en début de cours.
- QCM ou questions demandant des réponses courtes, sur le projet et l'utilisation du langage C
- L'objectif est de vérifier que vous avez participé activement aux projets
- Convocation en cas d'échec grave à l'interrogation et de réussite brillante au projet
- Intervient pour un tiers des points du projet

Remarque : En cas de réalisation des projets à deux, vous devrez remettre un document précisant la contribution (en %) de chacun au projet. La cote d'un étudiant pourrait être réduite en cas de participation trop faible.

En deuxième session :

- Les projets non rendus ou ratés devront être refaits (avec un énoncé éventuellement modifié).
- Les projets réussis ne peuvent pas être refaits.
- Les exercices de TP ne peuvent pas être refaits. Ils ne sont pris en compte dans le note finale que s'ils ont été rendus pendant le semestre et dans le cas où ils sont favorables à l'étudiant.
- Interrogations remplacée par un examen oral.
- Pondération identique à la première session.
- Pas d'encadrement garanti pendant l'été.

Pas de report de cote, même partiel, d'une année à l'autre

Plagiat

- La collaboration entre étudiants sur le cours théorique, la programmation C, l'utilisation d'outils et les exercices de répétition est fortement encouragée
- Discuter des concepts généraux liés aux projets entre étudiants est permis.
- **Montrer son code à d'autres étudiants, regarder/copier, même partiellement, le code d'autres étudiants ou du code obtenu via d'autres sources est strictement interdit !**
- Des outils sophistiqués de détection de plagiat, robustes à des modifications de noms de variables ou des réarrangements de code, seront utilisés systématiquement.
- **En cas de plagiat avéré (après convocation des étudiants incriminés), des sanctions seront appliquées et le président de jury sera informé.**

Politique de collaboration

	Partenaire*	Encadrants	Étudiants	Chatbots**	Autres***
Discuter les concepts avec	✓	✓	✓	✓	✓
Montrer votre code à	✓	✓	✗	✗	✗
Regarder le code de	✓	✗	✗	✗	✗
Copier le code de	✓	✗	✗	✗	✗

* de projet

** ChatGPT, Copilot, Claude, Gemini, etc.

*** Parents, amis, etc.

Notes de cours

Les transparents des cours théoriques disponibles sur la page web du cours avant chaque leçon.

Pas de livre de référence obligatoire mais les livres suivants ont été utilisés pour préparer le cours :

- C programming : a modern approach, K.N. King, W. W. Norton & Company, Second edition, 2008.
- Le langage C - Norme ANSI, Kernighan et Ritchie, Dunod, 2ème édition, 2000.
- Computer science : an interdisciplinary approach, Sedgewick et Wayne, Pearson, 2016.
- Introduction to algorithms, Cormen, Leiserson, Rivest, Stein, MIT press, Fourth edition, 2022.

Contenu du cours

- Partie 1: Rappel de C
- Partie 2: Construction et analyse d'algorithmes
- Partie 3: Algorithmique
- Partie 4: Organisation de programmes
- Partie 5: Structures de données
- Partie 6: Python et langages de programmation

Partie 1

Rappel de C

14 septembre 2026

Plan

1. Un tour rapide du C via un exemple
2. Rappel sur les pointeurs

Plan

1. Un tour rapide du C via un exemple
2. Rappel sur les pointeurs

Illustration : l'ensemble de Mandelbrot

En mathématique, l'ensemble de Mandelbrot $\mathcal{M}$ est défini comme l'ensemble des points $c \in \mathbb{C}$ du plan complexe pour lesquels la suite de nombres définie par la récurrence suivante :

$$\begin{cases} z_0 = 0, \\ z_{n+1} = z_n^2 + c \quad (n > 0). \end{cases}$$

est bornée.

Étudié depuis le début du vingtième siècle en dynamique complexe mais représenté pour la première fois à la fin des années septante grâce à l'ordinateur.

On aimerait implémenter un programme en C permettant de visualiser cet ensemble dans le plan complexe.

La première représentation de l'ensemble (en 1978)

[illegible]

https://fr.wikipedia.org/wiki/Ensemble_de_Mandelbrot

Vérifier l'appartenance à $\mathcal{M}$

Impossible à vérifier formellement mais on peut néanmoins démontrer que si la suite des modules devient strictement supérieure à 2 pour un certain indice n , alors, la suite est croissante et tend vers l'infini à partir de cet indice.

On peut donc calculer un surensemble de $\mathcal{M}$ en testant la contrainte $|z_n| \leq 2$ pour des valeurs aussi grande que possible de n .

En pratique, on est obligé de se limiter à un nombre maximum N d'itérations, si on veut implémenter ce test empiriquement sur ordinateur.

Notre programme vérifiera donc en fait l'appartenance à l'ensemble $\mathcal{M}' \subseteq \mathcal{M}$ défini comme suit :

$$\mathcal{M}' = \{c \in \mathbb{C} \mid \forall n, 0 \leq n \leq N : |z_n| \leq 2\}$$

Vérifier l'appartenance à $\mathcal{M}'$

L'idée du programme est donc de calculer z_n pour des valeurs de n allant de 0 à N , en s'arrêtant dès que $|z_n| > 2$.

A priori, on ne peut pas manipuler des nombres complexes directement en $\mathbb{C}$ mais la suite peut se réécrire comme suit sur base des parties réelles et imaginaires :

$$\begin{cases} \operatorname{Re}(z_0) = 0, \operatorname{Im}(z_0) = 0, \\ \operatorname{Re}(z_{n+1}) = \operatorname{Re}(z_n)^2 - \operatorname{Im}(z_n)^2 + \operatorname{Re}(c), \\ \operatorname{Im}(z_{n+1}) = 2\operatorname{Re}(z_n)\operatorname{Im}(z_n) + \operatorname{Im}(c), \end{cases}$$

où $\operatorname{Re}(z)$ et $\operatorname{Im}(z)$ désigne resp. les parties réelle et imaginaire d'un complexe z .

Vérifier l'appartenance à $\mathcal{M}'$ (en C)

1/3

En supposant que les variables `cr` et `ci` (double) contiennent les parties imaginaires et réelles de c , le code suivant vérifie la non-appartenance de c à $\mathcal{M}$:

```
double zr = 0;
double zi = 0;
int n = 0;

while((n < N) && (zr*zr + zi*zi <= 4.0)) {
    double temp;
    temp = zr*zr - zi*zi + cr;
    zi = 2*zr*zi + ci;
    zr = temp;
    n++;
}
if (zr*zr + zi*zi <= 4.0)
    printf("c belongs to M'\n");
else
    printf("c does not belong to M'\n");
```

Quel est l'invariant de cette boucle ?

Vérifier l'appartenance à $\mathcal{M}'$ (en C)

2/3

En utilisant un for à la place d'un while.

```
double zr = 0;
double zi = 0;

for(int n = 0; (n < N) && (zr*zr + zi*zi <= 4.0); n++) {
    double temp;
    temp = zr*zr - zi*zi + cr;
    zi = 2*zr*zi + ci;
    zr = temp;
}
if (zr*zr + zi*zi <= 4.0)
    printf("c belongs to M'\n");
else
    printf("c does not belong to M'\n");
```

Rappel de C : déclaration et types primitifs

```
int a = 1, b, c;  
float e;
```

- Toute variable doit être déclarée en spécifiant son type et peut être initialisée au moment de sa déclaration

- Type primitif :

bool	true ou false (ISO-C99, avec stdbool.h)
char	caractère signé
int	entier signé
size_t	entier non-signé représentant une taille ou un indice
float	nombre réel (précision simple)
double	nombre réel (précision double)

- Le typage du C est **statique** (le type d'une variable est déterminé à la compilation) et **faible** (une valeur peut être convertie implicitement vers le type adéquat).

Rappel de C : Opérateurs

(par ordre de précedence)

postfixe	<code>[] . -> expr++ expr--</code>
préfixe	<code>++expr --expr +expr -expr ~ !</code> <code>&expr *expr sizeof (type)expr</code>
multiplicatifs	<code>* / %</code>
additifs	<code>+ -</code>
décalages	<code><< >></code>
comparaisons	<code>< > <= >=</code>
égalité	<code>== !=</code>
ET binaire	<code>&</code>
OU exclusif binaire	<code>^</code>
OU binaire	<code> </code>
ET logique	<code>&&</code>
OU logique	<code> </code>
conditionnel	<code>?:</code>
affectations	<code>= += -= *= /= %= <<= >>= &= ^= =</code>

Rappel de C : choix conditions

Choix binaire

```
if (expr) {  
    ...  
}  
  
if (expr) {  
    ...  
} else {  
    ...  
}
```

Choix multiple

```
switch(expr) {  
    case const1 : instr1 break;  
    case const2 : instr2 break;  
    ...  
    default : instr  
}
```

Expression conditionnelle

```
expr1 ? expr2 : expr3;
```

Rappel de C : boucles

```
for (expr1; expr2; expr3) {  
    ...  
}  
  
while (expr) {  
    ...  
}  
  
do {  
    ...  
} while (expr);
```

Interruption :

- L'instruction `break` permet de quitter la boucle courante.
- L'instruction `continue` permet de passer à l'itération suivante, sans exécuter le restant de l'itération courante.

Rappel de C : entrées-sorties

```
#include <stdio.h>
...
int a, b;

printf("Entrez une première valeur: ");
scanf("%d", &a);
printf("Entrez une seconde valeur: ");
scanf("%d", &b);

printf("%d + %d = %d\n", a, b, a + b);
...
```

- `printf` prend comme premier argument une chaîne formatée. Les arguments suivant sont les valeurs affectées aux spécificateurs de format (c.f. http://en.wikipedia.org/wiki/Printf_format_string#Format_placeholders pour une spécification complète).
- `scanf` permet d'entrer une valeur au clavier et de la stocker à l'adresse spécifiée. Attention : la gestion propre des erreurs est difficile !

On peut emballer ce code dans une fonction

```
int mandelbrotSet(double cr, double ci) {  
    double zr = 0;  
    double zi = 0;  
    int n = 0;  
  
    while(n < N && ((zr*zr + zi*zi) <= 4.0)) {  
        double temp;  
        temp = zr*zr - zi*zi + cr;  
        zi = 2*zr*zi + ci;  
        zr = temp;  
        n++;  
    }  
    return (zr*zr + zi*zi <= 4.0);  
}
```

Rappel de C : fonctions et procédures

```
int fct1(int a, int b) {  
    ...  
    return 4;  
}  
int fct2(void) {  
    int a,b;  
    ...  
    return a+b;  
}  
void fct3(float b) {  
    ...  
    [return;]  
}
```

- Une fonction peut prendre zéro, un ou plusieurs arguments.
- Les arguments sont passés par valeur.
- Chaque fonction renvoie une valeur d'un type donné, ou void.
- Si une fonction renvoie une valeur, elle doit posséder une instruction `return` correspondant au bon type.

Un programme C complet pour visualiser l'ensemble

```
#include <stdio.h>
#include <stdlib.h>

#define N 1000 // Maximum number of iterations
#define W 1000 // Width/height of plot (pixels)

static int mandelbrotSet(double xc, double yc);

int main(int argc, char *argv[]) {
    double x = atof(argv[2]);
    double y = atof(argv[3]);
    double size = atof(argv[4]);

    FILE *fp = fopen(argv[1], "w");

    for (int i = 0; i < W; i++) {
        for (int j = 0; j < W; j++) {
            double cr = x - size/2 + size*j/W;
            double ci = y + size/2 - size*i/W;
            fprintf(fp, "%d ", mandelbrotSet(cr, ci));
        }
        fprintf(fp, "\n");
    }

    fclose(fp);

    return 0;
}
```

```
static int mandelbrotSet(double cr, double ci) {
    double zr = 0;
    double zi = 0;
    int n = 0;

    while(n < N && ((zr*zr + zi*zi) <= 4.0)) {
        double temp;
        temp = zr*zr - zi*zi + cr;
        zi = 2*zr*zi + ci;
        zr = temp;
        n++;
    }
    return n;
}
```

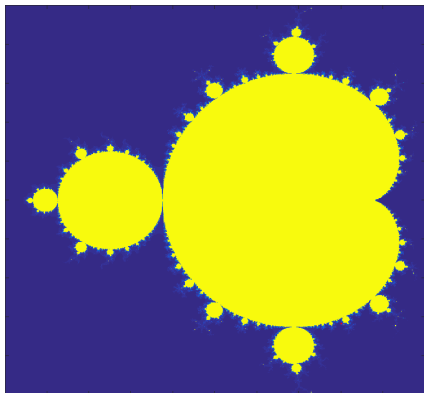
Utilisation :

```
> gcc mandelbrot.c -o mandelbrot
> ./mandelbrot m.amat -0.5 0.0 2.0
```

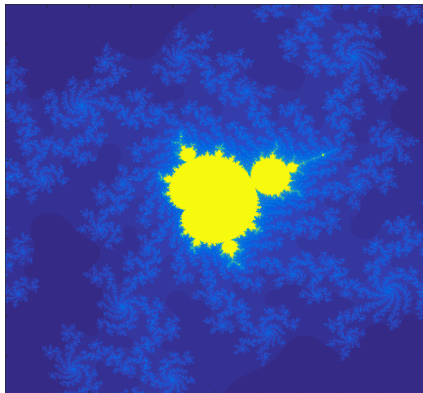
Pour voir l'image avec python

```
>> import matplotlib.pyplot as plt
>> import numpy as np
>> M = np.loadtxt('m.amat')
>> plt.pcolor(M); plt.show()
```

L'ensemble de Mandelbrot



```
./mandelbrot m.amat -0.5 0.0 2.0
```



```
./mandelbrot m.amat 0.1015 -0.633 0.001
```

Pour en voir plus : <http://tilde.club/~david/m>

Rappel de C : fonction main

```
int main() {  
    ...  
    return 0;  
}
```

```
int main(int argc, char *argv[]) {  
    ...  
    return 0;  
}
```

- Tout programme doit contenir une fonction main.
- La valeur de retour (entière) est rendue au système d'exploitation. Par convention, 0 signifie que tout s'est bien passé.
- On peut récupérer les paramètres passés au programme au moment de l'exécution en ajoutant deux arguments à la fonction :
 - ▶ `argc` : le nombre de paramètres passés (y compris le nom de l'exécutable)
 - ▶ `argv` : un tableau de chaînes de caractères dont les éléments contiennent ces arguments
- Ces arguments peuvent être analysés à l'aide des fonctions de la librairie `stdlib` (p.ex., `atoi`, `atof`).

Rappel de C : tableaux

```
int[5] a;  
a[0] = 1;  
a[4] = 42;  
a[-1] = 10; // Bug!  
a[5] = -5;   // Bug!  
  
char id[] = "texte"; // Chaîne de caractères  
  
int mat[3][4]; // Tableau multidimensionnel
```

- Un tableau est un type de données indexable contenant des éléments du même type.
- Les éléments sont indexés à partir de 0 et jusqu'à N-1.
- Les tableaux sont passés aux fonctions par **pointeurs** (voir plus loin). Leurs modifications sont donc répercutées à l'appelant.
- Une chaîne de caractère est un tableau de char terminé par un caractère null '\0'.

Rappel de C : organisation d'un programme

Organisation possible d'un programme C en un seul fichier (voir `mandelbrot.c`) :

- Directive d'inclusion (`#include`)
- Définition de constantes et macro (`#define`)
- Définitions de types (`typedef`, voir plus loin)
- Déclarations de variables globales
- Prototypes des fonctions autres que la fonction `main`
- Définition de la fonction `main`
- Définition des autres fonctions

Seul contrainte forte : toute fonction/variable/constante doit être définie avant d'être utilisée.

Autre variante utilisant un nouveau type

1/2

On peut simplifier le code principal et améliorer sa lisibilité en définissant un nouveau type de données pour les nombres complexes.

complex.c

complex.h

```
typedef struct {  
    double re, im;  
} complex;  
  
complex complex_new(double, double);  
complex complex_sum(complex, complex);  
complex complex_product(complex, complex);  
double complex_modulus(complex);  
...
```

```
#include <math.h>  
#include "complex.h"  
  
complex complex_new(double re, double im) {  
    complex c;  
    c.re = re;  
    c.im = im;  
    return c;  
}  
  
complex complex_sum(complex a, complex b) {  
    complex c;  
    c.re = a.re + b.re;  
    c.im = a.im + b.im;  
    return c;  
}  
  
complex complex_product(complex a, complex b) {  
    complex c;  
    c.re = a.re * b.re - a.im * b.im;  
    c.im = a.re * b.im + a.im * b.re;  
    return c;  
}  
  
double complex_modulus(complex c) {  
    return sqrt(c.re*c.re + c.im*c.im);  
}  
...
```

Rappel de C : structures

```
// Définition d'une structure
struct complex_t {
    double re, im;
};

// Définition d'un nouveau type
typedef struct {
    double re, im;
} complex;

// Utilisation
struct complex_t a, c = {1.2, 3.4};
complex b = {.im = 1.0, .re = 2.0};
b.re = 1.0;
a.im = 3.4;
```

- Une structure est un type de données composé, dont les éléments peuvent être de types différents.
- Les éléments de la structure sont accessibles par leurs noms via l'opérateur `'.'`.

```
#include <stdio.h>
#include <stdlib.h>
#include "complex.h"

#define N 1000 // Maximum number of iterations
#define W 1000 // Width/Height of the plot

static int mandelbrotSet(complex c);

int main(int argc, char *argv[]) {
    double x = atof(argv[2]);
    double y = atof(argv[3]);
    double size = atof(argv[4]);

    FILE *fp = fopen(argv[1], "w");

    for (int i = 0; i < W; i++) {
        for (int j = 0; j < W; j++) {
            double cr = x - size/2 + size*j/W;
            double ci = y + size/2 - size*i/W;
            complex c = complex_new(cr, ci);
            fprintf(fp, "%d ", mandelbrotSet(c));
        }
        fprintf(fp, "\n");
    }

    fclose(fp);

    return 0;
}
```

```
static int mandelbrotSet(complex c) {
    complex z = complex_new(0,0);
    int n = 0;
    while ((n < N) && (complex_modulus(z) <= 2.0)) {
        z = complex_plus(complex_product(z,z),c);
        n++;
    }
    return n;
}
```

Pour compiler :

```
> gcc mandelbrot.c complex.c -o mandelbrot
```

Autre variante utilisant la librairie complex du C99

```
#include <stdio.h>
#include <stdlib.h>
#include <complex.h>

#define N 1000 // Maximum number of iterations
#define W 1000 // Width/Height of the plot

static int mandelbrotSet(double _Complex c);

int main(int argc, char *argv[]) {
    double x = atof(argv[2]);
    double y = atof(argv[3]);
    double size = atof(argv[4]);

    FILE *fp = fopen(argv[1], "w");

    for (int i = 0; i < W; i++) {
        for (int j = 0; j < W; j++) {
            double cr = x - size/2 + size*j/W;
            double ci = y + size/2 - size*i/W;
            fprintf(fp, "%d ", mandelbrotSet(cr+I*ci));
        }
        fprintf(fp, "\n");
    }

    fclose(fp);

    return 0;
}
```

```
static int mandelbrotSet(double _Complex c) {
    double _Complex z = 0;
    int n = 0;
    while ((n < N) && (cabs(z) <= 2.0)) {
        z = z*z+c;
        n++;
    }
    return n;
}
```

Pour compiler :

```
> gcc mandelbrot.c -o mandelbrot
```

Le langage C

Le C est un langage de bas niveau (très proche du matériel) qui est très permissif (peu de choses sont interdites).

Avantages : efficacité¹, puissance, flexibilité.

Inconvénients : code souvent sujet aux erreurs (bugs) et parfois difficile à comprendre et à maintenir.

Conseil pour ce cours : éviter les “trucs” de programmation et privilégier la lisibilité à la compacité, voire l'efficacité², du code.

1. Le calcul de la figure du slide 31 prend 3s en C et 103s en Python. cf. Partie 4.

2. à complexité constante, cf. Partie 3

Un mauvais exemple

Que fait ce code ?

```
#include <stdio.h>
main() {
    float C,l,c,o,0,I=-20;char _;for(;I++<20;puts(""))
    for(0=-46;0<14;putchar(_?42:32),0++)for(C=l=_=0;o
    =l*l,c=C*C,l=2*C*l+I/20,C=c-o+0/20,o+c<4&&++_);
}
```

Source : <https://www.codeproject.com/Articles/2228/Obfuscating-your-Mandelbrot-code>

Un mauvais exemple

[illegible]

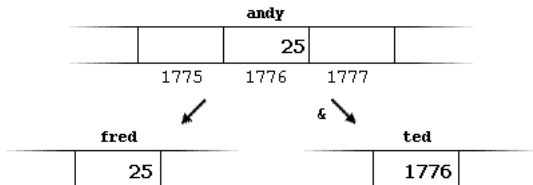
Plan

1. Un tour rapide du C via un exemple
2. Rappel sur les pointeurs

Variables et adresses

- L'identifiant d'une variable correspond à un emplacement mémoire, situé à une certaine adresse, contenant une valeur d'un certain type.
- Un pointeur est une variable dont la valeur est une adresse.
- Le type d'un pointeur est le type de la valeur pointée suivi de * (e.g., `int*` pour un pointeur vers un entier).

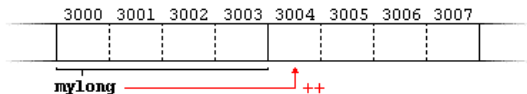
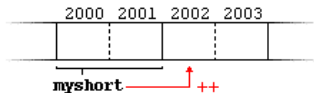
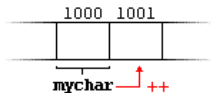
```
int andy = 25;  
int fred = andy;  
int *ted = &andy; // & dénote l'adresse de la variable andy
```



Arithmétique sur les pointeurs

- L'addition et la soustraction sont autorisées sur des pointeurs.
- $p + 1$ correspond à l'emplacement mémoire suivant p , du même type.
- $p + n$ correspond au n -ème emplacement mémoire après p , du même type.

```
char* mychar;  
short* myshort;  
long* mylong;  
mychar = mychar + 1;  
myshort++;  
mylong++;
```



Tableaux et pointeurs

L'identifiant d'un tableau est équivalent à un pointeur pointant vers le premier élément de ce tableau.

```
int a[5];  
int* p;  
  
p = a;  
a[0] = 10;  
*p = 10;           // Ces deux expressions sont équivalentes  
a[2] = 42;  
*(p + 2) = 42;    // Ces deux expressions sont aussi équivalentes
```

Où est le bug ?

```
#include <stdio.h>
int main() {
    int s[4], t[4];
    for (int i = 0; i <= 4; i++) {
        s[i] = t[i] = i;
    }
    printf("i:s:t\n");
    for (int i = 0; i <= 4; i++) {
        printf("%d:%d:%d\n", i, s[i], t[i]);
    }
    return 0;
}
```

Sortie :

```
i:s:t
0:4:0
1:1:1
2:2:2
3:3:3
4:4:4
```

Allocation / désallocation de mémoire

- Un bloc mémoire peut être alloué dynamiquement avec la fonction `malloc`.
- Renvoie `NULL` (`=0`) en cas d'échec, qui représente un pointeur vers rien.
- Tout bloc alloué dynamiquement **doit** être libéré explicitement avec la fonction `free`.

```
int* p = malloc(sizeof(int)); // Alloue un bloc de la taille d'un int
if (!p) {                      // Toujours vérifier le succès de malloc
    printf("Error");
    return 1;
}
free(p);                       // On libère le bloc

int* q = malloc(10 * sizeof(int)); // Alloue un bloc pour 10 int
q[0] = 42;
free(q);
```

Structures et pointeurs

```
typedef struct {  
    double re,im;  
} complex;  
// cette fonction n'a aucun effet sur la donnée  
void test1(complex a) {  
    a.re = a.re + 1.0;  
    a.im = a.im + 1.0;  
}  
// cette fonction modifie la donnée reçue en argument  
void test1(complex *a) {  
    a->re = a->re + 1.0;  
    a->im = a->im + 1.0;  
}
```

- Les structures sont passées (et renvoyées) par **valeurs** aux fonctions. En conséquence :
 - ▶ Les modifications des champs ne sont pas répercutées vers le code appelant.
 - ▶ Les structures sont copiées intégralement lors des appels de fonctions
- On manipule souvent les structures par l'intermédiaire des pointeurs.
- Si p est un pointeur vers une donnée possédant un champ c , alors la notation $p->c$ est une abbréviation pour $(*p).c$.

Une ré-implémentation des complexes par pointeur

complex.c

complex.h

```
typedef struct {  
    double re, im;  
} complex;  
  
complex *complex_new(double, double);  
double complex_real_part(complex *);  
double complex_imgry_part(complex *);  
void complex_sum(complex *, complex *);  
void complex_difference(complex *, complex *);  
void complex_product(complex *, complex *);  
double complex_modulus(complex *);  
double complex_distance(complex *, complex *);
```

```
#include <math.h>  
#include "complex.h"  
  
complex *complex_new(double re, double im) {  
    complex *c = malloc(sizeof(complex));  
    c->re = re;  
    c->im = im;  
    return c;  
}  
  
void complex_sum(complex *a, complex *b) {  
    a->re = a->re + b->re;  
    a->im = a->im + b->im;  
}  
  
void complex_product(complex *a, complex *b) {  
    double tmp_re, tmp_im;  
    tmp_re = a->re * b->re - a->im * b->im;  
    tmp_im = a->re * b->im + a->im * b->re;  
    a->re = tmp_re;  
    a->im = tmp_im;  
}  
  
...
```

Partie 2

Construction et analyse d'algorithmes

14 septembre 2026

Plan

1. Introduction
2. Construction d'algorithmes itératifs
3. Construction d'algorithmes récursifs
4. Analyse de performance

Construction de programme

Pour résoudre un problème de programmation complexe, on le découpe généralement en *sous-problèmes* plus simples à appréhender

- *Exemples de sous-problèmes pour afficher l'ensemble de Mandelbrot :*
 - ▶ *SP1 : calculer le module d'un nombre complexe,*
 - ▶ *SP2 : vérifier l'appartenance d'un point à l'ensemble,*
 - ▶ *SP3 : produire l'image en parcourant le plan complexe.*

Ces problèmes dépendent généralement les uns des autres.

- *Dans l'exemple, SP3 dépend de SP2 qui dépend de SP1.*

Avantages d'un découpage :

- Facilite l'implémentation : on peut résoudre chaque sous-problème indépendamment,
- Généralité : on peut partager du code entre différents programmes,
- Lisibilité et facilité de maintenance du code.

Construction de programmes

Résoudre un sous-problème élémentaire :

- Certains sont triviaux et requièrent d'établir une séquence simple d'instructions (p.ex., calculer le module d'un nombre complexe)
- D'autres sont plus complexes et nécessitent de **répéter** une séquence d'instructions selon un schéma **dépendant des données** (p.ex., déterminer l'appartenance à l'ensemble de Mandelbrot)

Deux grands types de solutions algorithmiques pour ces derniers cas :

- Solutions **itératives**, basées sur des boucles
- Solutions **récurives**, basées sur des fonctions qui s'invoquent elles-mêmes

On va (re)voir dans cette partie quelques grands principes pour la conception de ces deux types de solutions.

Plan

1. Introduction
2. Construction d'algorithmes itératifs
 - Technique de l'invariant
 - Illustrations
3. Construction d'algorithmes récursifs
4. Analyse de performance

Construction d'algorithmes itératifs

Concevoir un algorithme itératif (correct) peut être un exercice très compliqué, surtout si on cherche une solution **efficace**.

Deux difficultés principales :

- Imaginer le **schéma itératif** permettant de résoudre le problème.
- Générer le **code** implémentant ce schéma en évitant les bugs.

Le premier problème est de loin le plus compliqué et cette compétence s'acquière quasi uniquement via la pratique (cf. Partie 3 et le cours INFO0902 pour des techniques génériques néanmoins).

La **technique de l'invariant** permet d'aborder formellement le second problème une fois le schéma de la boucle imaginé.

Invariant de boucle

Un **invariant de boucle** est une propriété définie sur les variables du programme qui définit précisément ce qui doit être calculé à chaque itération pour arriver au résultat escompté. Il résume l'état courant des calculs.

Identifier l'invariant revient à imaginer le schéma itératif de résolution du problème et est parfois **non trivial**.

Une fois l'invariant établi, implémenter la boucle peut par contre se faire de manière relativement **automatique**.

L'utilisation de l'invariant permet donc d'éviter les erreurs d'implémentation.

Invariant de boucle

Une **assertion** est une relation entre les variables et les données utilisées par le programme qui est vraie à un moment donné lors de l'exécution du programme.

Deux assertions particulières :

- **Pré-condition P** : condition que doivent remplir les entrées valides du programme
- **Post-condition Q** : condition qui exprime que le résultat du programme est celui attendu.

On cherche donc à écrire un programme, noté S , dont l'exécution dans **tous** les cas où P est vraie mène à ce que Q soit **toujours** vraie.

Lorsque c'est le cas, on dira que le **triplet** $\{P\}S\{Q\}$ est **correct**.

Exemple : Si $P = \{x \geq 0\}$ et $Q = \{y^2 = x\}$, le code $S = "y = \text{sqrt}(x);"$ rend le triplet $\{P\}S\{Q\}$ correct.

Invariant de boucle : plus formellement

```
{P}  
INIT  
while (B)  
    CORPS  
FIN  
{Q}
```

```
{P}  
INIT  
{I}  
while (B)  
    {I et B} CORPS {I}  
    {I et non B}  
FIN  
{Q}
```

Dans le cas où le programme nécessite une boucle :

- On met en évidence une assertion particulière I , l'**invariant de boucle**, qui décrit l'état du programme pendant la boucle.
- On détermine ensuite le gardien B et les codes $INIT$, $CORPS$ et FIN tels que les trois triplets suivants soient corrects :
 - ▶ $\{P\} \text{ INIT } \{I\}$
 - ▶ $\{I \text{ et } B\} \text{ CORPS } \{I\}$
 - ▶ $\{I \text{ et non } B\} \text{ FIN } \{Q\}$

(Si on a plusieurs boucles imbriquées, on les traite séparément.)

Ecriture du code sur base de l'invariant

Trois parties de code à écrire **en se basant sur l'invariant** :

1. Initialisation : $\{P\}$ INIT $\{I\}$

- ▶ INIT doit rendre l'invariant vrai avant de rentrer dans la boucle et en partant de la pré-condition.
- ▶ L'invariant identifie les variables nécessaires et comment les initialiser.

2. Maintenance : $\{I \text{ et } B\}$ CORPS $\{I\}$

- ▶ CORPS doit faire **avancer** le problème en maintenant l'invariant en supposant que le gardien soit vrai.
- ▶ L'invariant définit le schéma de la boucle.

3. Terminaison : $\{I \text{ et non } B\}$ FIN $\{Q\}$

- ▶ FIN doit rendre la post-condition vraie en supposant que l'invariant est vérifié et le gardien est faux.
- ▶ L'invariant définit comment finalement résoudre le problème.

Détermination de l'invariant

Pas de recette miracle pour déterminer l'invariant (ou de manière équivalente le schéma d'une boucle).

Quelques trucs néanmoins :

- Enlever une partie de la post-condition
- Remplacer dans la post-condition une constante par une variable
- Combiner les pré-conditions et post-conditions
- Raisonner par induction (voir plus loin)

Dans tous les cas, l'invariant doit faire apparaître toutes les variables du programme.

Pour un même problème, plusieurs invariants (et/ou gardiens de boucles) sont possibles qui mèneront à différentes implémentations de la boucle.

Illustration : sous-tableau de somme maximale

Soit `tab` un tableau d'entiers (négatifs, positifs ou nuls) de longueur `length`. On cherche à écrire une fonction `int max_sum(int tab[], int length)` renvoyant la somme maximale d'une sous-séquence contiguë d'éléments du tableau.

13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7
----	----	-----	----	----	-----	-----	----	----	----	----	----	-----	----	----	---

Illustration : sous-tableau de somme maximale

```
int max_sum(int tab[], int length) {  
  
    int mmax = tab[0];  
    int j = 1;  
  
    while (j < length) {  
        int sum = 0;  
        int i = j;  
        while (i >= 0) {  
            sum += tab[i];  
            if (sum > mmax)  
                mmax = sum;  
            i--;  
        }  
        j++;  
    }  
  
    return mmax;  
}
```

Illustration : sous-tableau de somme maximale

```
int max_sum(int tab[], int length) {  
  
    int m = tab[0];  
    int mmax = tab[0];  
    int j = 1;  
  
    while (j < length) {  
        if (m <= 0)  
            m = tab[j];  
        else  
            m += tab[j];  
  
        if (m > mmax)  
            mmax = m;  
        j++;  
    }  
  
    return mmax;  
}
```

Illustration 1 : appartenance à l'ensemble de Mandelbrot

Pré et post-conditions :

- $P = \{cr \in \mathbb{R}, ci \in \mathbb{R}\}$
- $Q = \{r = 1 \text{ si } \forall n, 0 \leq n \leq N : |z_n| \leq 2, 0 \text{ sinon}\}$

où cr et ci sont les entrées du programme, r le résultat, N une constante, et z_n est la n -ème valeur de la suite définie précédemment avec $c \in \mathbb{C}$ tel que $c = cr + ici$.

Schéma de la boucle :

- On calcule z_n pour des valeurs croissantes de n allant de 0 à N .
- A chaque itération, on calcule z_n à partir de la valeur z_{n-1} calculée à l'itération précédente.
- On s'arrête dès que $|z_n| > 2$.

Illustration 1 : appartenance à l'ensemble de Mandelbrot

Invariant :

Illustration 1 : appartenance à l'ensemble de Mandelbrot

Invariant :

$$I = \{(\forall n' : 0 \leq n' < n : |z_{n'}| \leq 2) \text{ et } (z_r + iz_i = z_n) \text{ et } (0 \leq n \leq N)\},$$

où n est le compteur de boucle et z_r et z_i sont deux variables qui contiendront les résultats intermédiaires.

$$\overbrace{z_0, z_1, \dots, z_{n-1}}^{|\dots| \leq 2}, \underbrace{\overbrace{z_n, z_{n+1}, \dots, z_N}^{|\dots| ?}}_{z_r + iz_i = z_n}$$

Gardien :

Illustration 1 : appartenance à l'ensemble de Mandelbrot

Invariant :

$$I = \{(\forall n' : 0 \leq n' < n : |z_{n'}| \leq 2) \text{ et } (z_r + iz_i = z_n) \text{ et } (0 \leq n \leq N)\},$$

où n est le compteur de boucle et z_r et z_i sont deux variables qui contiendront les résultats intermédiaires.

$$\overbrace{z_0, z_1, \dots, z_{n-1}}^{|\dots| \leq 2}, \underbrace{z_n}_{z_r + iz_i = z_n}, \overbrace{z_{n+1}, \dots, z_N}^{|\dots| ?}$$

Gardien :

$$B = \{n < N, z_r^2 + z_i^2 \leq 4\}.$$

Illustration 1 : appartenance à l'ensemble de Mandelbrot

$$\overbrace{z_0, z_1, \dots, z_{n-1}}^{|\dots| \leq 2}, \underbrace{z_n}_{z_r + iz_i = z_n}, \overbrace{z_{n+1}, \dots, z_N}^{|\dots| ?}$$

```
{P}                                     {cr ∈ ℝ, ci ∈ ℝ}
INIT
{I}
while((n < N) && (zr*zr + zi*zi <= 4.0)) {
    {I et B}
    CORPS
    {I}
}
{I et non B}
FIN
{Q}                                     {r = 1 si ∃n : 0 ≤ n ≤ N : |zn| > 2, 0 sinon}
```

Illustration 1 : appartenance à l'ensemble de Mandelbrot

$$\overbrace{z_0, z_1, \dots, z_{n-1}}^{|\dots| \leq 2}, \underbrace{z_n}_{z_r + iz_i = z_n}, \overbrace{z_{n+1}, \dots, z_N}^{|\dots| ?}$$

```
{P}                                                                 {cr ∈ ℝ, ci ∈ ℝ}
double zr = 0;
double zi = 0;
int n = 0;
{I}
while((n < N) && (zr*zr + zi*zi <= 4.0)) {
    {I et B}
    CORPS
    {I}
}
{I et non B}
FIN
{Q}                                                                 {r = 1 si ∃n : 0 ≤ n ≤ N : |zn| > 2, 0 sinon}
```

Illustration 1 : appartenance à l'ensemble de Mandelbrot

$$\overbrace{z_0, z_1, \dots, z_{n-1}}^{|\dots| \leq 2}, \underbrace{z_n}_{z_r + iz_i = z_n}, \overbrace{z_{n+1}, \dots, z_N}^{|\dots| ?}$$

```
{P}                                                                 {cr ∈ ℝ. ci ∈ ℝ}
double zr = 0;
double zi = 0;
int n = 0;
{I}
while((n < N) && (zr*zr + zi*zi <= 4.0)) {
    {I et B}
    double temp;
    temp = zr*zr - zi*zi + cr;
    zi = 2*zr*zi + ci;
    zr = temp;
    n++;
    {I}
}
{I et non B}
FIN
{Q}                                                                 {r = 1 si ∃n : 0 ≤ n ≤ N : |zn| > 2, 0 sinon}
```

Illustration 1 : appartenance à l'ensemble de Mandelbrot

$$\overbrace{z_0, z_1, \dots, z_{n-1}}^{|\dots| \leq 2}, \underbrace{z_n}_{z_r + iz_i = z_n}, \overbrace{z_{n+1}, \dots, z_N}^{|\dots| ?}$$

```
{P}                                                                 {cr ∈ ℝ. ci ∈ ℝ}
double zr = 0;
double zi = 0;
int n = 0;
{I}
while((n < N) && (zr*zr + zi*zi <= 4.0)) {
    {I et B}
    double temp;
    temp = zr*zr - zi*zi + cr;
    zi = 2*zr*zi + ci;
    zr = temp;
    n++;
}
{I et non B}
int r = (zr*zr+zi*zi <= 4.0);
{Q}                                                                 {r = 1 si ∃n : 0 ≤ n ≤ N : |zn| > 2, 0 sinon}
```

Illustration 1 : appartenance à l'ensemble de Mandelbrot

```
double zr = 0;
double zi = 0;
int n = 0;
while((n < N) && (zr*zr + zi*zi <= 4.0)) {
    double temp;
    temp = zr*zr - zi*zi + cr;
    zi = 2*zr*zi + ci;
    zr = temp;
    n++;
}
int r = (zr*zr+zi*zi <= 4.0);
```


Illustration 2 : tri par insertion

On souhaite écrire une fonction pour trier un tableau A de valeurs entières. Le tri doit être effectué dans le tableau lui-même, via échanges d'éléments.

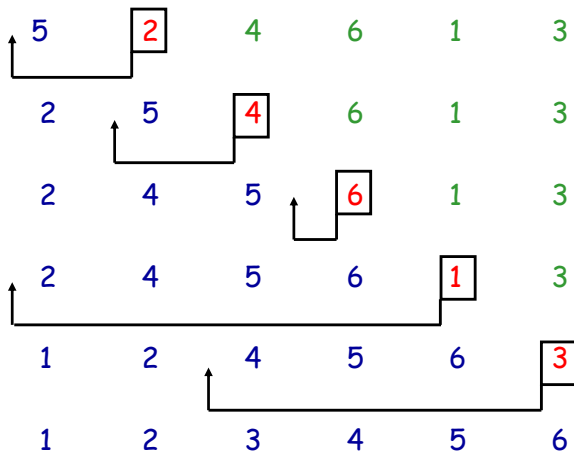
```
void insertion_sort(int A[], int N);
```

Principe de l'algorithme :

- On parcourt le tableau de gauche à droite en triant successivement les préfixes du tableau de tailles 2, 3, ..., N .
- A chaque itération, on augmente la taille du préfixe trié en insérant le nouvel élément $A[i]$ à sa position dans le sous-tableau $A[0..i-1]$ précédemment ordonné.



Tri par insertion : graphiquement



Tri par insertion : boucle externe

Pré-condition P_e : A est un tableau d'entiers de taille N .

Post-condition Q_e : A contient les éléments du tableau de départ triés par ordre croissant.

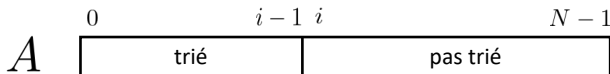
Invariant I_e (de la boucle externe) :

Tri par insertion : boucle externe

Pré-condition P_e : A est un tableau d'entiers de taille N .

Post-condition Q_e : A contient les éléments du tableau de départ triés par ordre croissant.

Invariant I_e (de la boucle externe) : Le sous-tableau $A[0..i-1]$, avec $1 \leq i \leq N$, contient les i premiers éléments du tableau initial triés par ordre croissant.



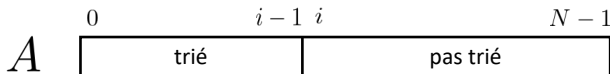
Gardien B_e :

Tri par insertion : boucle externe

Pré-condition P_e : A est un tableau d'entiers de taille N .

Post-condition Q_e : A contient les éléments du tableau de départ triés par ordre croissant.

Invariant I_e (de la boucle externe) : Le sous-tableau $A[0 \dots i - 1]$, avec $1 \leq i \leq N$, contient les i premiers éléments du tableau initial triés par ordre croissant.



Gardien B_e : $i < N$.

Tri par insertion : boucle externe

$\{P_e\}$	$\{A \text{ est un tableau d'entiers de taille } N\}$
INITe	
$\{I_e\}$	$\{A[0..i-1] \text{ trié}\}$
while ($i < N$) {	
$\{I_e \text{ et } B_e\}$	$\{A[0..i-1] \text{ trié et } i < N\}$
CORPSe	
$\{I_e\}$	$\{A[0..i-1] \text{ trié}\}$
}	
$\{I_e \text{ et non } B\}$	$\{A[0..i-1] \text{ trié et } i = N\}$
FINe	
$\{Q_e\}$	$\{A \text{ contient les éléments du tableau de départ triés}\}$

Tri par insertion : boucle externe

$\{P_e\}$	$\{A \text{ est un tableau d'entiers de taille } N\}$
<code>int i = 1;</code>	
$\{I_e\}$	$\{A[0..i-1] \text{ trié}\}$
<code>while (i < N) {</code>	
$\{I_e \text{ et } B_e\}$	$\{A[0..i-1] \text{ trié et } i < N\}$
CORPSe	
$\{I_e\}$	$\{A[0..i-1] \text{ trié}\}$
<code>}</code>	
$\{I_e \text{ et non } B\}$	$\{A[0..i-1] \text{ trié et } i = N\}$
FINe	
$\{Q_e\}$	$\{A \text{ contient les éléments du tableau de départ triés}\}$

Tri par insertion : boucle externe

$\{P_e\}$	$\{A \text{ est un tableau d'entiers de taille } N\}$
<code>int i = 1;</code>	
$\{I_e\}$	$\{A[0..i-1] \text{ trié}\}$
<code>while (i < N) {</code>	
$\{I_e \text{ et } B_e\}$	$\{A[0..i-1] \text{ trié et } i < N\}$
CORPSe	
$\{I_e\}$	$\{A[0..i-1] \text{ trié}\}$
<code>}</code>	
$\{I_e \text{ et non } B\}$	$\{A[0..i-1] \text{ trié et } i = N\}$
<code>-</code>	
$\{Q_e\}$	$\{A \text{ contient les éléments du tableau de départ triés}\}$

Tri par insertion : boucle externe

$\{P_e\}$	$\{A \text{ est un tableau d'entiers de taille } N\}$
<code>int i = 1;</code>	
$\{I_e\}$	$\{A[0..i-1] \text{ trié}\}$
<code>while (i < N) {</code>	
$\{I_e \text{ et } B_e\}$	$\{A[0..i-1] \text{ trié et } i < N\}$
CORPSe'	
$\{Q_i\}$	$\{A[0..i] \text{ trié}\}$
<code>i++;</code>	
$\{I_e\}$	$\{A[0..i-1] \text{ trié}\}$
<code>}</code>	
$\{I_e \text{ et non } B\}$	$\{A[0..i-1] \text{ trié et } i = N\}$
<code>-</code>	
$\{Q_e\}$	$\{A \text{ contient les éléments du tableau de départ triés}\}$

Tri par insertion : boucle interne (CORPSe')

Pré-condition P_i : $\{I_e \text{ et } B_e\} = \{A[0..i-1] \text{ trié et } i < N\}$

Post-condition Q_i : $I_e = \{A[0..i] \text{ trié}\}$

Idée de la boucle : Soit $key = A[i]$, la valeur à déplacer :

- On parcourt le sous-tableau $A[0..i-1]$ de droite à gauche.
- Tant que les éléments parcourus sont supérieurs à key , on les décale d'une position vers la droite.
- On insère key à la position finalement atteinte.

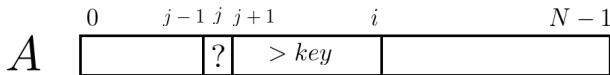
Tri par insertion : boucle interne

Invariant I_i :

Tri par insertion : boucle interne

Invariant I_j : Soit j un nouvel indice et $key = A[j]$ la valeur à insérer :

- $A[0..j-1]$ et $A[j+1..i]$ sont triés par ordre croissant et ensemble contiennent tous les éléments du sous-tableau $A[0..i]$ initial, excepté key .
- $key < A[j+1]$

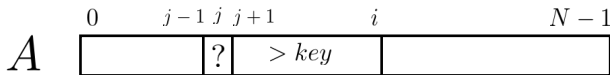


Gardien B_j :

Tri par insertion : boucle interne

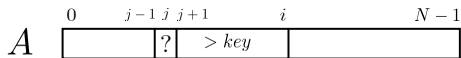
Invariant I_j : Soit j un nouvel indice et $key = A[j]$ la valeur à insérer :

- $A[0..j-1]$ et $A[j+1..i]$ sont triés par ordre croissant et ensemble contiennent tous les éléments du sous-tableau $A[0..i]$ initial, excepté key .
- $key < A[j+1]$



Gardien B_j : $\{j > 0 \text{ et } A[j-1] > key\}$

Tri par insertion : boucle interne

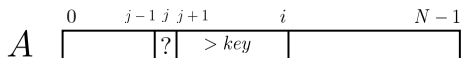


```
{ $P_i$ }
INITi
{ $I_i$ }
while ( $j > 0$  &&  $A[j-1] > key$ ) {
    { $I_i$  et  $B_i$ }
    CORPSi
    { $I_i$ }
}
{ $I_i$  et non  $B_i$ }
FINi
{ $Q_i$ }
```

$\{A[0 \dots i-1] \text{ trié et } i < N\}$

$\{A[0 \dots i-1] \text{ trié}\}$

Tri par insertion : boucle interne



$\{P_i\}$ $\{A[0..i-1] \text{ trié et } i < N\}$

```
int key = A[i];
```

```
int j = i;
```

$\{I_i\}$

```
while (j > 0 && A[j-1] > key) {
```

$\{I_i \text{ et } B_i\}$

CORPS_i

$\{I_i\}$

```
}
```

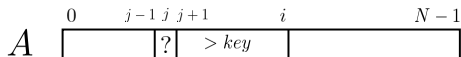
$\{I_i \text{ et non } B_i\}$

FINI_i

$\{Q_i\}$

$\{A[0..i-1] \text{ trié}\}$

Tri par insertion : boucle interne



```
{ $P_i$ }                                     { $A[0 \dots i-1]$  trié et  $i < N$ }
```

```
int key = A[i];
```

```
int j = i;
```

```
{ $I_i$ }
```

```
while (j > 0 && A[j-1] > key) {
```

```
    { $I_i$  et  $B_i$ }
```

```
    A[j] = A[j-1];
```

```
    j--;
```

```
    { $I_i$ }
```

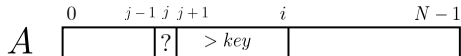
```
}
```

```
{ $I_i$  et non  $B_i$ }
```

```
FINi
```

```
{ $Q_i$ }                                     { $A[0 \dots i-1]$  trié}
```


Tri par insertion : boucle interne



```
{ $P_i$ }                                     { $A[0..i-1]$  trié et  $i < N$ }  
int key = A[i];  
int j = i;  
{ $I_i$ }  
while (j > 0 && A[j-1] > key) {  
    { $I_i$  et  $B_i$ }  
    A[j] = A[j-1];  
    j--;  
    { $I_i$ }  
}  
{ $I_i$  et non  $B_i$ }  
A[j] = key;  
i++;  
{ $Q_i$ }                                     { $A[0..i-1]$  trié}
```

Tri par insertion : code complet

```
void insertion_sort(int A[], int N) {  
    int i = 1;  
    while (i < N) {  
        int key = A[i];  
        int j = i;  
        while (j > 0 && A[j-1]>key) {  
            A[j] = A[j-1];  
            j--;  
        }  
        A[j] = key;  
        i++;  
    }  
}
```

Synthèse

- L'expression d'un invariant permet de limiter les erreurs lors de l'implémentation d'une boucle.
- Vous devez prendre l'habitude d'exprimer l'invariant de toutes vos boucles **avant** leur implémentation, au minimum de manière informelle ou graphique.
- Dans la suite du cours, on fournira ponctuellement les invariants des boucles les plus compliquées.

Plan

1. Introduction
2. Construction d'algorithmes itératifs
3. Construction d'algorithmes récursifs
 - Principe
 - Illustrations
4. Analyse de performance

Algorithme récursif

- Un **algorithme** de résolution d'un problème P sur une donnée a est dit **récursif** si parmi les opérations utilisées pour le résoudre, on trouve une résolution du même problème P sur une donnée b .
- Dans un algorithme récursif, on nommera **appel récursif** toute étape résolvant le même problème sur une autre donnée.
- Un algorithme récursif s'implémente généralement via des **fonctions récursives**.
- Forme générale d'une fonction récursive (directe) :

```
type f(P) {  
    ...  
    x = f(Pr);  
    ...  
    return r;  
}
```

Exemple 1 : fonction factorielle

La définition mathématique de la factorielle est récursive :

$$n! = \begin{cases} 1 & \text{si } n \leq 1, \\ n * (n - 1)! & \text{sinon} \end{cases}$$

et se prête donc naturellement à une implémentation via une fonction récursive :

```
int fact(int n) {  
    if (n <= 1)  
        return 1;  
  
    return n * fact(n-1);  
}
```

Exemple 1 : fonction factorielle

Trace des appels de fonctions pour `fact(5)` :

```
fact(5)
  |fact(4)
  |  |fact(3)
  |  |  |fact(2)
  |  |  |  |fact(1)
  |  |  |  |  |return 1
  |  |  |  |  |return 2*1 = 2
  |  |  |  |  |return 3*2 = 6
  |  |  |  |  |return 4*6 = 24
  |  |  |  |  |return 5*24 = 120
```

Exemple 2 : algorithme d'Euclide

L'algorithme du calcul du pgcd d'Euclide est basé sur la propriété récursive suivante :

Soient deux entiers positifs a et b . Si $a > b$, le pgcd de a et b est égal au pgcd de b et de $(a \bmod b)$.

Suggère l'implémentation récursive suivante :

Exemple :

```
int pgcd(int a, int b) {  
    if (b > a)  
        return pgcd(b,a);  
  
    if (b == 0)  
        return a;  
  
    return pgcd(b, a % b);  
}
```

```
pgcd(1440, 408)  
  |pgcd(408, 216)  
  |  |pgcd(216, 192)  
  |  |  |pgcd(192, 24)  
  |  |  |  |pgcd(24,0)  
  |  |  |  |return 24  
  |  |  |return 24  
  |  |return 24  
  |return 24  
return 24
```


Conception de fonctions récursives

Deux conditions pour qu'une fonction récursive soit bien définie :

- Présence d'un **cas de base** (condition d'arrêt)
- La "taille" du problème doit être **réduite** à chaque étape

Forme générale d'une fonction récursive :

```
type f(P) {  
  if (cas_de_base) { // cas de base  
    ... // pas d'appel récursif  
    return [...];  
  }  
  
  // étape de réduction  
  ...  
  [x =] f(Pr); // avec Pr plus "simple" que P  
  ...  
  [return r;]  
}
```

Conception de fonctions récursives

D'autres formes peuvent néanmoins être valides (mais sont plutôt à éviter).

Récursivité non décroissante

(Suite de Syracuse)

```
int Syracuse(int n) {  
    if (n == 1)  
        return 1;  
  
    if (n % 2) // n est impair  
        return Syracuse(3*n+1);  
    else      // n est pair  
        return Syracuse(n/2);  
}
```

Récursivité imbriquée

(fonction d'Ackermann)

```
int ack(int m, int n) {  
    if (m==0)  
        return n+1;  
    else {  
        if (n==0)  
            return ack(m-1,1);  
        else  
            return ack(m-1,ack(m,n-1));  
    }  
}
```

Récursivité croisée

```
int P(int n) {  
    if (n==0)  
        return 1;  
    else  
        return I(n-1);  
}
```

```
int I(int n) {  
    if (n==0)  
        return 0;  
    else  
        return P(n-1);  
}
```

Que calcule la fonction P ?

Principe de construction d'un algorithme récursif

- Trouver un ou plusieurs paramètres de **taille** sur lesquels construire la récursion

Factorielle : n , le nombre dont on veut calculer la factorielle.

- Trouver une solution pour **le(s) cas de base**, c'est-à-dire des problèmes de petites tailles (la plupart du temps trivial).

Factorielle : $n! = 1$ si $n \leq 1$.

- Trouver comment **réduire** le problème à un ou plusieurs sous-problème de tailles strictement plus petites.

Factorielle : $n! = n * (n - 1)!$ si $n > 1$

La dernière étape est la plus délicate. Equivalent à trouver l'invariant dans le cas d'algorithmes itératifs.

Fonction puissance : solution itérative

On souhaite calculer a^x , avec $a \in \mathbb{R}$ et x entier ≥ 1 .

Version itérative

```
float pow_iter(float a, int x) {  
    float res = a;  
    for (int i = 1; i < x; i++)  
        res = res * a;  
    return res;  
}
```

(Invariant ?)

Nombre de multiplications nécessaires : $x - 1$

Une première formulation récursive :

$$a^x = \begin{cases} a & \text{si } x = 1 \\ a \cdot a^{x-1} & \text{si } x > 1 \end{cases}$$

```
float pow_rec1(float a, int x) {  
    if (x == 1)  
        return a;  
    return a*pow_rec1(a, x-1);  
}
```

Nombre de multiplications nécessaires : $x - 1$

Cette version est une simple réécriture de la version itérative.

Peut-on faire mieux ?

Fonction puissance : récursivement

(2/2)

Une deuxième formulation récursive :

$$a^x = \begin{cases} a & \text{si } x = 1 \\ (a \cdot a)^{x/2} & \text{si } x > 1 \text{ et pair} \\ a \cdot (a \cdot a)^{(x-1)/2} & \text{si } x > 1 \text{ et impair} \end{cases}$$

Fonction puissance : récursivement

(2/2)

Une deuxième formulation récursive :

$$a^x = \begin{cases} a & \text{si } x = 1 \\ (a \cdot a)^{x/2} & \text{si } x > 1 \text{ et pair} \\ a \cdot (a \cdot a)^{(x-1)/2} & \text{si } x > 1 \text{ et impair} \end{cases}$$

```
float pow_rec2(float a, int x) {  
    if (x == 1)  
        return a;  
    if (x % 2 == 0) // x pair  
        return pow_rec2(a * a, x/2);  
    else           // x impair  
        return a * pow_rec2(a * a, (x-1)/2);  
}
```

Fonction puissance : récursivement

(2/2)

Une deuxième formulation récursive :

$$a^x = \begin{cases} a & \text{si } x = 1 \\ (a \cdot a)^{x/2} & \text{si } x > 1 \text{ et pair} \\ a \cdot (a \cdot a)^{(x-1)/2} & \text{si } x > 1 \text{ et impair} \end{cases}$$

```
float pow_rec2(float a, int x) {  
    if (x == 1)  
        return a;  
    if (x % 2 == 0) // x pair  
        return pow_rec2(a * a, x/2);  
    else // x impair  
        return a * pow_rec2(a * a, (x-1)/2);  
}
```

Nombre de multiplications nécessaires : entre $\log_2(x)$ et $2 \log_2(x)$

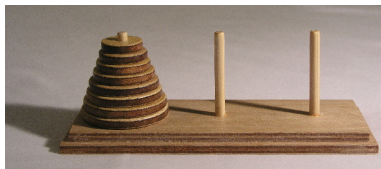
- $x = 128 \Rightarrow 7$ multiplications au lieu de 127.

Fonction puissance : remarque

La solution précédente peut aussi s'implémenter de manière itérative.

Mais l'invariant de la boucle est obtenu sur base de la même formulation récursive et l'implémentation serait (un peu) moins immédiate.

Il existe des problèmes pour lesquels une solution itérative serait nettement plus complexe à implémenter qu'une solution récursive.



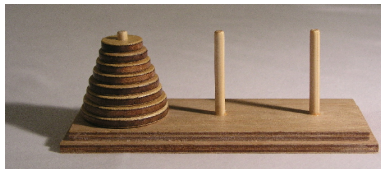
Source : wikipedia

Objectif : transférer les n disques de la tour 1 à la tour 2, sans jamais mettre un disque sur un plus petit.

Solution récursive : Pour déplacer n disques :

- On déplace les $n - 1$ disques du dessus vers la tour 3.
- On déplace le n -ème disque de la tour 1 vers la tour 2.
- On déplace les $n - 1$ disques de la tour 3 vers la tour 2.

Solution itérative ?



Source : wikipedia

En fait, une solution itérative simple existe :

Tant que la pile n'est pas dans sa position finale :

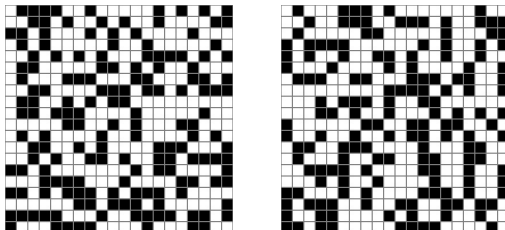
- *Bouger le plus petit disque à droite ($1 \rightarrow 2$, $2 \rightarrow 3$ ou $3 \rightarrow 1$)*
- *Bouger le seul autre disque qui peut être bougé*

Cette solution bien que strictement identique à la solution récursive est beaucoup plus compliquée à concevoir à partir de rien.

Son implémentation est aussi plus complexe car elle demande de représenter **explicitement** l'état des trois tours à chaque itération.

Percolation

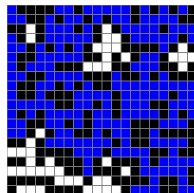
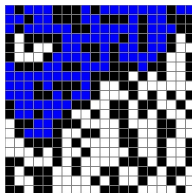
(Sedgewick et Wayne, 2016)



Problème : déterminer si de l'eau peut s'écouler dans un matériau poreux.

- Entrée : un tableau `grid` de taille $n \times n$, tel que `grid[i][j] = 0` si le matériau est vide à la position (i, j) , 1 si le matériau est plein.
- Sortie : vrai s'il existe un chemin partant de la première ligne, arrivant à la dernière ligne de la matrice et passant uniquement par des cases vides, faux sinon.

Percolation : découpage en sous-problème



Idée de la solution :

- Marquer chaque case de la grille pour laquelle il existe un chemin depuis une case vide de la première ligne.
- Vérifier qu'une case de la dernière ligne au moins a été marquée.

```
// Valeur 2 utilisée comme marquage
int percolate(int **grid, int n) {
    flow(grid,n); // fonction qui effectue le marquage des cases
    for (int j = 0; j < n; j++)
        if (grid[n-1][j] == 2)
            return 1;
    return 0;
}
```

Fonction flow : découpage en sous-problèmes

On définit une fonction `flowrec` :

```
void flowrec(int **grid, int n, int i, int j);
```

qui va marquer toutes les cases accessibles à partir de la position (i, j) .

La fonction `flow` consiste alors simplement à appliquer `flowrec` à toutes les cases de la première ligne du tableau.

```
void flow(int **grid, int n) {  
    for (int j = 0; j < n; j++)  
        flowrec(grid, n, 0, j);  
}
```

Une solution récursive pour flowrec

```
void flowrec(int **grid, int n, int i, int j);
```

Cas de base :

Une solution récursive pour flowrec

```
void flowrec(int **grid, int n, int i, int j);
```

Cas de base : Ne rien faire si

- $i < 0$, $i \geq n$, $j < 0$, ou $j \geq n$,
(la case est hors de la grille)

Une solution récursive pour flowrec

```
void flowrec(int **grid, int n, int i, int j);
```

Cas de base : Ne rien faire si

- $i < 0$, $i \geq n$, $j < 0$, ou $j \geq n$,
(la case est hors de la grille)
- $\text{grid}[i][j] == 1$,
(la case est pleine)

Une solution récursive pour flowrec

```
void flowrec(int **grid, int n, int i, int j);
```

Cas de base : Ne rien faire si

- $i < 0$, $i \geq n$, $j < 0$, ou $j \geq n$,
(la case est hors de la grille)
- $\text{grid}[i][j] == 1$,
(la case est pleine)
- $\text{grid}[i][j] == 2$
(la case a déjà été visitée lors d'un précédent appel récursif).

Cas inductif : Si on n'est pas dans un cas de base :

Une solution récursive pour flowrec

```
void flowrec(int **grid, int n, int i, int j);
```

Cas de base : Ne rien faire si

- $i < 0$, $i \geq n$, $j < 0$, ou $j \geq n$,
(la case est hors de la grille)
- $\text{grid}[i][j] == 1$,
(la case est pleine)
- $\text{grid}[i][j] == 2$
(la case a déjà été visitée lors d'un précédent appel récursif).

Cas inductif : Si on n'est pas dans un cas de base :

- On marque la case (i, j) qui est accessible.
- On applique récursivement la fonction `flowrec` à toutes les cases adjacentes à la case (i, j) .

Une solution récursive pour flowrec

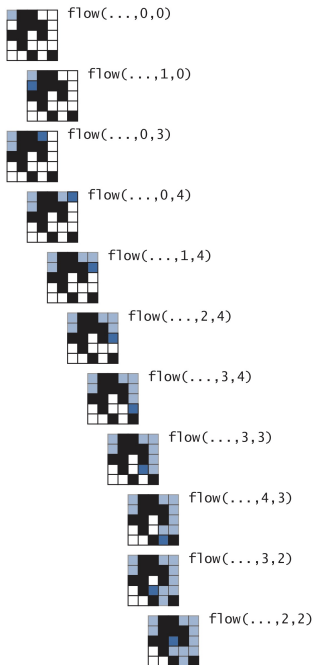
```
void flow(int **grid, int n) {
    for (int j = 0; j < n; j++)
        flowrec(grid, n, 0, j);
}

void flowrec(int **grid, int n, int i, int j) {
    // Cas de base
    if (i < 0 || i >= n || j < 0 || j >= n) return;
    if (grid[i][j] == 1 || grid[i][j] == 2) return;
    // Cas inductif
    grid[i][j] = 2;
    flowrec(grid, n, i+1, j); // Bas
    flowrec(grid, n, i, j+1); // Droite
    flowrec(grid, n, i, j-1); // Gauche
    flowrec(grid, n, i-1, j); // Haut
}
```

Correction ? Complexité ?

Note : flowrec implémente ce qu'on appelle une recherche en profondeur d'abord (*depth-first search*).

Illustration

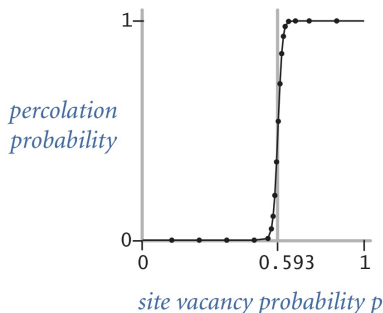


(Sedgewick et Wayne, 2016)

Application : simulations numériques

Cette fonction permet par exemple d'étudier l'impact de la porosité du matériau sur la probabilité de percolation.

Pour des grilles de taille 100x100 (10000 grilles aléatoires générées par point) :



(Sedgewick et Wayne, 2016)

Synthèse

- Principal intérêt de la récursivité : élégance, simplicité, et lisibilité du code.
- Moins sujet à des bugs et analyse de correction facilitée par rapport aux boucles.
- Beaucoup d'algorithmes efficaces sont basés sur la récursivité (cf. Partie 3 et INFO0902).
- Mais la récursivité mène parfois à des solutions très inefficaces, en temps de calcul et en espace (voir section suivante)

Plan

1. Introduction
2. Construction d'algorithmes itératifs
3. Construction d'algorithmes récursifs
4. Analyse de performance
 - Introduction
 - Approche empirique
 - Approche mathématique
 - Complexité en espace

Pourquoi se soucier des performances ?

L'intérêt pratique d'un programme est très souvent dicté par deux questions :

- Est-il possible d'obtenir les résultats voulus en un temps raisonnable ?
- Peut-on traiter des volumes de données suffisamment importants ?

⇒ La performance d'un algorithme est généralement mesurée par son coût en terme de **temps de calcul** et d'**espace mémoire consommé**.

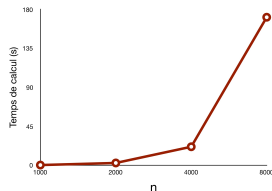
Étudier la performance d'un programme permet :

- de **prédire** son comportement
 - ▶ Est-ce que mon programme va se terminer ? Après combien de temps ?
- de **comparer** différents algorithmes et implémentations
 - ▶ Puis-je rendre mon programme plus rapide ? Si oui, comment ?

Analyse de performance

Pour analyser les performances d'un programme, on adopte une approche **scientifique** classique basée soit sur :

- **l'expérimentation** : on mesure les temps de calcul dans des conditions réelles
- **la modélisation mathématique** : on dérive une formule mathématique liant les performances aux données d'entrée.



$$T(n) = O(n^3)$$

Contrairement à d'autres sciences (chimie, biologie, physique, sociologie...), en sciences informatiques :

- les expérimentations sont quasi gratuites
- la modélisation mathématique est nettement plus aisée

Illustration : problème 3SUM

Le problème 3SUM :

Étant donné n nombres entiers, énumérer tous les triplets de valeurs sommant à 0.

Trouve des applications théoriques et pratiques (en géométrie et cryptographie).

Illustration : problème 3SUM

Le problème 3SUM :

Étant donné n nombres entiers, énumérer tous les triplets de valeurs sommant à 0.

Trouve des applications théoriques et pratiques (en géométrie et cryptographie).

Solution naïve (pour le comptage) : on énumère tous les triplets de valeurs et on teste leur somme.

```
int count_3sum(int tab[], int n) {  
    int count = 0;  
    for (int i = 0; i < n-2; i++)  
        for (int j = i+1; j < n-1; j++)  
            for (int k = j+1; k < n; k++)  
                if (tab[i]+tab[j]+tab[k] == 0)  
                    count++;  
    return count;  
}
```

Combien de temps prendra ce programme pour un tableau d'un million de valeurs ?

Approche empirique

Principe : On implémente l'algorithme, on l'exécute et on mesure ses performances.

Il faut trouver des données **représentatives** sur lesquels tester l'algorithme. Deux options :

- On **collecte** des données réelles
- On écrit un programme pour **générer** des données

Exemple : un générateur de données pour le problème 3SUM

```
void generate_3sum_data(int tab[], int n, int m) {  
    for (int i = 0; i < n; i++)  
        tab[i] = rand() % (2*m) - m;  
}
```

Génère un tableau aléatoire de n nombres entiers pris dans $\{-m, \dots, m-1\}$.
La probabilité de trouver des triplets sommant à zéro dépend de n et m .

Comment mesurer les temps de calcul ?

Pour mesurer le temps d'exécution d'un programme, on peut utiliser les fonctions de `time.h`

```
#include <time.h>

....
clock_t begin = clock();

// The code you want to monitor should be here

clock_t end = clock();

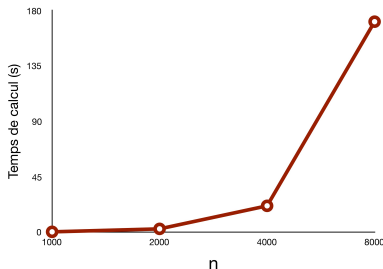
double time_spent = (double)(end-begin) / (double)CLOCKS_PER_SEC;
```

La variable `time_spent` contient le temps (en secondes) pris par le code entre les appels à `clock()`.

Illustration sur 3SUM

On double la taille du tableau d'un essai à l'autre avec des entiers compris entre -1000000 et +999999.

n	temps (s)
1000	0.34
2000	2.75
4000	21.45
8000	170.9



(Sur un Intel Core i7 2.5 GHz)

⇒ Le temps de calcul est $\approx \frac{1}{3}10^{-9}n^3$.

Prédiction : pour $n = 10^6$, la fonction prendra $333 \cdot 10^6$ s (> 10 ans)

Avantages et limitations de l'approche empirique

Avantages :

- Expériences très faciles à réaliser.
- On mesure les temps de calcul de l'implémentation réelle.

Limitations :

- Demande d'implémenter l'algorithme (pour peut-être se rendre compte qu'il est inefficace).
- Temps de calcul dépendent de l'implémentation et de la machine, même à solution algorithmique fixée.
- Ne fournit pas une preuve formelle de l'évolution des temps de calcul avec n . Tirer des conclusions à partir de valeurs expérimentales peut mener à des erreurs.

Approche mathématique

Principe :

- On fait des hypothèses sur le **modèle** d'exécution du programme
 - ▶ Exécution séquentielle (pas de parallélisme)
 - ▶ Les instructions élémentaires prennent un temps constant
 - ▶ ...
- On compte le nombre de fois que chaque instruction est exécutée.
- On obtient les temps de calcul en sommant le temps d'exécution (constant) de chaque instruction multiplié par son nombre d'exécutions.

Le temps dépend en général des données d'entrée :

- On exprime le temps de calcul en fonction de la **taille des données d'entrée**.
- Si les nombres d'exécutions des instructions dépendent de l'entrée, on se place dans le **cas le plus défavorable** (*worst case*).

Illustration : 2SUM

```
1 int count_2sum(int tab[], int n) {  
2     int count = 0;  
3     for (int i = 0; i < n-1; i++)  
4         for (int j = i+1; j < n; j++)  
5             if (tab[i] + tab[j] == 0)  
6                 count++;  
7     return count;  
8 }
```

Ligne	Coût	Nombre d'exécutions
2	c_1	1
3	c_2	n
4	c_2	$\sum_{i=0}^{n-2} (n-i)$
5	c_3	$\sum_{i=0}^{n-2} (n-i-1)$
6	c_4	n_4
7	c_5	1

Sachant que

- $\sum_{i=0}^{n-2} (n-i) = \frac{n(n+1)}{2} - 1$
- $\sum_{i=0}^{n-2} (n-i-1) = \frac{(n-1)n}{2}$ (le nombre de paires à tester)
- n_4 dépend du tableau mais vaut **au pire cas** $\frac{(n-1)n}{2}$ (toutes les paires somment à 0)

on a :

$$T(n) = \frac{c_2 + c_3 + c_4}{2} n^2 + \frac{3c_2 - c_3 - c_4}{2} n + (c_1 - c_2 + c_5)$$

Illustration : 2SUM

$$T(n) = an^2 + bn + c$$

Les constantes a , b , c et d dépendent :

- de l'implémentation exacte (via les nombres d'exécutions)
- de la machine (via les constantes c_i)

Idéalement, on aimerait caractériser les performances d'un algorithme **indépendamment de l'implémentation et de la machine**.

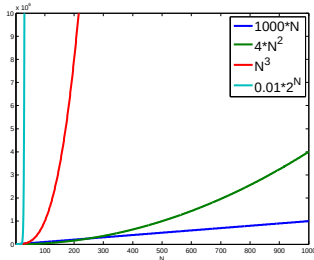
Solution : on se focalise sur la **vitesse de croissance asymptotique** des temps de calcul.

Analyse asymptotique

- On s'intéresse à la vitesse de croissance ("order of growth") de $T(n)$ lorsque n est très grand ($n \rightarrow \infty$).
 - ▶ Tous les algorithmes sont rapides pour des petites valeurs de n

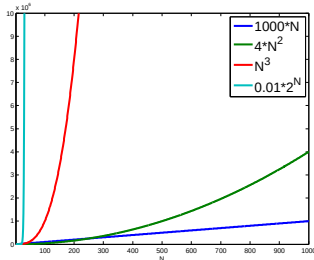
Analyse asymptotique

- On s'intéresse à la vitesse de croissance ("order of growth") de $T(n)$ lorsque n est très grand ($n \rightarrow \infty$).
 - ▶ Tous les algorithmes sont rapides pour des petites valeurs de n
- On simplifie généralement $T(n)$:
 - ▶ en ne gardant que le **terme dominant**
 - ▶ Exemple : $T(n) = 10n^3 + n^2 + 40n + 800$
 - ▶ $T(1000) = 100001040800$, $10 \cdot 1000^3 = 100000000000$
 - ▶ en **ignorant le coefficient** du terme dominant
 - ▶ Asymptotiquement, ça n'affecte pas l'ordre relatif



Analyse asymptotique

- On s'intéresse à la vitesse de croissance ("order of growth") de $T(n)$ lorsque n est très grand ($n \rightarrow \infty$).
 - ▶ Tous les algorithmes sont rapides pour des petites valeurs de n
- On simplifie généralement $T(n)$:
 - ▶ en ne gardant que le **terme dominant**
 - ▶ Exemple : $T(n) = 10n^3 + n^2 + 40n + 800$
 - ▶ $T(1000) = 100001040800$, $10 \cdot 1000^3 = 100000000000$
 - ▶ en **ignorant le coefficient** du terme dominant
 - ▶ Asymptotiquement, ça n'affecte pas l'ordre relatif



- Exemple : 2SUM : $T(n) = an^2 + bn + c \rightarrow n^2$.

Notation asymptotique “grand-O”

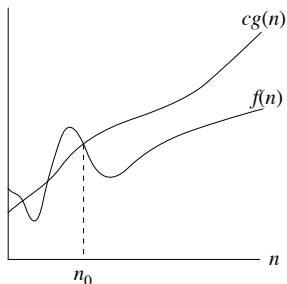
Les vitesses de croissance sont généralement précisées en utilisant la notation asymptotique “grand-O” (ou encore notation de [Landau](#)).

Définition : Soient f et g deux fonctions $\mathbb{N} \rightarrow \mathbb{R}^+$. On dira que

$$f \in O(g)$$

ssi

$$\exists n_0 \in \mathbb{N}, c \in \mathbb{R}^+ : \forall n > n_0 : f(n) \leq cg(n)$$



Par abus de notation, on écrira aussi : $f(n) \in O(g(n))$ ou $f(n) = O(g(n))$.

Complexité en temps

On dira qu'un algorithme a une **complexité en temps** $O(f(n))$ (ou plus simplement est $O(f(n))$) si ses temps de calcul **dans le pire cas** $g(n) \in O(f(n))$.

- Exemple : La complexité de `count_2sum` est $O(n^2)$.

Remarque importante : La notation grand- O sert à exprimer une **borne supérieure** sur la complexité.

- Généralement, quand on dit qu'un algorithme est $O(f(n))$, on suppose que $O(f(n))$ est **le plus petit sous-ensemble** contenant la fonction $g(n)$ exprimant les temps de calcul de l'algorithme **dans le pire cas**.
- Par ex. : on ne dira pas que `count_2sum` est $O(n^3)$ même si $O(n^2) \subset O(n^3)$.

Exemples :

- $n^2 + 2n + 2$
- $n^2 + 100000n + 3^{1000}$
- $\log(n) + n + 4$
- $10^{-4}n \log(n) + 3000n$
- $2n^{30} + 3^n$

Complexité en temps

On dira qu'un algorithme a une **complexité en temps** $O(f(n))$ (ou plus simplement est $O(f(n))$) si ses temps de calcul **dans le pire cas** $g(n) \in O(f(n))$.

- Exemple : La complexité de `count_2sum` est $O(n^2)$.

Remarque importante : La notation grand- O sert à exprimer une **borne supérieure** sur la complexité.

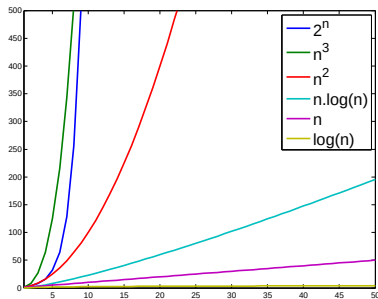
- Généralement, quand on dit qu'un algorithme est $O(f(n))$, on suppose que $O(f(n))$ est **le plus petit sous-ensemble** contenant la fonction $g(n)$ exprimant les temps de calcul de l'algorithme **dans le pire cas**.
- Par ex. : on ne dira pas que `count_2sum` est $O(n^3)$ même si $O(n^2) \subset O(n^3)$.

Exemples :

- | | |
|---|---|
| ■ $n^2 + 2n + 2 \Rightarrow O(n^2)$ | ■ $10^{-4}n \log(n) + 3000n \Rightarrow O(n \log(n))$ |
| ■ $n^2 + 100000n + 3^{1000} \Rightarrow O(n^2)$ | |
| ■ $\log(n) + n + 4 \Rightarrow O(n)$ | ■ $2n^{30} + 3^n \Rightarrow O(3^n)$ |

Hiérarchie de classes de complexité

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^{a>1}) \subset O(2^n)$$



Analyse de complexité en pratique

En pratique, il n'est (la plupart du temps) pas nécessaire de compter explicitement le nombre d'exécutions de chaque instruction.

On peut calculer la complexité en notation grand- O directement en se basant sur les propriétés suivantes :

- Si $f(n) \in O(g(n))$, alors pour tout $k \in \mathbb{N}$, on a $k \cdot f(n) \in O(g(n))$
 - ▶ Exemple : $\log_a(n) \in O(\log_b(n))$, $a^{n+b} \in O(a^n)$
- Si $f_1(n) \in O(g_1(n))$ et $f_2(n) \in O(g_2(n))$, alors
 $f_1(n) + f_2(n) \in O(g_1(n) + g_2(n))$ et $f_1(n) + f_2(n) \in O(\max\{g_1(n), g_2(n)\})$
 - ▶ Exemple : $\sum_{i=1}^m a_i n^i \in O(n^m)$
- Si $f_1(n) \in O(g_1(n))$ et $f_2(n) \in O(g_2(n))$, alors
 $f_1(n) \cdot f_2(n) \in O(g_1(n) \cdot g_2(n))$

Analyse de complexité en pratique

Quelques règles simples :

- Affectation, accès à un tableau, opérations arithmétiques, appel de fonction : $O(1)$
- Instruction If-Then-Else : $O(\text{complexité max des deux branches})$
- Séquence d'opérations : l'opération la plus couteuse domine (règle de la somme)
- Boucle simple : $O(nf(n))$ si le corps de la boucle est $O(f(n))$

Analyse de complexité en pratique

- Double boucle complète : $O(n^2 f(n))$ où $f(n)$ est la complexité du corps de la boucle
- Boucles incrémentales : $O(n^2)$ (si corps $O(1)$)

```
for (int i = 0; i < n; i++)  
    for (int j = i+1; j < n; j++)  
        ...
```

- Boucles avec un incrément exponentiel : $O(\log n)$ (si corps $O(1)$)

```
for (int i = 1; i <= n; i = 2*i)  
    ...
```

Exemple : 3SUM

```
1  int count_3sum(int tab[], int n) {  
2      int count = 0;  
3      for (int i = 0; i < n-2; i++)  
4          for (int j = i+1; j < n-1; j++)  
5              for (int k = j+1; k < n; k++)  
6                  if (tab[i]+tab[j]+tab[k] == 0)  
7                      count++;  
8      return count;  
9  }
```

Exemple : 3SUM

```
1  int count_3sum(int tab[], int n) {  
2      int count = 0;  
3      for (int i = 0; i < n-2; i++)  
4          for (int j = i+1; j < n-1; j++)  
5              for (int k = j+1; k < n; k++)  
6                  if (tab[i]+tab[j]+tab[k] == 0)  
7                      count++;  
8      return count;  
9  }
```

- Ligne 3 exécutée $O(n)$ fois.
- Ligne 4 exécutée $O(n^2)$ fois.
- Lignes 5, 6, et 7 exécutées une fois par triplet de valeurs. Nombre de triplets :

$$C_n^3 = \frac{n(n-1)(n-2)}{6} \in O(n^3).$$

⇒ Complexité en temps : $O(n^3)$.

Limitations de l'analyse asymptotique

- Les facteurs constants ont de l'importance pour des problèmes de petites tailles
 - ▶ Si n est petit, il vaut mieux un algorithme s'exécutant en $O(n^2)$ secondes qu'un algorithme s'exécutant en $O(\log n)$ années.
- Deux algorithmes de même complexité (grand-O) peuvent avoir des propriétés très différentes
 - ▶ Comme `count_3sum`, les deux algorithmes suivants sont $O(n^3)$
 - ▶ `count_3sum_2` est 6 fois plus lent et `has_3sum` peut traiter des tableaux aléatoires de taille 1 million en moins d'un centième de seconde.

```
int count_3sum_2(int tab[], int n) {  
    int count = 0;  
    for (int i = 0; i < n-2; i++)  
        for (int j = 0; j < n-1; j++)  
            for (int k = 0; k < n; k++)  
                if (i < j && j < k)  
                    if (tab[i]+tab[j]+tab[k] == 0)  
                        count++;  
    return count;  
}
```

```
int has_3sum(int tab[], int n) {  
    for (int i = 0; i < n-2; i++)  
        for (int j = i+1; j < n-1; j++)  
            for (int k = j+1; k < n; k++)  
                if (tab[i]+tab[j]+tab[k] == 0)  
                    return 1;  
    return 0;  
}
```

⇒ Important de tester l'algorithme dans des conditions réelles (ou de réaliser une analyse mathématique plus fine).

Complexité d'algorithmes récursifs

L'analyse de la complexité d'algorithmes récursifs mène généralement à une équation récurrente, dont la résolution n'est pas toujours aisée.

Exemples :

■ Fonction factorielle

```
int fact(int n) {  
    if (n <= 1)  
        return n;  
    return n * fact(n-1);  
}
```

$$T(0) = c_0$$

$$T(n) = T(n-1) + c_1$$

$$\Rightarrow T(n) = c_1 n + c_0 \in O(n)$$

■ Nombres de Fibonacci

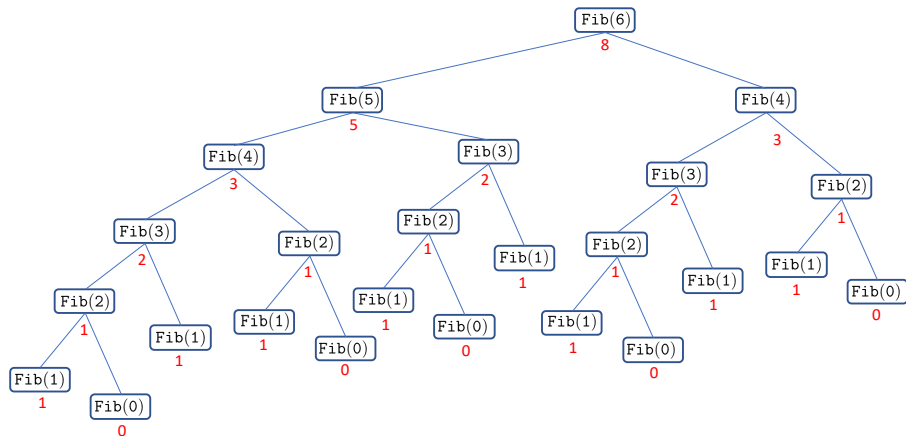
```
int Fib(int n) {  
    if (n <= 1)  
        return n;  
    return Fib(n-1)+Fib(n-2);  
}
```

$$T(0) = T(1) = c_0$$

$$T(n) = T(n-1) + T(n-2) + c_1$$

$$\Rightarrow T(n) > Fib(n) \approx \frac{1.618^n}{\sqrt{5}}$$

Nombres de Fibonacci : arbre des appels récursifs



Beaucoup de valeurs sont recalculées inutilement.

Plan

1. Introduction
2. Construction d'algorithmes itératifs
3. Construction d'algorithmes récursifs
4. Analyse de performance
 - Introduction
 - Approche empirique
 - Approche mathématique
 - Complexité en espace

Complexité en espace

La complexité **en espace** d'un algorithme mesure l'espace mémoire utilisé par l'algorithme en fonction de la taille de l'entrée.

Comme la complexité en temps :

- On la calcule dans le **pire cas**.
- On l'exprime en utilisant la **notation grand- O** .

Dans le cas des algorithmes **récurifs**, il faut prendre en compte l'espace mémoire nécessaire **au stockage du contexte des appels récurifs**, qui est proportionnel à la profondeur de l'arbre des appels récurifs.

Contextes d'appels de fonctions

A chaque appel de fonction, on doit retenir en mémoire (dans une zone appelée le **Stack**) le contexte de l'appel, c'est-à-dire :

- L'**endroit** où le code appelant doit continuer son exécution à la fin de l'appel
- La valeur des **arguments** fournis à la fonction
- La valeur des **variables locales** à la fonction

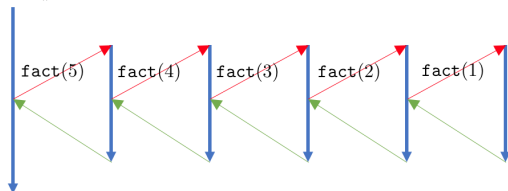
Cette information ne peut être **supprimée** qu'une fois l'appel terminé.

Le coût mémoire est directement proportionnel à la **profondeur** de l'arbre des appels récursifs.

Contextes d'appels de fonctions : illustration

```
int fact(int n) {  
    if (n == 1)  
        return 1;  
    return n*fact(n-1);  
}  
  
int main() {  
    fact(5);  
    return 0;  
}
```

main()



Contexte appel fact(1)
Contexte appel fact(2)
Contexte appel fact(3)
Contexte appel fact(4)
Contexte appel fact(5)

Exercice

Quelles sont les complexités en temps et en espace de la fonction `pow_rec2` ?

```
float pow_rec2(float a, int x) {  
    if (x == 1)  
        return a;  
    if (x % 2 == 0) // x pair  
        return pow_rec2(a * a, x/2);  
    else           // x impair  
        return a * pow_rec2(a * a, (x-1)/2);  
}
```

Partie 3

Algorithmique

14 septembre 2026

Plan

1. Recherche dichotomique
2. Diviser pour régner
3. Problèmes 2SUM et 3SUM

Introduction

Pour beaucoup de problèmes, il est crucial de développer un algorithme performant.

Des solutions efficaces ont été proposées pour la plupart des problèmes standards.

Pour aborder un nouveau problème, il existe des techniques génériques de résolution de problèmes.

Objectifs de cette leçon :

- Vous présenter des solutions efficaces pour la recherche et le tri.
- Vous présenter la technique du “diviser-pour-régner”,
- Vous montrer que le tri et la recherche permettent de résoudre efficacement beaucoup de problèmes algorithmiques.

Illustration : filtrage d'adresses emails

On gère un serveur d'emails et on aimerait ajouter une fonctionnalité de filtrage des adresses, soit :

- Liste noire : on ne veut pas laisser passer les emails des personnes de la liste
- Liste blanche : on ne veut laisser passer que les emails des personnes de la liste.

Combien d'emails peut-on espérer filtrer à la seconde en fonction de la longueur de la liste ?

Recherche linéaire : tableau quelconque

Warm-up : recherche dans un tableau d'entiers :

- Soit un tableau d'entiers, on veut déterminer si une valeur *key* se trouve dans le tableau.

Solution naïve sans faire d'hypothèse sur les valeurs du tableau :

```
int linear_search(int key, int tab[], int n) {  
    for (int i = 0; i < n; i++) {  
        if (tab[i] == key)  
            return i;  
    }  
    return -1;  
}
```

Complexité : $O(n)$ pour un tableau de taille n .

- Pire cas : l'entier recherché n'est pas dans la liste.

Recherche linéaire : tableau trié

Si on suppose que le tableau est trié, on peut s'arrêter plus tôt dans la recherche.

```
int sorted_linear_search(int key, int tab[], int n) {  
    int i = 0;  
    while (i < n && key > tab[i])  
        i++;  
  
    if (tab[i] == key)  
        return i;  
    else  
        return -1;  
}
```

Complexité identique : $O(n)$ pour un tableau de taille n .

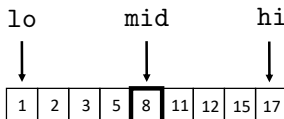
- Pire cas : l'entier recherché est plus grand que toutes les valeurs dans le tableau

Recherche dichotomique (*binary search*)

On peut faire (beaucoup) mieux si on suppose que le tableau est trié.

Idée : On compare la valeur recherchée à la valeur au milieu du tableau :

- Si elle est égale, on la renvoie
- Si elle est plus petite, on recherche **récurivement** la valeur dans la première moitié du tableau
- Si elle est plus grande, on recherche **récurivement** la valeur dans la seconde moitié du tableau



Recherche dichotomique : implémentation récursive

```
int binary_search_aux(int key, int tab[], int lo, int hi) {
```

Recherche dichotomique : implémentation récursive

```
if (lo > hi) return -1;

int mid = lo + (hi - lo) / 2;

if (key == tab[mid])
    return mid;
else if (key < tab[mid])
    return binary_search_aux(key, tab, lo, mid-1);
else
    return binary_search_aux(key, tab, mid+1, hi);
}

int binary_search(int key, int tab[], int n) {
    return binary_search_aux(key, tab, 0, n-1);
}
```

Remarque : $lo + (hi - lo) / 2$ est préférable à $(lo + hi) / 2$ pour éviter un dépassement de valeur si lo est un entier très grand.

Analyse de complexité

Le nombre d'appels récursifs est maximum lorsque la valeur ne se trouve pas dans le tableau.

Supposons pour simplifier les calculs que la taille du tableau n soit telle que $n = 2^k - 1$ pour un entier $k > 0$ ($\Rightarrow k = \log_2(n + 1)$).

Les tailles des sous-tableaux considérés à chaque étape sont :

$$2^k - 1, 2^{k-1} - 1, 2^{k-2} - 1, \dots, 2^1 - 1, 2^0 - 1$$

Il faudra donc $k + 1$ appels récursifs avant d'arriver au cas de base (low).

En dehors de l'appel récursif, le corps de la fonction est $O(1)$.

La complexité en temps est donc $O(\log n)$.

La complexité en espace est aussi $O(\log n)$ (au plus k appels récursifs imbriqués).

Recherche dichotomique : implémentation itérative

```
int binary_search_iter(int key, int tab[], int n) {  
    int lo = 0;  
    int hi = n-1;  
  
    while (lo <= hi) {  
        int mid = lo + (hi - lo) / 2;  
        if (key < tab[mid])  
            hi = mid - 1;  
        else if (key > tab[mid])  
            lo = mid + 1;  
        else  
            return mid;  
    }  
  
    return -1;  
}
```

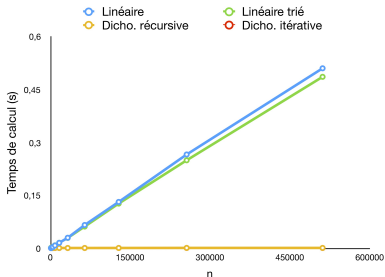
Complexité en temps : $O(\log n)$

Complexité en espace : $O(1)$

Analyse des temps de calcul

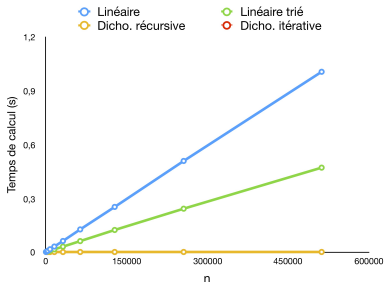
Recherche positive :

- Tableau $[0, 1, \dots, n - 1]$
- 1000 recherches d'une clé : `rand()%n`



Recherche négative :

- Tableau $[0, 2, 4, \dots, 2n - 2]$
- 1000 recherches d'une clé : `rand()%(2*n)+1`



Filtrage d'adresses email : implémentation

Adaptation du code à la recherche d'une chaîne de caractères.

```
int binary_search_iter(char *key, char **tab, int n) {
    int lo = 0;
    int hi = n-1;

    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int cmp = strcmp(key, tab[mid]);
        if (cmp < 0)
            hi = mid - 1;
        else if (cmp > 0)
            lo = mid + 1;
        else
            return mid;
    }

    return -1;
}
```

Remarques :

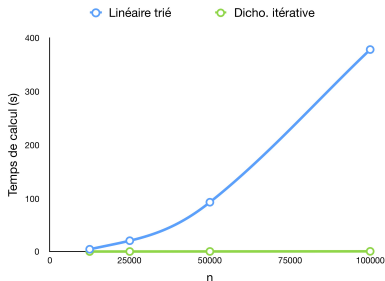
- `strcmp(s1,s2)` renvoie 0 si les deux chaînes sont identiques ou un entier < 0 (resp. > 0) si `s1` est avant (resp. après) `s2` dans l'ordre lexicographique (`string.h`).
- On peut aussi écrire une fonction générique en se basant sur des pointeurs sur void et de fonctions (cf. partie 3).

Filtrage d'adresses email : temps de calcul

Génération de données :

- Liste blanche : n chaînes de caractères (a-z) aléatoires de longueur 10.
- Requêtes : $10n$ chaînes prises au hasard dans la liste (que des recherches positives)

n	Rech. lin.(s)	Rech. dich.(s)
12500	4,30	0,03
25000	20,31	0,08
50000	92,41	0,19
100000	378,42	0,39



Pour une table de $n = 100000$ adresses email :

- Recherche dichotomique : 256000 vérifications par seconde.
- Recherche linéaire : 264 vérifications par seconde.

Plan

1. Recherche dichotomique
2. Diviser pour régner
3. Problèmes 2SUM et 3SUM

Diviser pour régner

La recherche binaire est basée sur le “diviser pour régner”, une technique générique de résolution de problèmes basée sur la récursivité.

Principe général : Soit un problème algorithmique à résoudre de taille n

- Si le problème est trivial (typiquement, n est petit), on le résout directement
- Sinon :
 - ▶ On **divise** le problème en b sous-problèmes de tailles n/a (avec $b \geq 0$ et $a > 1$ deux entiers).
 - ▶ On résout **récursivement** ces sous-problèmes
 - ▶ On **fusionne** les solutions aux sous-problèmes pour produire une solution au problème original

Le tri par fusion (merge sort) est également basé sur ce principe :

- On **divise** le tableau en deux.
- On trie les deux sous-tableaux **récurivement**.
- On **fusionne** les deux sous-tableaux triés.

```
static void mergesort_aux(int tab[], int lo, int hi) {  
    int n = hi - lo + 1;  
    if (n <= 1)  
        return;  
    int mid = lo + (n + 1) / 2;  
    mergesort_aux(tab, lo, mid - 1);  
    mergesort_aux(tab, mid, hi);  
    merge(tab, lo, mid, hi); // fusionne les sous-tableaux triés  
                             // tab[lo..mid-1] et tab[mid..hi]  
}
```

Complexité : $O(n \log n)$. On peut montrer qu'on ne peut pas faire mieux (cf INFO0902).

Diviser pour régner : analyse de complexité

La complexité d'une solution diviser pour régner s'écrit :

$$T(n) = bT(n/a) + f(n)$$

où $f(n)$ est la complexité de l'opération de fusion.

D'autant plus efficaces que $f(n)$ est faible, a est grand, et b est petit.

Sans intérêt si $f(n)$ n'est pas strictement meilleur que la complexité d'une solution naïve au problème.

Exemples :

- Recherche dichotomique : $a = 2$, $b = 1$, $f(n) = c \Rightarrow T(n) \in O(\log n)$
- Tri par fusion : $a = 2$, $b = 2$, $f(n) = cn \Rightarrow T(n) \in O(n \log n)$
- Calcul d'une puissance entière :
 $a = 2$, $b = 1$, $f(n) = c \Rightarrow T(n) \in O(\log n)$

Un exemple : recherche de pics

- Soit un tableau $tab[0..n+1]$ de $n+2$ valeurs réelles. On supposera que $tab[0] = tab[n+1] = -\infty$.
- Définition : $tab[i]$, avec $i \in \{1, \dots, n\}$, est un **pic** s'il n'est pas plus petit que ses voisins :

$$tab[i-1] \leq tab[i] \geq tab[i+1]$$

($tab[i]$ est un maximum local)

- **Problème algorithmique** : trouver un pic dans le tableau (n'importe lequel)
- Note : il en existe toujours un

Solution naïve

On teste toutes les positions séquentiellement :

```
int findPeak(float *tab, int n) {  
    for (int i = 1; i <= n; i++) {  
        if ((tab[i-1] <= tab[i]) && (tab[i] <= tab[i+1]))  
            return i;  
    }  
    return -1; // En principe, inutile  
}
```

Complexité : $O(n)$

Une solution par diviser-pour-régner

Idée :

- Sonder un élément $tab[i]$ et ses voisins $tab[i - 1]$ et $tab[i + 1]$
- Si c'est un pic : renvoyer i
- Sinon :
 - ▶ les valeurs doivent croître au moins d'un côté

$$tab[i - 1] > tab[i] \text{ ou } tab[i] < tab[i + 1]$$

- ▶ Si $A[i - 1] > A[i]$, on cherche le pic (récursivement) dans $tab[1 \dots i - 1]$
- ▶ Si $A[i + 1] > A[i]$, on cherche le pic (récursivement) dans $tab[i + 1 \dots n]$

(Implémentation en C laissée comme exercice.)

Analyse

Est-ce que l'algorithme est correct ? Oui. Il existera toujours un pic du côté choisi (peut se montrer par l'absurde).

Complexité : $a = 2$, $b = 1$, $f(n) = 1 \Rightarrow O(\log n)$ (comme la recherche dichotomique)

1. Recherche dichotomique
2. Diviser pour régner
3. Problèmes 2SUM et 3SUM

Problème 2SUM

*Étant donné un tableau de n entiers **uniques**, trouver (ou compter) les paires d'entiers qui somment à 0.*

Solution naïve :

```
int count_2sum(int tab[], int n) {  
    int count = 0;  
    for (int i = 0; i < n-1; i++)  
        for (int j = i+1; j < n; j++)  
            if (tab[i] + tab[j] == 0)  
                count++;  
    return count;  
}
```

Complexité en temps : $O(n^2)$

Problème 2SUM : 1ère solution basée sur le tri

Une première solution basée sur le [tri](#) et la [recherche dichotomique](#) :

- On trie le tableau (en utilisant le tri par fusion).
- Pour chaque élément `tab[i]` (négatif), on recherche `-tab[i]` dans le reste du tableau en utilisant la recherche dichotomique.

```
int count_2sum_bs(int tab[], int n) {  
    mergesort(tab, n);  
    int count = 0;  
    int i = 0;  
    while (i < n-1 && tab[i] < 0) {  
        if (binary_search(-tab[i], tab+i+1, n-i) != -1)  
            count++;  
        i++;  
    }  
    return count;  
}
```


Problème 2SUM : 1ère solution basée sur le tri

```
int count_2sum_bs(int tab[], int n) {  
    mergesort(tab, n);  
    int count = 0;  
    int i = 0;  
    while (i < n-1 && tab[i] < 0) {  
        if (binary_search(-tab[i], tab+i+1, n-i) != -1)  
            count++;  
        i++;  
    }  
    return count;  
}
```

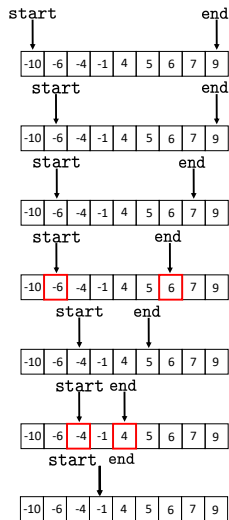
Complexité en temps : $O(n \log n)$

- Tri par fusion : $O(n \log n)$
- Pire cas pour la boucle for : toutes les valeurs sont négatives.
 - ▶ n recherches dichotomiques en $O(\log n) \Rightarrow O(n \log n)$.

Problème 2SUM : 2ème solution basée sur le tri

On peut se passer de la recherche dichotomique.

```
int count_2sum_fast(int tab[], int n) {  
    mergesort(tab, n);  
    int count = 0;  
    int start = 0;  
    int end = n-1;  
    while (start < end) {  
        int sum = tab[start] + tab[end];  
        if (sum == 0) {  
            count++;  
            start++;  
            end--;  
        } else if (sum > 0) {  
            end--;  
        } else {  
            start++;  
        }  
    }  
    return count;  
}
```



Problème 2SUM : 2ème solution basée sur le tri

```
int count_2sum_fast(int tab[], int n) {
    mergesort(tab, n);
    int count = 0;
    int start = 0;
    int end = n-1;
    while (start < end) {
        int sum = tab[start] + tab[end];
        if (sum == 0) {
            count++;
            start++;
            end--;
        } else if (sum > 0)
            end--;
        else
            start++;
    }
    return count;
}
```

Complexité en temps identique à la 1ère solution : $O(n \log n)$

- Tri par fusion : $O(n \log n)$
- Boucle while : $O(n)$, négligeable par rapport au tri

Problème 3SUM : 1ère solution basée sur le tri

En se basant sur la recherche dichotomique :

```
int count_3sum_bs(int tab[], int n) {
    mergesort(tab,n);
    int count = 0;
    int i = 0;
    while (i < n-2 && tab[i] < 0) {
        int j = i+1;
        while (j < n-1 && tab[i]+tab[j] < 0) {
            if (binary_search(-(tab[i]+tab[j]), tab+j+1, n-j) != -1)
                count++;
            j++;
        }
        i++;
    }
    return count;
}
```

Complexité : $O(n^2 \log n)$

- Double boucle : $O(n^2)$ itérations et recherche dichotomique en $O(\log n) \Rightarrow O(n^2 \log n)$
- Tri par fusion, $O(n \log n)$, négligeable

Problème 3SUM : 2ème solution basée sur le tri

```
int count_3sum_fast(int tab[], int n) {
    mergesort(tab,n);
    int count = 0;
    int i = 0;
    while (i < n-1 && tab[i] < 0) {
        int start = i + 1;
        int end = n - 1;
        while (start < end) {
            int sum = tab[i] + tab[start] + tab[end];
            if (sum == 0) {
                count++; start++; end--;
            } else if (sum > 0)
                end--;
            else
                start++;
        }
        i++;
    }
    return count;
}
```

Complexité en temps³ : $O(n^2)$

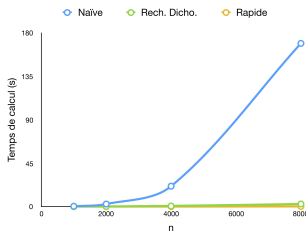
- Double boucle : $O(n^2)$ itérations
- Tri par fusion, $O(n \log n)$, négligeable

3. Longtemps considérée comme la solution optimale mais une solution $O(n^2/(\log n/ \log \log n)^{2/3})$ a été proposée en 2014.

Analyse empirique

Tableaux d'entiers compris entre -1000000 et +999999 sans doublons.

n	Naïve	Rech. bin.	Rapide
1000	0,36	0,03	< 0,01
2000	2,68	0,14	0,01
4000	21,20	0,60	0,04
8000	169,29	2,75	0,18



(Sur un Intel Core i7 2.5 Ghz)

Prédiction pour $n = 1000000$:

- > 10 ans pour la version naïve $O(n^3)$ (cf. partie 4)
- 17 heures pour la version utilisant la recherche dichotomique $O(n^2 \log n)$
- 1 heure pour la version rapide $O(n^2)$

Synthèse

- Le tri est un composant essentiel dans beaucoup d'applications.
- Le tri par fusion offre une solution optimale au problème.
- Passer de $O(n^2)$ à $O(n \log n)$ ou de $O(n)$ à $O(\log n)$ fait une énorme différence dans les applications pratiques.
- La recherche dichotomique et le tri par fusion sont basés sur l'idée générale du “diviser-pour-régner”, qui permet d'obtenir des solutions efficaces à beaucoup de problèmes.

Partie 4

Organisation de programmes

14 septembre 2026

Plan

1. Programmation modulaire
2. Masquage de l'information
3. Généricité
4. Compilation
5. Tests et débogage
6. Style

Plan

1. Programmation modulaire

Principe

Types de modules

Illustrations

Inclusion gardée

2. Masquage de l'information

3. Généricité

4. Compilation

5. Tests et débogage

6. Style

Programmation modulaire : principes

Un programme (en C ou un autre langage) est souvent découpé en une série de **modules** indépendants.

Un **module** est une collection de **services**, mis à la disposition de **clients**, c'est-à-dire d'autres modules ou programmes.

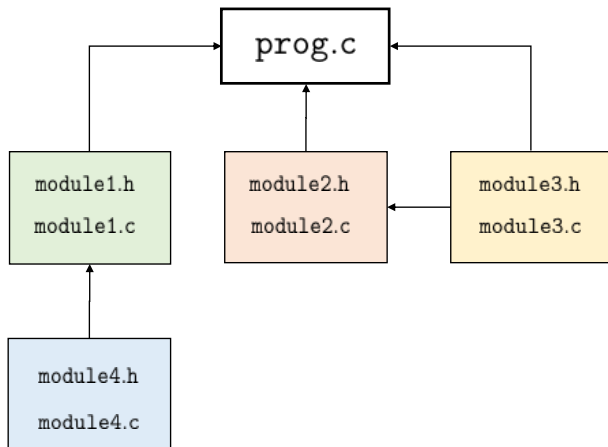
Chaque module a une **interface** qui décrit précisément les services qu'il fournit.

Les détails du module sont contenus dans son **implémentation**.

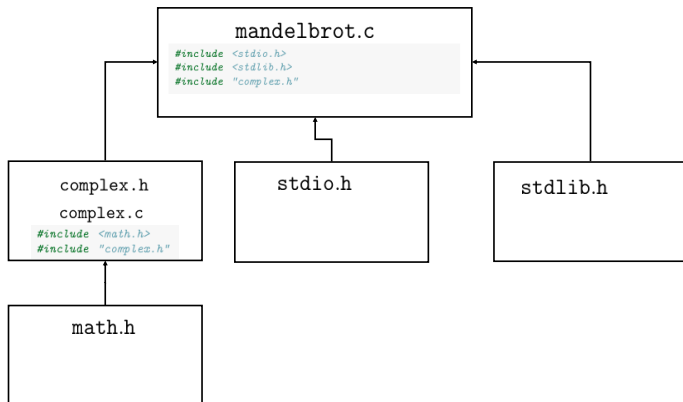
En C :

- Chaque **service** correspond à une **fonction ou procédure**.
- L'**interface** est précisée dans un fichier **d'entête** (.h).
- L'**implémentation** est fournie dans un fichier **source** (.c).

Programmation modulaire : illustration



Exemple : Mandelbrot



(Code : voir slides 35 et 37)

Avantages de l'approche modulaire

Abstraction :

- Chaque module peut être traité comme une abstraction : on sait ce qu'il fait mais on ne se tracasse pas de comment il le fait.
- Une fois que l'interface du module est bien précisée, on peut séparer les tâches d'implémentation module par module.

Réutilisabilité :

- Tout module est potentiellement réutilisable par d'autres clients.
- Les modules peuvent (devraient) être désignés avec cet objectif en tête.

Maintenabilité :

- Rend la maintenance du programme plus aisée.
- On peut corriger, améliorer chaque module séparément.

Découpage en modules

Pas facile de déterminer le découpage optimal. Souvent plusieurs bonnes solutions.

Deux propriétés principales d'un bon module :

- **Forte cohésion** : les services offerts au sein d'un même module doivent être liés les uns aux autres.
- **Couplage faible** : les modules doivent être aussi indépendants que possible les uns des autres.

Exemples

- Bons modules : fonctions de manipulations de matrices, fonctions de traitement de chaînes de caractères (`<string.h>`), définition de structures de nombres complexes.
- Mauvais modules : modules 'help', 'truc', ou 'divers' contenant toutes les fonctions auxiliaires d'un programme.

Grands types de modules

Data pool : une collection de variables et de constantes liées entre elles.

- Exemples : `<limits.h>` et `<float.h>`.

Librairie (bibliothèque) : une collection de fonctions liées entre elles

- Exemples : ensemble de fonctions mathématiques `<math.h>`, fonctions de manipulations de chaînes de caractères `<string.h>`.

Objet abstrait : une collection de fonctions opérant sur une structure de données cachées

- Exemples : un dictionnaire de la langue français, une pile.

Type abstrait de données : définition d'un nouveau type de données avec les opérations associées.

- Contrairement à un objet abstrait, on peut créer plusieurs objets de même type.
- Exemples : type complexe, type dictionnaire, type pile.

Data pool : implémentation typique

Juste un fichier .h avec des définitions de constantes (macro or const)

Exemple : limits.h (extrait)

```
/* Copyright (C) 1991-2018 Free Software Foundation, Inc.
   This file is part of the GNU C Library. */
...
/* Minimum and maximum values a `signed short int' can hold. */
# define SHRT_MIN      (-32768)
# define SHRT_MAX      32767

/* Maximum value an `unsigned short int' can hold. (Minimum is 0.) */
# define USHRT_MAX     65535

/* Minimum and maximum values a `signed int' can hold. */
# define INT_MIN       (-INT_MAX - 1)
# define INT_MAX       2147483647
/* Maximum value an `unsigned int' can hold. (Minimum is 0.) */
# define UINT_MAX      4294967295U

/* Minimum and maximum values a `signed long int' can hold. */
# if __WORDSIZE == 64
#   define LONG_MAX     9223372036854775807L
# else
#   define LONG_MAX     2147483647L
# endif
# define LONG_MIN      (-LONG_MAX - 1L)
...
```

Librairie : implémentation typique

Un fichier d'entête contenant les prototypes des fonctions accessibles au client et un fichier source les implémentant.

Exemple : une librairie de fonctions mathématiques

fctmath.h

```
#ifndef _FCTMATH_H
#define _FCTMATH_H

...

double cos(double);
float cosf(float);

double sin(double);
float sinf(float);

double pow(double, double);
float powf(float, float);

double sqrt(double);
float sqrtf(float);
...

#endif
```

fctmath.c

```
#include "fctmath.h"

...

double cos(double x) {
    ...
};

float cosf( float x) {
    ...
};

double sin( double ) {
    ...
};

float sinf( float ) {
    ...
};
...
```

Objet abstrait : implémentation typique

Idem qu'une librairie mais les fonctions sont toutes relatives à une structure de données **abstraite** qui est définie **concrètement** dans le fichier source.

Exemple : un dictionnaire français

dicofr.h

```
#ifndef _DICOFR_H
#define _DICOFR_H

int dicofr_init();
int dicofr_destroy();
int dicofr_search_word(char *word);
char *dicofr_get_definition(char *word);
int dicofr_add_word(char *word, char *definition);
...

#endif
```

dicofr.c

```
#include "dicofr.h"

// définition de la structure interne au module
static struct {
    ...
} dicofr;

// définition des fonctions de l'interface

int dicofr_init() {
    ...
}

int dicofr_destroy() {
    ...
};

int dicofr_search_word(char *word) {
    ...
}
...
```

Type abstrait de données : implémentation typique

Le fichier d'entête définit le nouveau type et fournit le prototype des fonctions opérants sur les objets de ce type. Le fichier source définit concrètement les fonctions.

Exemple 1 : type de données complexe

complex.h

```
#ifndef _COMPLEXE_H
#define _COMPLEXE_H

// définition du nouveau type

typedef struct {
    double re, im;
} complex;

// prototypes des fonctions

complex complex_new(double, double);
complex complex_sum(complex, complex);
complex complex_product(complex, complex);
double complex_modulus(complex);
...

#endif
```

complex.c

```
#include "complex.h"

// Implémentation des fonctions

complex complex_new(double re, double im) {
    ...
}

complex complex_sum(complex a, complex b) {
    ...
}

complex complex_product(complex a, complex b) {
    ...
}

double complex_modulus(complex c) {
    ...
}
...
```

Type abstrait de données : implémentation typique

Exemple 2 : type de données matrice

matrix.h

```
#ifndef _MATRIX_H
#define _MATRIX_H

// définition du nouveau type

typedef struct {
    double **el;
    unsigned n, m;
} matrix;

// prototypes des fonctions

matrix *matrix_create(unsigned, unsigned);
void matrix_free(matrix *);
matrix *matrix_sum(matrix *, matrix *);
matrix *matrix_product(matrix *, matrix *);
double matrix_determinant(matrix *);
...

#endif
```

matrix.c

```
#include <stdio.h>
#include <stdlib.h>
#include "matrix.h"

// Implémentation des fonctions
matrix *matrix_create(unsigned n, unsigned m) {
    ...
}

void matrix_free(matrix *m) {
    ...
}

matrix *matrix_sum(matrix *a, matrix *b) {
    ...
}
...
```

Remarque 1 : documentation

Il est important de documenter précisément toutes les fonctions de l'interface, dans le fichier d'entête et/ou via un document séparé.

Exemple :

```
/* ----- *
 * Creates a m by n matrix with all values set to 0. Returns NULL if
 * m <= 0 or n <= 0 and otherwise a pointer to the new matrix.
 *
 * ARGUMENTS
 * m          The number of rows
 * n          The number of columns
 *
 * RETURN
 * matrix     A pointer to the new matrix or NULL
 *
 * NOTE
 * The returned matrix should be cleaned with matrix_free after usage
 * -----*/

matrix *matrix_create(unsigned m, unsigned n)
```

Remarque 2 : conventions de nommage

Lorsqu'on utilise plusieurs modules, il est possible que des fonctions avec le même nom soient présentes dans différents modules si on n'y prête pas attention.

Particulièrement probable avec des types abstraits de données (exemple : `init`, `create`, `is_full...`).

Pour éviter les conflits de noms, il est utile de mettre le nom de module ou du type de données en préfixe des noms de fonctions du module.

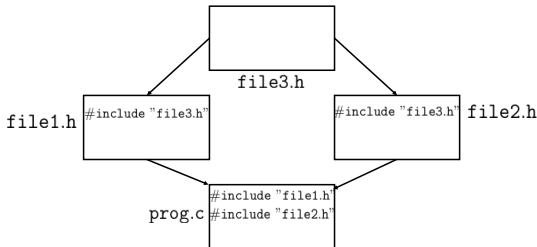
Exemples :

- `complex_new`, `complex_add`, `matrix_create`, `matrix_new...`
- `complexNew`, `complexAdd`, `matrixCreate`, `matrixNew...`

Inclusion gardée

Tout client d'un module doit inclure l'entête de ce module pour pouvoir accéder aux fonctions, variables et types qui y sont définis.

Un fichier d'entête peut inclure un autre fichier d'entête et donc un même fichier d'entête peut être compilé plusieurs fois.



Engendre des erreurs si un fichier d'entête inclus plusieurs fois contient :

- des définitions de types
- des déclarations de variables avec **initialisation**

Inclusion gardée

Deux solutions :

- S'arranger pour qu'un fichier d'entête ne soit inclus qu'une seule fois (compliqué)
- Utiliser la technique de l'inclusion gardée.

On entoure le fichier d'entête (`module.h`) des instructions suivantes :

```
#ifndef _MODULE_H // N'inclut ce qui suit que si _MODULE_H n'est pas définie
#define _MODULE_H // définit la constante _MODULE_H
...
// corps du fichier d'entête
...
#endif
```

Le nom de la constante n'a pas d'importance (`_MODULE_H`, `__MODULE_....`) pour autant qu'il soit le plus éloigné possible de noms communs de constantes (`H`, `N...`).

Pas toujours nécessaire mais autant le faire systématiquement.

Plan

1. Programmation modulaire
2. Masquage de l'information
 - Principe
 - Fonctions statiques
 - Types opaques
3. Généricité
4. Compilation
5. Tests et débogage
6. Style

Masquage de l'information

Un bon module garde **secret les détails d'implémentation** non utiles au client.

Deux avantages :

- **Sécurité** : le client ne peut pas corrompre les données. Il est obligé d'utiliser les fonctions de l'interface.
- **Flexibilité** : le programmeur peut modifier l'implémentation sans devoir en référer au client.

Deux mécanismes sont disponibles en C pour faire ça :

- **Déclaration statique** des variables globales et fonctions/procédures privées.
- Utilisation d'un **type opaque**.

Variables globales et fonctions statiques

Le mot-clé `static` permet de rendre les variables **globales** et les fonctions/procédures **inaccessibles** en dehors du code source où elles sont définies.

Doit être utilisé pour toutes les fonctions et variables internes au module non destinées à être utilisées par le client.

Exemples :

`dicoifr.h`

```
#ifndef _DICOFR_H
#define _DICOFR_H

int dicoifr_init();
...

#endif
```

`dicoifr.c`

```
#include "dicoifr.h"

// définition de la structure interne au module
static struct {
    ...
} dicoifr;
```

// définition des fonctions de l'interface

```
static int load_data() {
    ...
}

static void terminate(char *message) {
    printf("%s\n", message);
    exit(EXIT_FAILURE);
}

int dicoifr_init() {
    ...
    if (!load_data()) {
        terminate("error when loading data");
    }
    ...
}
...
```

Remarque : classe de stockage statique

Le mot-clé `static` a une signification tout à fait différente quand il est utilisé pour des variables locales.

```
int func(int n) {  
    static int a;  
    ...  
}
```

Dans ce contexte, `static` indique que la variable est allouée de manière permanente (statique) et garde sa valeur d'un appel à l'autre de la fonction `func`.

Utilisation très spécifique, à éviter dans le contexte du cours.

Types opaques : motivation

Problème de l'implémentation précédente du type de données 'matrix' : la structure est visible dans l'entête et donc est accessible au client :

```
matrix.h
#ifndef _MATRIX_H
#define _MATRIX_H

// définition du nouveau type

typedef struct {
    double **el;
    unsigned n, m;
} matrix;
...
```

Le client est autorisé à utiliser `mat.m` ou `mat.n` directement s'il le désire et à créer une matrice sans passer par `matrix_new`.

Quid si on veut changer la structure ? Il faudra modifier tous les clients utilisant l'ancienne structure.

La solution est de rendre la structure **opaque**.

Types opaques : exemple 1

matrix.h

```
#ifndef _MATRIX_H
#define _MATRIX_H

// définition du nouveau type

typedef struct matrix_t matrix;

// prototypes des fonctions

matrix *matrix_create(unsigned, unsigned);
void matrix_free(matrix *);
double matrix_getelement(matrix *,int,int);
void matrix_setelement(matrix *,int,int);
unsigned matrix_getnbrows(matrix *);
unsigned matrix_getnbcols(matrix *);
...

#endif
```

matrix.c

```
#include <stdio.h>
#include <stdlib.h>
#include "matrix.h"

// Définition concrète du type
struct matrix_t {
    double **el;
    unsigned n, m;
};

// Implémentation des fonctions
matrix *matrix_create(unsigned n, unsigned m) {
    ...
}

void matrix_free(matrix *m) {
    ...
}
...
```

- 'typedef struct matrix_t matrix' définit le type matrix comme une structure matrix_t mais ne la définit pas.
- La définition concrète est déplacée dans le fichier source.
- Les clients, qui incluent matrix.h, n'y ont donc plus accès.

Types opaques

Une fois le type rendu opaque, on peut changer l'implémentation de la structure sans affecter les clients.

Restriction : le client n'ayant pas accès aux détails de la structure, il ne peut y accéder que par pointeur.

Exemple :

```
#include "matrix.h"
...
matrix m; // Impossible !
matrix *m; // Ok !
...
```

En conséquence, le code du type `complex` du slide 173 ne peut pas être rendu opaque. Il faut plutôt utiliser l'implémentation par pointeur du slide 49.

Types opaques : exemple 2

complex.h

```
typedef struct complex_t complex;

// constructeur
complex *complex_new(double, double);
// destructeur
void complex_destroy(complex *);
// accesseurs
double complex_real_part(complex *);
double complex_imgry_part(complex *);
void complex_set_real_part(complex *, double);
void complex_set_imgry_part(complex *, double);
// opérateurs
void complex_sum(complex *, complex *);
void complex_product(complex *, complex *);
double complex_modulus(complex *);
...
```

complex.c

```
#include <math.h>
#include "complex.h"

struct complex_t {
    double re, im;
};

complex *complex_new(double re, double im) {
    complex *c = (complex *)malloc(sizeof(complex));
    c->re = re;
    c->im = im;
    return c;
}

double complex_real_part(complex *a) {
    return a->re;
}

void complex_set_real_part(complex *a, double re) {
    a->re = re;
    return a;
}
...
```

Plan

1. Programmation modulaire
2. Masquage de l'information
3. Généricité
 - Principe
 - Macros
 - Pointeurs de fonctions
 - Pointeur sur void
4. Compilation
5. Tests et débogage
6. Style

Généricité

Pour augmenter la réutilisabilité d'un module, on souhaite souvent le rendre le plus **générique** possible.

Exemple : module de tri (basé sur le tri par insertion)

sort.h

```
void sort(int array[], int length);
```

sort.c

```
#include "sort.h"

void sort(int array[], int length) {
    int i = 1;
    while (i < length) {
        int key = array[i];
        int j = i;
        while (j > 0 && array[j-1]>key) {
            array[j] = array[j-1];
            j--;
        }
        array[j] = key;
        i++;
    }
}
```

Inutilisable si on veut trier des tableaux d'autre chose que des entiers.

Généricité en C

Le C n'a pas de mécanisme simple pour rendre le code générique. Il faut un peu bricoler.

Trois techniques néanmoins :

- Utilisation de macros (ou typedef)
- Pointeur de fonctions
- Pointeur sur void

Généricité en C via des macros

On peut **paramétrer** le type du tableau trié en utilisant un `#define` (ou un `typedef`).

`sort.h`

```
#define SORTTYPE int
[Ou bien: typedef int SORTTYPE;]

void sort(SORTTYPE array[], int length);
```

`sort.c`

```
#include "sort.h"

void sort(SORTTYPE array[], int length) {
    int i = 1;
    while (i < length) {
        SORTTYPE key = array[i];
        int j = i;
        while (j > 0 && array[j-1]>key) {
            array[j] = array[j-1];
            j--;
        }
        array[j] = key;
        i++;
    }
}
```

Le client peut définir `SORTTYPE` au type désiré avant d'inclure `"sort.h"`.

On peut utiliser des macros pour faire des choses plus sophistiquées, au détriment de la lisibilité du code.

Pointeurs de fonctions

Soit l'implémentation suivante de la méthode de la sécante :

```
double secant_method(double approx0, double approx1, double min_error) {
    double xcurrent, xp, xpp;

    xcurrent = approx1;
    xp = approx0;

    while (fabs(xcurrent-xp)) {
        xpp = xp;
        xp = xcurrent;
        xcurrent = ((xpp*f(xp))-((xp)*f(xpp)))/((f(xp)-f(xpp)));
    }

    return xcurrent;
}
```

Le mécanisme de passage de la fonction `f` dont on veut calculer la racine n'est pas satisfaisant.

Solution plus appropriée : passer la fonction `f` en argument. C'est possible en utilisant un [pointeur de fonction](#).

Pointeurs de fonctions

Comme tout élément en C, une fonction dispose d'une **adresse en mémoire**. Son nom peut être utilisé pour dénoter cette adresse.

Il est possible de stocker ces adresses dans des variables et de les passer en arguments à des fonctions. Les variables et arguments sont alors de type **pointeur de fonction**.

La **déclaration** d'un pointeur de fonction se fait comme suit :

```
type (* id) ([type1[, type2] [,...]]);
```

Note : les parenthèses autour du * sont nécessaires pour distinguer le pointeur de fonction d'une fonction renvoyant un pointeur.

Exemples :

- `int (*fonction)(int, int);`
- `void (*procedure)(float);`

Pointeurs de fonctions

Une version générique de la méthode de la sécante :

```
double secant_method(double (* f)(double), double approx0,  
                    double approx1, double min_error) {  
    ...  
    xcurrent = ((xpp*f(xp))-((xp)*f(xpp)))/((f(xp)-f(xpp)));  
    ...  
}
```

L'appel à `f` peut aussi se faire via `(*f)(.)` ou directement via `f(.)`.

Au niveau du client :

```
double myfunction(double x) {  
    return (pow(x,3)-18);  
}  
  
int main() {  
    double root = secant_method(myfunction, -2.0, 2.0, 0.0005);  
}
```


Pointeur sur void

Ce mécanisme n'est pas suffisant pour écrire une fonction de tri générique.

Il faut pour cela qu'on puisse manipuler des données dont le type n'est pas connu à l'avance.

Solution : **pointeur sur void** :

- On passe les données via un **pointeur sur void** qui peut pointer vers n'importe quoi et est déclaré comme suit :

```
void *p;
```

- Quand on veut manipuler réellement la donnée, on doit néanmoins préciser le type en utilisant une **conversion de type**. Par exemple :

```
(int *)p
```

- **Idée** : on demande au client de fournir les fonctions de manipulation des données en utilisant des pointeurs de fonctions.

Exemple d'algorithme de tri générique

Manipulation de données dans le contexte du tri : comparaison et échange de valeurs.

```
void sort(void *array, int length, int (*compare)(void*, int, int),
          void (*swap)(void *, int, int)) {
    int i = 1;
    while (i < length) {
        int j = i;
        while (j > 0 && !(compare(array, j-1, j))) {
            swap(array, j-1, j);
            j--;
        }
        i++;
    }
}
```

- `swap(array, i, j)` échange les éléments aux positions `i` et `j` dans `array`.
- `compare(array, i, j)` vaut 1 si l'élément `i` est avant l'élément `j` en terme d'ordre, 0 sinon.

Tri générique : application 1

Implémentation de compare et swap pour le tri d'un tableau d'entiers :

```
void swap_int(void *array, int i, int j) {  
    int temp = ((int*)array)[i];  
    ((int*)array)[i] = ((int*)array)[j];  
    ((int*)array)[j] = temp;  
}  
  
int compare_int(void *array, int i, int j) {  
    return (((int*)array)[i] <= ((int*)array)[j]);  
}
```

Utilisation au niveau du client :

```
int A[5]={5, 4, 3, 2, 1};  
sort(A, 5, compare_int, swap_int);
```

Tri générique : application 2

Implémentation de compare et swap pour trier un tableau de **complexes** selon leurs modules :

```
void swap_complex(void *array, int i, int j) {
    complex temp = ((complex*)array)[i];
    ((complex*)array)[i] = ((complex*)array)[j];
    ((complex*)array)[j] = temp;
}

int compare_complex_mod(void *array, int i, int j) {
    return (complex_modulus(((complex*)array)[i]) <= complex_modulus(((complex*)array)[j]));
}
```

Utilisation au niveau du client :

```
complex C[5]={ {2,5}, {3,4}, {0,3}, {1,2}, {1,0}};
sort(C, 5, compare_complex_mod, swap_complex);
```

La programmation modulaire en C

Les mécanismes de programmation modulaire en C, tels qu'exposés, sont fonctionnels mais relativement rudimentaires.

Dans d'autres langages de programmation, la modularité est gérée de manière plus naturelle, même si les concepts restent les mêmes.

Par exemple :

- En programmation **orientée objet** (C++, Java, etc.), il existe de nombreux mécanismes pour générer le masquage de l'information, la définition de nouveau type abstrait et la généricité.
- En programmation **fonctionnelle** (Scheme, Lisp, Haskell), les fonctions sont des valeurs comme les autres qui peuvent être passées comme arguments sans passer par un mécanisme de pointeurs.

Plan

1. Programmation modulaire
2. Masquage de l'information
3. Généricité
4. Compilation
Principe
Make
5. Tests et débogage
6. Style

Compilation d'un seul fichier C

Si tout le code source tient dans un seul fichier (`nom_programme.c`), on peut le compiler via la commande suivante :

```
gcc -o nom_executable nom_programme.c
```

Si le code est séparé en différents modules, on peut les compiler en une seule ligne :

```
gcc -o nom_executable nom_programme.c module1.c module2.c
```

Cette commande compile en fait **séparément** les différents fichiers avant de les **lier** pour créer l'exécutable.

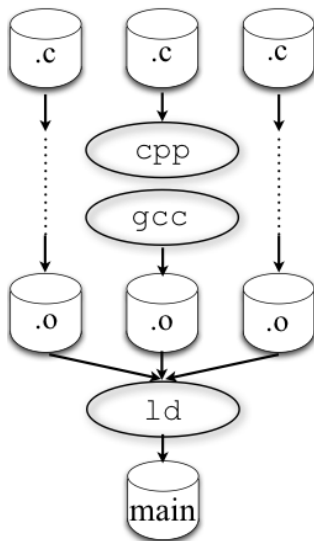
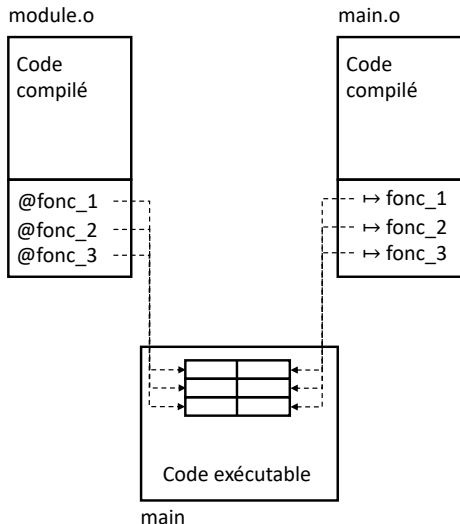
Les différentes étapes de la compilation

Pour chaque fichier source séparément, on passe par les étapes suivantes :

- **Pré-traitement** : éliminations des commentaires, remplacement des macros, inclusion des sous-fichiers
- Génération d'un **fichier objet** (.o) contenant :
 - ▶ le code compilé en langage d'assemblage
 - ▶ La table des liens (les variables/fonctions exportées ou importées par le module)

Ensuite, l'exécutable est obtenu à partir des fichiers objets en utilisant **l'éditeur de liens** (ld). Ce dernier assemble les codes compilés et fait les liens entre les appels de fonctions et de variables.

Les différentes étapes de la compilation



Les différentes étapes de la compilation

Pour faire les étapes manuellement :

```
> gcc -c nom_programme.c
> gcc -c module1.c
> gcc -c module2.c
> gcc -o nom_executable nom_programme.o module1.o module2.o
```

Avec l'approche manuelle, on ne doit pas recompiler la totalité des modules, seulement ceux qui ont été modifiés.

Par contre, il peut être très fastidieux de gérer les recompilations à la main.

Deux solutions :

- Utiliser un IDE (environnement de développement intégré. Par exemple CodeBlocks)
- Utiliser l'outil unix Make en ligne de commande

Make

Make est un utilitaire permettant de gérer la compilation d'un programme réparti en plusieurs fichiers (pas nécessairement en C).

Make est un outil très puissant dont on va juste voir ici le strict minimum.

Principe :

- On place un fichier appelé Makefile (ou makefile) dans le répertoire où se trouvent les sources.
- On lance la compilation en faisant :

```
> make [cible]
```

où `cible` est défini dans le fichier Makefile.

Fichier Makefile

Le fichier est constitué d'une suite de règles :

```
cible: dépendances  
[tabulation] actions
```

où :

- cible est un nom de fichier ou un simple label
- dépendances est une liste de fichiers dont dépend la cible
- actions est une liste d'actions à effectuer

Suite à un appel à `make cible`, les actions ne sont effectuées que si la date du fichier cible est moins récente qu'au moins l'un des fichiers de dépendances.

Si les fichiers de dépendances apparaissent comme cibles dans une règle, ils sont mis à jour récursivement selon le même principe avant de gérer la cible courante.

Illustration : version 1

Exemple de Makefile dans le cas d'un programme `prog.c` basé sur deux modules, `module1.c` et `module2.c`.

```
1  main: prog.o module1.o module2.o
2      gcc -o main prog.o module1.o module2.o
3  prog.o: prog.c module1.h module2.h
4      gcc -c prog.c
5  module1.o: module1.c module1.h
6      gcc -c module1.c
7  module2.o: module2.c module2.h
8      gcc -c module2.c
```

On construit le programme en faisant `make main` ou plus simplement `make` (qui utilise la première cible du fichier par défaut).

Remarque : on peut obtenir toutes les dépendances dans un répertoire (les lignes 3, 5, et 7) en faisant `'gcc -MM *.c'`.

Illustration : version 2

- Make sait comment obtenir un fichier o à partir d'un fichier c. On peut enlever les lignes 4, 6, et 8.
- Il faut néanmoins lui dire quel compilateur utiliser via la définition d'une variable CC au début du fichier (ligne 1 ci-dessous).
- On peut fournir les flags de compilation via une variable CFLAGS.

```
1  CC = gcc
2  CFLAGS = -Wall -Wextra -Wmissing-prototypes --pedantic\
3          -std=c99
4  main: prog.o module1.o module2.o
5      gcc -o main prog.o module1.o module2.o
6  prog.o: prog.c module1.h module2.h
7  module1.o: module1.c module1.h
8  module2.o: module2.c module2.h
```

Remarque sur les flags de compilation

Pour ce cours (devoirs et projets), on utilise les flags suivants ;

- `--std=c99` : spécifie la norme C99
- `-pedantic` : application stricte de la norme C99
- `-Wall` : affiche (presque) tous les warnings
- `-Wextra` : affiche d'autres warnings
- `-Wmissing-prototypes` : affiche un warning pour les prototypes non définis

Une liste complète des warnings générés par les options `-Wall` et `-Wextra` peut être obtenue ici :

<https://gcc.gnu.org/onlinedocs/gcc-6.1.0/gcc/Warning-Options.html>

Illustration : version 3

On peut encore simplifier le fichier en utilisant des variables, substituées dans les règles via `$(VAR)`, où `VAR` est le nom de la variable.

```
1  OFILES = prog.o module1.o module2.o
2  TARGET = main
3  CC = gcc
4  CFLAGS = -Wall -Wextra -Wmissing-prototypes --pedantic\
5           -std=c99
6  $(TARGET): $(OFILES)
7      $(CC) $(OFILES) -o $(TARGET)
8  prog.o: prog.c module1.h module2.h
9  module1.o: module1.c module1.h
10 module2.o: module2.c module2.h
```

Les seules lignes à écrire sont les lignes 1 et 2. Les lignes 3-7 sont génériques et les lignes 8-10 sont obtenues directement via `'gcc -MM *.c'`.

Illustration : version finale

- Il peut être utile d'ajouter des règles non liées à des fichiers.
- Par ex. pour supprimer les fichiers compilés (clean), exécuter le programme (run) et créer une archive avec le code (archive)

```
1  OFILES = prog.o module1.o module2.o
2  TARGET = main
3  CC = gcc
4  CFLAGS = -Wall -Wextra -Wmissing-prototypes --pedantic\
5           -std=c99
6  .PHONY: all clean run archive
7
8  all: $(TARGET)
9  clean:
10     rm -f $(OFILES) $(TARGET)
11  run: $(TARGET)
12     ./$(TARGET)
13  archive:
14     tar cvfz illustration_makefile.tar.gz *.h *.c Makefile README
15  $(TARGET): $(OFILES)
16     $(CC) $(OFILES) -o $(TARGET)
17  prog.o: prog.c module1.h module2.h
18  module1.o: module1.c module1.h
19  module2.o: module2.c module2.h
```

- La ligne 6 (`.PHONY: ...`) précise que les cibles qui suivent ne correspondent pas à des fichiers.
- La cible `all` en première position est celle qui est exécutée par défaut.

Synthèse

make est un outil très puissant et très complexe. Des livres entiers lui sont consacrés.

C'est l'outil de base pour faciliter la distribution de code source sous environnements de type Unix/Linux.

Beaucoup plus portable que de distribuer des projets produits par un IDE.

Pour plus d'information :

<https://www.gnu.org/software/make/manual/make.html>.

Plan

1. Programmation modulaire
2. Masquage de l'information
3. Généricité
4. Compilation
- 5. Tests et débogage**
6. Style

Tests

Une fois qu'un programme compile, il faut le **tester**, c'est-à-dire vérifier que son comportement est **conforme** au comportement attendu (**test de conformité**)

Un des avantages de la programmation modulaire est qu'on peut tester chaque module séparément.

Vérifier le fonctionnement de **portions de code** (p.ex, une fonction, un module) indépendamment des programmes qui les utilisent est ce qu'on appelle un **test unitaire**.

Il existe aussi des tests **d'intégration**, qui vérifient les interactions entre modules, et des tests **systèmes**, qui vérifient le comportement d'un système complet.

Tests unitaires en pratique

Pour chaque module, on crée un fichier source `module_main_test.c` avec une fonction `main` chargée de réaliser les tests.

Ces tests doivent :

- mettre en œuvre l'ensemble des fonctionnalités décrites dans les spécifications du module
- explorer le fonctionnement du module dans des conditions non-spécifiées

Établir ces tests peut constituer un défi en soi.

- Par exemple, comment tester la validité d'un générateur de nombres aléatoires, une fonction mathématique, un générateur d'images ?

Une bonne pratique est d'écrire les tests avant d'implémenter le module.

Exemple 1

module.h

```
...  
int abs(int a);  
...
```

module.c

```
#include "module.h"  
...  
int abs(int a) {  
    if (a < 0) return -a;  
    return a;  
}  
...
```

test_main_module.c

```
#include "module.c"  
#include <assert.h>  
  
int main() {  
    ...  
    // tests de la fonction abs  
    int x = -3;  
    int y = abs(x);  
    assert (x == y || -x == y);  
    assert (y >= 0);  
    x = abs(y);  
    assert (y == x);  
    ...  
    return 0;  
}
```

La macro assert

```
void assert(scalar expression)
```

assert est une macro définie dans `assert.h`.

Son unique argument est une assertion, c'est-à-dire une expression qu'on suppose être vraie à un moment de l'exécution du programme.

Fonctionnement :

- Si l'expression a une valeur non nulle, assert ne fait rien.
- Si l'expression a une valeur nulle, un message est affiché et l'exécution du programme est arrêtée.

Exemple de message :

```
a.out: module.c:9: main: Assertion `y >= 0' failed.
```

Exemple 2

module.h

```
...  
int swap(int *x, int *y);  
...
```

module.c

```
#include "module.h"  
...  
int swap(int * x, int * y) {  
    if (x == NULL || y == NULL)  
        return -1;  
    int t = *x;  
    *x = *y;  
    *y = t;  
    return 0;  
}  
...
```

test_main_module.c

```
#include "module.c"  
#include <assert.h>  
  
int main() {  
    ...  
    // tests de la fonction swap  
    int a = 0, b = 1;  
    int rv = swap(&a, &b);  
    assert (rv == 0);  
    assert (a == 1);  
    assert (b == 0);  
    rv = swap(&a, NULL);  
    assert (rv != 0);  
    assert (a == 1);  
    ...  
    return 0;  
}
```


Tests unitaires

L'approche informelle manuelle précédente est suffisante dans le cadre de ce cours.

C'est celle que nous utiliserons pour tester vos codes.

Il existe cependant de nombreux outils pour construire ces tests de façon plus systématique. Par exemple : CUnit, cmocka, Seatest...

Débogage

Une fois qu'un test a mis en évidence un problème dans une fonction d'un module, il faut isoler et corriger l'erreur (bogue) dans le programme. C'est ce qu'on appelle le débogage.

Activité assez compliquée en général : le bogue peut être très éloigné du symptôme.

Un programmeur passe en général plus de temps à déboguer du code existant qu'à écrire du nouveau code.

Bogues fréquents en C

- Mauvais cast (ou cast implicite) provoquant une perte de précision
 - ▶ `(float)(x/y)` au lieu de `((float) x)/((float y))`
- Accès en dehors des tableaux : en particulier le `'\0'` des chaînes de caractères.
- Variable/mémoire non initialisée
- Confusion entre `=` et `==`
- Ne pas traiter le code de retour d'une fonction pouvant indiquer une erreur
- Boucle infinie

Prévenir les bogues

Pour prévenir les bogues (ou faciliter le débogage) :

- Raisonnablez sur papier avant de vous lancer dans l'implémentation (pensez aux invariants)
- Documentez votre code (surtout lorsque vous travaillez à plusieurs)
- Privilégiez toujours la lisibilité et la clarté du code à sa compacité.
Un mauvais exemple :

```
void my_strcpy(char dest[], char src[]) {  
    while ((*dst++ = *src++));  
}
```

- Evitez les effets de bord (variables globales, variables locales statiques...)
- Activez (et supprimez) tous les warnings de compilation
- Adoptez une programmation **défensive** (voir plus loin).

Techniques de débogage

Lecture du code

- En général, inefficace si l'auteur est le lecteur

Exécution instrumentée :

- Ajouts d'assertions
- Ajouts de `printf`

Exécution contrôlée

- Utilisation d'un débogueur, permettant d'exécuter le code pas-à-pas, de mettre des points d'arrêts, de consulter les variables en temps réel...
- Par exemple, `gdb` ou votre IDE préféré (p. ex., CodeBlocks).

Utilisation de `printf` pour le débogage

Approche très simple et facile à mettre en œuvre mais peut être longue et fastidieuse.

Un classique : afficher les indices d'accès à un tableau pour détecter les débordements.

Remarque :

- La commande `printf` est bufferisée : l'appel n'affiche pas tout de suite le résultat à l'écran.
- Le bug pourrait donc suivre un `printf` avorté plutôt que le précéder.
- Préférez l'instruction `fprintf(stderr, ...)` plutôt que `printf(...)`, qui n'est pas bufferisée.

Utilisation de printf pour le débogage

Des macros permettent d'afficher de l'information utile lors du débogage :

- `__LINE__` : le numéro de ligne,
- `__FILE__` : le nom de fichier,
- `__FUNCTION__` : le nom de la fonction.

Macro générale d'affichage d'un message de debugage :

```
#define DEBUG(message, indice) \  
    fprintf(stderr, "Error (%s%d): ligne %d, fonction \  
    [%s], fichier [%s]\n", message, indice, __LINE__, \  
    __FUNCTION__, __FILE__)
```

Utilisation :

```
DEBUG("indice i=", i);
```

Programmation défensive

La programmation défensive consiste à penser à tous les cas possibles de mauvaises utilisations de ses fonctions, à ajouter des tests pour détecter ces situations, et à rapporter les erreurs correspondantes.

Elle consiste essentiellement à vérifier la pré-condition des algorithmes mais on peut également tester des invariants.

Elle permet de détecter les symptômes d'un bug au plus tôt.

Deux solutions principales en cas d'erreur :

- Afficher un message et arrêter le programme.
- Prévoir un résultat spécial de la fonction (cfr. le return de la fonction `main()`)

Programmation défensive : un exemple

(Bradley, 1998)

```
int reverse(char *in, char *out) {
    if (!in || !out) return -1;

    int i;
    for (i = 0; in[i]; i++);

    assert(in[i] == '\0');

    for (i--; i >= 0; i--) {
        assert(in[i] != '\0');
        *out = in[i];
        out++;
    }
    *out = '\0';
    return 0;
}
```

- La fonction renvoie 0 si tout s'est bien passé, -1 sinon.
- Les assert permettent de détecter un problème au niveau de l'algorithme.
- Ce code reste cependant vulnérable à des argument mal formés (*pourquoi ?*).

Une version plus robuste, en ajoutant comme argument la longueur maximale de la chaîne pointée par `in` :

```
int nreverse(char *in, char *out, int maxLength) {
    if (!in || !out) return -1;

    int i;
    for (i = 0; in[i]; i++) {
        if (i > maxLength) return -2;
    }

    assert(in[i] == '\0');

    for (i--; i >= 0; i--) {
        assert(in[i] != '\0');
        *out = in[i];
        out++;
    }
    *out = '\0';
    return 0;
}
```

Programmation défensive : remarques

Faire des tests systématiques des arguments a un coût en terme de temps de calcul.

Généralement seulement utile pour les fonctions de l'interface d'un module auxquels ont accès les utilisateurs. Pas nécessaire pour les fonctions statiques du module.

On réservera l'utilisation de `assert` à des tests du bon fonctionnement de l'algorithme, pas aux tests de vérification des arguments.

La macro `assert` peut être désactivée en faisant.

```
#define NDEBUG
```

Permet de maintenir deux versions du code compilé, une version pour le débogage et une version de production.

Plan

1. Programmation modulaire
2. Masquage de l'information
3. Généricité
4. Compilation
5. Tests et débogage
6. Style

Euh ?

```
#include "stdio.h"
#define e 3
#define g (e/e)
#define h ((g+e)/2)
#define f (e-g-h)
#define j (e*e-g)
#define k (j-h)
#define l(x) tab2[x]/h
#define m(n,a) ((n%l(a))==a))

long tab1[]={ 989L,5L,26L,0L,88319L,123L,0L,9367L };
int tab2[]={ 4,6,10,14,22,26,34,38,46,58,62,74,82,86 };

main(m1,s) char *s; {
    int a,b,c,d,o[k],n=(int)s;
    if(m1==1){ char b[2*j+f-g]; main(l(h+e)+h+e,b); printf(b); }
    else switch(m1-h){
        case f:
            a=(b=(c=(d=g)<<g)<<g)<<g;
            return(m(n,a|c)|m(n,b)|m(n,a|d)|m(n,c|d));
        case h:
            for(a=f;a<j;++a)if(tab1[a]&&!(tab1[a]%((long)l(n))))return(a);
        case g:
            if(n<h)return(g);
            if(n<j){n-=g;c='D';o[f]=h;o[g]=f;}
            else{c='\r'\b';n-=j-g;o[f]=o[g]=g;}
            if((b=n)>=e)for(b=g<<g;b<n;++b)o[b]=o[b-h]+o[b-g]+c;
            return(o[b-g]%n+k-h);
        default:
            if(m1==e) main(m1-g+e+h,s+g); else *(s+g)=f;
            for(*s=a=f;a<e;) *s=(s<<e)|main(h+a++,(char *)m1);
    }
}
```

Style

- Le **style de programmation** est un ensemble de lignes directrices utilisées lors de l'écriture d'un programme informatique.
- Inclut principalement des questions liées à l'**aspect visuel** du code mais pas uniquement.
- Suivre un (bon) style de programmation permet de rendre le code source plus **lisible** par soi-même et par d'autres (y compris les correcteurs) et permet d'éviter les erreurs.
- Important dans la mesure où un programme est souvent développé par plusieurs auteurs et où une grande partie de la vie d'un programme est consacrée à sa maintenance.
- La qualité d'un style est assez difficile à apprécier et subjective.
- Pour ce cours, pas de règles strictes, plutôt une série de conseils.

Identifiants

- Utilisés pour la dénomination de variables, fonctions, types, et constantes.
- Suites de lettres, chiffres et de ' '. Doivent commencer par une lettre ou ' '. La casse (majuscule/minuscule) compte.
- Dénominations en anglais (de préférence) ou en français, pour autant que vous soyez cohérent.
- Différents styles :
 - ▶ noms composés séparés par des ' ' : `add_interest`,
`number_of_days...` ("old school")
 - ▶ noms composés séparés par des majuscule : `addInterest`,
`numberOfDays...` ("camel case").

Identifiants

En général :

- Ne pas utiliser d'abréviations :
 - ▶ `firstName`, `lastName` au lieu de `fname`, `lname`
- Pas de noms trop longs
 - ▶ `setField` au lieu de `setTheLengthField`
- Eviter les dénominations trop proches
 - ▶ `thisPerishableProduct` au lieu de `perishableProduct` si `perishableProducts` existe.
- Utiliser des noms informatifs pour que le code soit auto-documenté.

```
double tax1;           // sales tax rate
double tax2;           // income tax rate

double salesTaxRate;
double incomeTaxRate;
```


Identifiants

Convention courante (mais non obligatoire) en fonction de la nature de l'identifiant :

- **Variables et fonctions** commencent par une minuscule
 - ▶ `myVar`, `my_var`, `myFunction`, `my_function`
- **Types** commencent par une majuscule
 - ▶ `MyType`, `My_type`
- **Constantes** en majuscule et utilisation de '_'
 - ▶ `MY_CONST`

Identifiants : fonctions

- Utiliser des verbes d'action.
 - ▶ `addInterest`, `convertToAustralianDollars`
- Préfixe `get` et `set` pour obtenir et donner une valeur à une variable.
 - ▶ `getBalance`, `setBalance` pour des opérations sur la variable `balance`.
- Préfixe `is` et `has` pour des fonctions retournant un booléen.
 - ▶ `isOverdrawn`, `hasCreditLeft`

Formatage du code

- Concerne l'indentation, les alignements, l'utilisation des espaces...
- Affecte uniquement la lisibilité du code.
- Une fois un style choisi, le conserver.
- Il existe des outils de formatage automatique
 - ▶ Par exemple, `indent` sous unix/linux.

Formatage : indentation et blocs

■ Indentation :

- ▶ Ajouter un nombre fixe d'espaces (2 ou 4 par exemple) à chaque rentrée dans un nouveau bloc
- ▶ Éviter l'utilisation de tabulations (qui dépendent du système)
- ▶ Souvent gérée automatiquement par votre éditeur de texte

■ Sous-blocs :

- ▶ Découper le code en sous-blocs en laissant une ligne vide entre ces blocs (composés de quelques lignes)
- ▶ Laisser deux lignes vides entre chaque fonction

Formatage : espaces

`for(int i=0;i<n;i++)` vs. `for (int i = 0; i < n; i++)`

Mettre des espaces :

- Avant et après les opérateurs binaires, arithmétiques, et logiques

`b = 1 + 2;`

sauf éventuellement pour mettre en évidence la précedence :

`a*x + b`

- Après les virgules et les points-virgules

`myfunction(3, 4, 5)`

- Après les mots-clés réservés (`for`, `if`, `else`, `do...`)

- Après le signe de début de commentaires inline :

`// this is a comment`

- Pour aligner du code si ça améliore la lisibilité

```
int n      = atoi(argv[1]);    // size of population
int trials = atoi(argv[2]);    // number of trials
```

Formatage : espaces

Ne pas en mettre :

- Entre les opérateurs unaires et leur variable
`*p++ = !a;`
- Autour des parenthèses
`myfunction(5 * (4+5))`
- Autour des opérateurs de sélection
`member.data, node->next, vec[i]...`
- Avant la ponctuation
`myfunction(3, 4, 5)`

Formatage : accolades

- Deux écoles (parmi d'autres) :

```
int add(int a, int b) {  
    int result;  
    if (a) {  
        result = a + b;  
        return result;  
    } else {  
        return result;  
    }  
}
```

```
int add(int a, int b)  
{  
    int result;  
    if (a)  
    {  
        result = a + b;  
        return result;  
    }  
    else  
    {  
        return result;  
    }  
}
```

- Les deux sont valides. La première est utilisée dans ces slides pour gagner de la place.

Formatage : les accolades

Les accolades ne sont pas obligatoires lorsque le bloc ne contient qu'une instruction :

```
if (currentHour < AFTERNOON) {  
    printf("Morning\n");  
} else if (currentHour < EVENING) {  
    printf("Afternoon\n");  
} else {  
    printf("Evening\n");  
}
```

```
if (currentHour < AFTERNOON)  
    printf("Morning\n");  
else if (currentHour < EVENING)  
    printf("Afternoon\n");  
else  
    printf("Evening\n");
```

Attention cependant aux cas ambigus :

```
if (b1)  
    if (b2)  
        printf("here");  
else  
    printf("there");
```

⇔

```
if (b1) {  
    if (b2)  
        printf("here");  
    else  
        printf("there");  
}
```


Documentation et commentaires

Différents types de commentaires :

- Commentaires d'entête : précise l'auteur du code, la date de création/modification, description succincte du contenu du module, copyright, etc.
- Commentaires de documentation : définit le contrat de chaque fonction/procédure de l'interface du module.
- Commentaires de bloc : résume l'action d'une partie de code
- Commentaires *inline* : précise l'intérêt d'une ligne particulière de code

Documentation

```
/* ----- *
 * Creates a m by n matrix with all values set to 0. Returns NULL if
 * m <= 0 or n <= 0 and otherwise a pointer to the new matrix.
 *
 * ARGUMENTS
 * m          The number of rows
 * n          The number of columns
 *
 * RETURN
 * matrix     A pointer to the new matrix or NULL
 *
 * NOTE
 * The returned matrix should be cleaned with matrix_free after usage
 * -----*/
```

```
matrix *matrix_create(unsigned m, unsigned n)
```

Définition non ambiguë des fonctions de l'interface à l'adresse du client.

Un autre programmeur doit être capable de réimplémenter la fonction uniquement sur base des commentaires.

Il existe des outils permettant de générer automatiquement la documentation d'un code source à partir des commentaires et du code.

- Par exemple, [doxygen](#).

Commentaires de bloc et inline

Les commentaires de bloc ou inline doivent décrire principalement le **pourquoi** d'une portion de code, le **comment** étant expliqué par le code lui-même.

Ni trop, ni trop peu :

- les réserver aux parties non triviales.

Un mauvais exemple :

```
i++           // increment i by one
```

- Privilégier l'auto-documentation en utilisant des noms de fonctions et de variables informatifs
- S'il y a besoin d'en mettre trop, c'est probablement que le code manque de clarté. Il vaut mieux alors le réécrire.

Remarque : commentaires imbriqués

En C, on ne peut pas imbriquer des commentaires délimités par `/*...*/`.

Le code suivant génèrera une erreur (pourquoi?) :

```
/* /* coucou */ */
```

Par contre, cette construction est possible :

```
/*  
// coucou  
*/
```

Que vaut la variable `nest` ?

```
int nest = /**/0*/*/1;
```

Quelques bonnes pratiques en vrac

- Déclarer des constantes au lieu d'utiliser des nombres/chiffres dans le code :

Non pas :

```
day = (3 + numberOfDays) % 7;
```

Mais plutôt :

```
const int WEDNESDAY = 3;  
const int DAYS_IN_WEEK = 7;  
day = (WEDNESDAY + numberOfDays) % DAYS_IN_WEEK;
```

- Créer des fonctions courtes
 - ▶ Si elle ne s'affiche pas sur un écran, il est probablement possible de la découper en plusieurs autres méthodes
- Se limiter à maximum trois niveaux de boucles imbriquées
- Pas plus de 80 caractères par ligne.
- Si nécessaire, aller à la ligne après une virgule ou avant un opérateur.

Quelques bonnes pratiques en vrac

- Éviter les écritures ambiguës même si elles sont autorisées par le langage

```
int b1 = 1;  
int b2 = 2;  
if (b1 == b2)  
    printf("%d\n", b1);
```

```
int b1 = 1;  
int b2 = 2;  
if (b1 = b2)  
    printf("%d\n", b1);
```

- Ne pas déclarer les variables sur la même ligne

```
int a, b, c;
```

⇒

```
int a;  
int b;  
int c;
```

- Déclarer les variables juste avant leur utilisation et initialiser les en même temps que leur déclaration.

Quelques bonnes pratiques en vrac

- Ne pas réutiliser une même variable dans des situations différentes
 - ▶ Si elle est judicieusement nommée, cela ne devrait pas vous effleurer l'esprit
- Éviter les répétitions de code. Mieux vaut créer une fonction que copier dix fois la même suite d'instructions.
- S'intéresser au langage
 - ▶ Utiliser les idiomes appropriés
 - ▶ `for` au lieu de `while` si gardien pas trop compliqué, `switch` au lieu de `if` imbriqués, `++`, `+=...`
 - ▶ Fouiller pour vérifier l'existence d'une fonction plutôt que de réinventer la roue

Sources

Programmation modulaire :

- C programming : a modern approach, K.N. King, W. W. Norton & Company, Second edition, 2008.
- Slides du cours INFO0030, Benoît Donnet

Style :

- [A guide to coding style](#), Justus Piater, 2005.
- Notes du cours Algorithmique I, Renaud Dumont, 2009-2010.

Partie 5

Structures de données

14 septembre 2026

Plan

1. Introduction
2. Rappel : Pile et file
3. Arbre
4. Dictionnaire

Plan

1. Introduction
2. Rappel : Pile et file
3. Arbre
4. Dictionnaire

Types de données abstraits

Un **type de données abstrait** définit :

- Un ensemble de données
- Un ensemble d'opérations sur ces données

Types d'**opérations** standards :

- Création, destruction d'un objet du type donné
- Accès aux données
- Modification des données
 - ▶ Insertion et suppression de nouvelle données si l'ensemble est dynamique

Exemples jusqu'ici : nombres complexes, matrices, Grid, PNMLImage et Animal (cf. projets 1 et 2).

Types de données abstraits : implémentation

On implémente **concrètement** un type de données abstrait en utilisant une **structure de données**.

Une structure de données consiste en :

- une représentation des données
- une représentation des **relations** entre ces données

Exemples : tableaux 1D, 2D, liste liée, arbres, graphes...

Pour un même TDA, **plusieurs** implémentations (structures de données) sont généralement possibles.

On analyse les **performances** d'une structure particulière selon deux critères :

- Complexité en **temps** des opérations
- Complexité en **espace** nécessaire pour la structure

Exprimées dans le **pire cas** en fonction de la **quantité** de données présentes dans la structure.

Types de données abstraits : en C (rappel)

Fichier d'entête (.h) contient les prototypes des opérations et la définition du type (opaque). Le fichier source (.c) contient la définition concrète de la structure et l'implémentation des opérations

```
// complex.h

#ifndef _COMPLEXE_H
#define _COMPLEXE_H

// définition du nouveau type

typedef struct complex_t complex;

// prototypes des fonctions

complex *complex_new(double, double);
void complex_destroy(complex *);
void complex_sum(complex *, complex *);
void complex_product(complex *, complex *);
...

#endif
```

```
// complex.c

#include <math.h>
#include "complex.h"

struct complex_t {
    double re, im;
};

complex *complex_new(double re, double im) {
    ...
}

void complex_sum(complex *a, complex *b) {
    ...
}

void complex_product(complex *a, complex *b) {
    ...
}
...
```

On utilise des **pointeurs sur void** (et éventuellement des pointeurs de fonction) si on veut pouvoir stocker des valeurs arbitraires dans la structure.

TDA standard

Depuis plus de 50 ans, plusieurs TDA standards, utiles dans de nombreuses applications, ont été définis et étudiés dans la littérature.

Principalement des ensemble de données dynamiques.

Quelques exemples :

- **Pile et file** : collection d'objets accessibles selon un politique LIFO/FIFO.
- **File à priorité** : collection d'objets accessibles selon un ordre de priorité.
- **Liste** : séquence d'objets accessibles à partir de leur position relative.
- **Arbre** : collection de valeurs associées au nœuds d'un arbre
- **Dictionnaire** : collection d'objets accessibles de manière arbitraire via une clé.
- **Graphe** : collection de valeurs associées aux nœuds d'un graphe et accessible selon ce graphe.

Plan

1. Introduction

2. Rappel : Pile et file

- Principe et applications

- Implémentation par tableau

- Listes liées

- Implémentation par liste liée

3. Arbre

4. Dictionnaire

Plan

1. Introduction

2. Rappel : Pile et file

Principe et applications

Implémentation par tableau

Listes liées

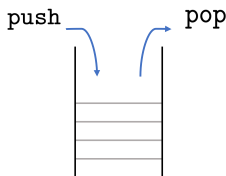
Implémentation par liste liée

3. Arbre

4. Dictionnaire

Pile

Une **pile** est une collection de valeurs, accessibles selon une discipline **LIFO** (Last In First Out)



Interface :

- $\text{push}(s, v)$: ajoute la valeur v au **sommet** de la pile s
- $\text{pop}(s)$: retire la valeur au **sommet** de la pile s , et retourne cette valeur. Signale une erreur si la pile est vide.
- $\text{top}(s)$: retourne la valeur présente au **sommet** de la pile s , sans la dépiler. Signale une erreur si la pile est vide.
- $\text{size}(s)$: retourne le nombre de valeurs présentes dans la pile s .
- $\text{isEmpty}(s)$: détermine si la pile s est vide.
- + création de la structure et libération de la mémoire

File

Une **file** est une collection de valeurs, accessibles selon une discipline **FIFO** (First In First Out)



Interface :

- `enqueue(q, v)` : ajoute la valeur v à la **fin** de la file q
- `dequeue(q)` : retire la valeur au **début** de la file q et la retourne. Signale une erreur si la file est vide.
- `front(q)` : retourne la valeur présente au **début** de la file q , sans la retirer. Signale une erreur si la file est vide.
- `size(q)` : retourne le nombre de valeurs présentes dans la file q .
- `isEmpty(q)` : détermine si la file q est vide.
- + création de la structure et libération de la mémoire

Exemple d'interface en C

stack.h

```
#ifndef _STACK_H
#define _STACK_H

typedef struct Stack_t Stack;

Stack *stackCreate();
void    stackFree(Stack *);
int     stackPush(Stack *, void *);
void    *stackPop(Stack *);
void    *stackTop(Stack *);
int     stackSize(Stack *);
int     stackIsEmpty(Stack *);

#endif
```

queue.h

```
#ifndef _QUEUE_H
#define _QUEUE_H

typedef struct Queue_t Queue;

Queue *queueCreate();
void    queueFree(Queue *);
void    queueEnqueue(Queue *, void *);
void    *queueDequeue(Queue *);
void    *queueHead(Queue *);
int     queueSize(Queue *);
int     queueIsEmpty(Queue *);

#endif
```

Les données sont stockées dans la pile/file sous la forme de pointeurs sur void

- type de retour des fonctions stackPop, stackTop, queueDequeue et queueHead et type du deuxième argument des fonctions stackPush et queueEnqueue).

Applications

Ces deux structures fondamentales ont de nombreuses applications.

Piles

- Allocation de ressources selon un politique “dernier arrivé premier servi”
- Mécanisme d'appels de fonctions dans les langages de programmation
- Implémentation des compilateurs
- Opération undo/back dans certains applications.
- Parcours en profondeur d'abord d'un graphe

Files

- Gestion de ressources selon une politique “premier arrivé, premier servi”
- Transfert de données asynchrone (gestion des opérations printf)
- Gestion de requêtes sur un serveur
- Simulation de files d'attente dans la vie réelle
- Parcours en largeur d'abord d'un graphe.

Une application de la file

On aimerait trier les adresses emails d'une liste en vue d'implémenter le filtre de notre serveur d'emails.

Les adresses se trouvent dans un fichier `addresses.txt` (une adresse par ligne) et on aimerait les placer dans un tableau pour le tri.

Solution sur base d'une file :

- Lire les adresses une par une en les stockant dans une file
- Créer un tableau de la même taille que la file
- Retirer les adresses une par une de la file en les stockant dans le tableau

Peut-on utiliser une pile ?

Une application de la file : en C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "queue.h"

const int BUFFER_SIZE = 1000;
int main() {

    char buffer[BUFFER_SIZE];
    // lecture du fichier (sur l'entrée) et stockage dans une file
    Queue *q = queueCreate();
    while (fgets(buffer, BUFFER_SIZE, stdin)) {
        int lenstr = strlen(buffer)-1;
        buffer[lenstr]='\0';    // supprime la fin de ligne

        queueEnqueue(q, strdup(buffer));
    }
    // Creation et remplissage du tableau
    int sizeQ = queueSize(q);
    char **array = malloc(sizeQ * sizeof(char *));
    for (int i = 0; i<sizeQ; i++)
        array[i] = (char *) queueDequeue(q);

    queueFree(q);
    ...
}
```

Une application de la file : en C

Remarques :

- `char *fgets(char *str, int size, FILE *fp)` : lit une ligne dans le fichier `fp` (qui peut être l'entrée standard) et place le résultat dans la chaîne `str`. La chaîne est terminée par le caractère de fin de ligne (`'\n'`) et le caractère NUL (`'\0'`). Si la ligne fait plus de `size` caractères, seulement les `size-1` premiers sont lus pour éviter un dépassement de `str`. Renvoie `str` si tout s'est bien passé.
- `size_t strlen(char *str)` : renvoie la longueur de la chaîne (caractère NUL non compris).
- `char *strdup(const char *src)` : copie la chaîne `src` dans une nouvelle chaîne (en allouant la mémoire nécessaire) et retourne un pointeur vers cette chaîne.

Plan

1. Introduction

2. Rappel : Pile et file

Principe et applications

Implémentation par tableau

Listes liées

Implémentation par liste liée

3. Arbre

4. Dictionnaire

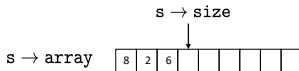
Implémentation par tableau d'une pile : principe

Principes :

- Les valeurs contenues dans la pile sont placées dans les composantes successives d'un **tableau**
- Un indice `size` contient la taille de la pile, qui est aussi la première position libre dans le tableau

La taille du tableau doit être fixée a priori et une erreur signalée lorsque la taille de la pile excède la taille du tableau.

```
const int MAX_STACK_SIZE = 1000;  
  
struct Stack_t {  
    void *array[MAX_STACK_SIZE];  
    int size;  
};
```



Implémentation par tableau d'une pile : code C

Création et suppression.

```
#include <stdlib.h>
#include <stdio.h>
#include "stack.h"

const int MAX_STACK_SIZE = 1000;

struct Stack_t {
    void *array[MAX_STACK_SIZE];
    int size;
};

static void terminate(char *m) {
    printf("%s\n",m);
    exit(EXIT_FAILURE);
}
```

```
Stack *stackCreate() {
    Stack *s = malloc(sizeof(Stack));
    if (!s)
        terminate("Stack can not"\
                  "be created");
    s -> size = 0;
    return s;
}

void stackFree(Stack *s) {
    free(s);
}
```

Implémentation par tableau d'une pile : code C

Accès et insertion.

```
void stackPush(Stack *s, void *data) {  
    if (s -> size >= MAX_STACK_SIZE)  
        terminate("Maximum stack size"\  
                  "reached");  
  
    s -> array[s -> size++] = data;  
}
```

```
void *stackTop(Stack *s) {  
  
    if (s -> size == 0)  
        terminate("Stack is empty");  
  
    return s -> array[s -> size - 1];  
}
```

```
void *stackPop(Stack *s) {  
    if (s -> size == 0)  
        terminate("Stack is empty");  
  
    return s -> array[--(s -> size)];  
}
```

```
int stackSize(Stack *s) {  
    return s -> size;  
}  
  
int stackIsEmpty(Stack *s) {  
    return (s -> size == 0);  
}
```

Complexité en temps et en espace

La complexité en **temps** de toutes les opérations est $O(1)$

La complexité en **espace** est en fait $O(1)$ si la pile/file contient n valeurs, qui exprime que la taille de la structure ne dépend pas de la quantité de données qui y est stockée.

Deux **inconvénients** :

- Il y a une **limite sur le nombre de valeurs** qu'on peut stocker dans la structure
- Utiliser un tableau de taille fixe entraîne un **gaspillage** en terme de mémoire

Pour résoudre ce problème, il faut utiliser une structure de données **dynamique** $\Rightarrow$ **la liste liée**

Plan

1. Introduction

2. Rappel : Pile et file

Principe et applications

Implémentation par tableau

Listes liées

Implémentation par liste liée

3. Arbre

4. Dictionnaire

Liste (simplement) liée

Structure de données composée d'une séquence d'**éléments de liste**.

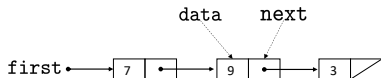
Chaque élément de liste (aussi appelé un nœud) est composé :

- d'un **contenu** utile de type arbitraire (les valeurs qu'on souhaite stocker dans la structure)
- d'un **pointeur vers l'élément suivant** dans la séquence (NULL si l'élément est le dernier de la liste)

Une liste liée est un pointeur vers le premier élément de la liste.

Exemple d'une liste liée d'**entiers** :

```
typedef struct Node_t {  
    int data;  
    struct Node_t *next;  
} Node;
```



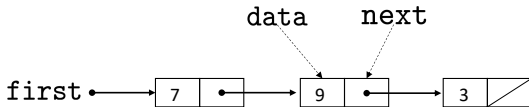
Manipulation d'une liste liée

Construction d'une liste :

```
Node *n1 = malloc(sizeof(Node));  
Node *n2 = malloc(sizeof(Node));  
Node *n3 = malloc(sizeof(Node));
```

```
n1 -> data = 7;  
n1 -> next = n2;  
n2 -> data = 9;  
n2 -> next = n3;  
n3 -> data = 3;  
n3 -> next = NULL;
```

```
Node *first = n1;
```



Manipulation d'une liste liée

Extraction du premier élément

```
int value = first -> data;
```

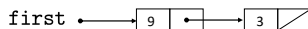
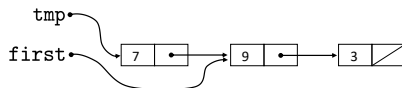
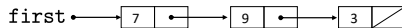
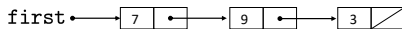
```
Node *tmp = first;  
first = first -> next;
```

```
free(tmp);  
return value;
```

value
7

value
7

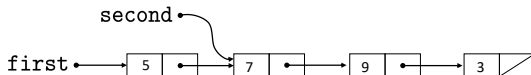
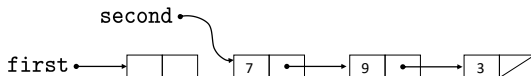
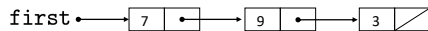
value
7



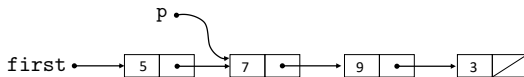
Manipulation d'une liste liée

Ajout d'un élément en début de liste

```
Node *second = first;  
first = malloc(sizeof(Node));  
first -> data = value;  
first -> next = second;
```



Manipulation d'une liste liée : traverser la liste



```
Node *p = first;
while (p != NULL) {
    printf("%d\n", p -> data);
    p = p -> next;
}
```

Sortie :

5
7
9
3

Liste liée versus tableau

Liste liée et tableau peuvent tous deux représenter une séquence de valeurs.

Liste liée :

- Accès relatif uniquement aux éléments de la séquence (via pointeur `next`)
- Taille dépend directement (linéairement) du nombre d'éléments
- Insertion aisée ($O(1)$) de valeurs au milieu de la séquence

Tableau :

- Accès direct aux éléments en fonction de leur rang
- Taille fixée à priori
- Insertion compliquée ($O(n)$) de valeurs au milieu de la séquence.

Plan

1. Introduction

2. Rappel : Pile et file

Principe et applications

Implémentation par tableau

Listes liées

Implémentation par liste liée

3. Arbre

4. Dictionnaire

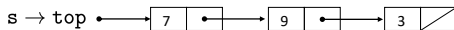
Implémentation d'une pile par liste liée

Principe :

- Les valeurs contenues dans la pile sont retenues dans une **liste liée**.
- L'opération **push** place la valeur en tête de liste. Les opérations **pop** et **top** travaillent en tête de liste également.
- Un indice **size** contient la taille de la pile.

```
struct Node_t {  
    void *data;  
    struct Node_t *next;  
};
```

```
struct Stack_t {  
    Node *top;  
    int size;  
};
```



s → size [3]

Implémentation d'une pile par liste liée

Création et suppression.

```
#include <stdlib.h>
#include <stdio.h>
#include "stack.h"

typedef struct Node_t {
    void *data;
    struct Node_t *next;
} Node;

struct Stack_t {
    Node *top;
    int size;
};

static void terminate(char *m) {
    printf("%s\n",m);
    exit(EXIT_FAILURE);
}
```

```
Stack *stackCreate() {
    Stack *s = malloc(sizeof(Stack));
    if (!s)
        terminate("Stack can not be created");
    s -> top = NULL;
    s -> size = 0;
    return s;
}

void stackFree(Stack *s) {
    Node *n = s -> top;
    while (n) {
        Node *nNext = n -> next;
        free(n);
        n = nNext;
    }
    free(s);
}
```

Implémentation d'une pile par liste liée

Accès et insertion.

```
void stackPush(Stack *s,
               void *data) {
    Node *n = malloc(sizeof(Node));

    if (!n)
        terminate("Stack node can "\
                  "not be created");

    n -> data = data;
    n -> next = s -> top;
    s -> top = n;
    s -> size++;
}

void *stackTop(Stack *s) {
    if (!(s -> top))
        terminate("Stack is empty");

    return s -> top -> data;
}
```

```
void *stackPop(Stack *s) {
    if (!(s -> top))
        terminate("Stack is empty");

    Node *n = s -> top;
    void *data = n -> data;

    s -> top = n -> next;
    s -> size--;

    free(n);

    return data;
}

int stackSize(Stack *s) {
    return s -> size;
}

int stackIsEmpty(Stack *s) {
    return (s -> size == 0);
}
```


Implémentation par liste liée : complexité

Complexité en **temps** est $O(1)$ pour toutes les opérations, comme pour la représentation par tableau.

Complexité en **espace** est $O(n)$ si la pile/file contient n éléments.

Il n'y a **plus de limite** a priori sur la taille de la pile/file.

Plan

1. Introduction

2. Rappel : Pile et file

3. Arbre

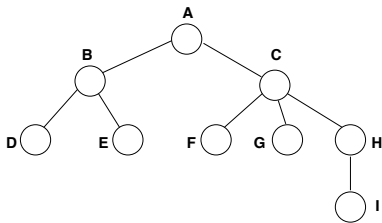
Définitions et opérations

Implémentation

Parcours

4. Dictionnaire

Arbre : définitions



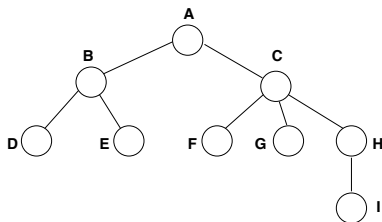
Un arbre (enraciné) T est un graphe dirigé (N, E) , où :

- N est un ensemble de nœuds, et
- $E \subset N \times N$ est un ensemble d'arêtes,

possédant les propriétés suivantes :

- T est connexe et acyclique
- Si T n'est pas vide, alors il possède un nœud distingué appelé **racine** (*root node*). Cette racine est unique.
- Pour toute arête $(n_1, n_2) \in E$, le nœud n_1 est le **parent** de n_2 .
 - ▶ La racine de T ne possède pas de parent.
 - ▶ Les autres nœuds de T possèdent un et un seul parent.

Arbres : définitions



- Si n_2 est le parent de n_1 , alors n_1 est le **fil** (*child*) de n_2 .
- Deux nœuds n_1 et n_2 qui possèdent le même parent sont des **frères** (*siblings*).
- Un nœud qui possède au moins un fils est un nœud **interne**.
- Un nœud **externe** (c'est-à-dire, non interne) est une **feuille** (*leaf*) de l'arbre.

Arbre : structure de données

■ Principe :

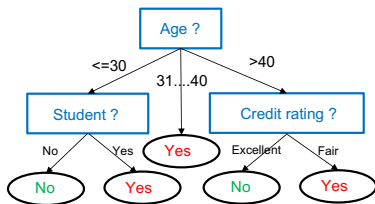
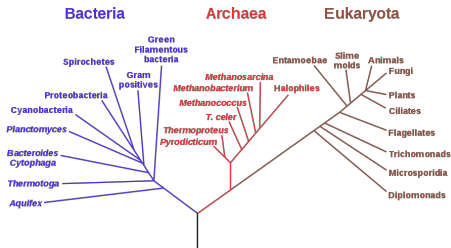
- ▶ Des données sont associées aux nœuds d'un arbre
- ▶ Les nœuds sont accessibles les uns par rapport aux autres selon leur position dans l'arbre

■ Interface : Pour un arbre T et un nœud n

- ▶ $\text{PARENT}(T, n)$: renvoie le parent d'un nœud n (signale une erreur si n est la racine)
- ▶ $\text{ISEMPTY}(T)$: renvoie vrai si l'arbre est vide
- ▶ $\text{CHILDREN}(T, n)$: renvoie une structure de données (ordonnée ou non) contenant les fils du nœud n (liste, tableau, etc.)
- ▶ $\text{ISROOT}(T, n)$: renvoie vrai si n est la racine de l'arbre
- ▶ $\text{ISINTERNAL}(T, n)$: renvoie vrai si n est un nœud interne
- ▶ $\text{ISEXTERNAL}(T, n)$: renvoie vrai si n est un nœud externe
- ▶ $\text{GETDATA}(T, n)$: renvoie les données associées au nœud n
- ▶ $\text{ROOT}(T)$: renvoie le nœud racine de l'arbre
- ▶ Pour un arbre binaire (ordonné) :
 - ▶ $\text{LEFT}(T, n)$, $\text{RIGHT}(T, n)$: renvoie les fils gauche et droit de n
 - ▶ $\text{HASLEFT}(T, n)$, $\text{HASRIGHT}(T, n)$: détermine si le nœud n possède un fils respectivement gauche et droit.

Arbre : applications

Beaucoup de données sont représentées de manière hiérarchique : arbres généalogiques, arbres de décision, arbre phylogénétique, etc.



Les arbres sont utilisés comme structures de données pour de nombreux algorithmes : tri par tas, files à priorité, arbres binaires de recherche, arbre de couverture minimale, etc (voir INFO0902).

Exemple d'interface en C (arbre binaire)

BTree.h

```
#ifndef _BTREE_H
#define _BTREE_H

typedef struct BTreeNode_t BTreeNode;
typedef struct BTree_t BTree;

BTree *btCreate():
void    btFree(BTree *tree);
BTreeNode *btCreateRoot(BTree *tree, int data);
BTreeNode *btInsertLeft(BTree *tree, BTreeNode *n1, int data);
BTreeNode *btInsertRight(BTree *tree, BTreeNode *n2, int data);

BTreeNode *btRoot(BTree *tree);
BTreeNode *btLeft(BTree *tree, BTreeNode *n);
BTreeNode *btRight(BTree *tree, BTreeNode *n);
BTreeNode *btParent(BTree *tree, BTreeNode *n);
int        btGetData(BTree *tree, BTreeNode *n);
int        btSize(BTree *tree);

int        btIsRoot(BTree *tree, BTreeNode *n);
int        btIsInternal(BTree *tree, BTreeNode *n);
int        btIsExternal(BTree *tree, BTreeNode *n);
int        btHasLeft(BTree *tree, BTreeNode *n);
int        btHasRight(BTree *tree, BTreeNode *n);
#endif
```

Exemples d'opération sur un arbre

Calcul de la **profondeur** d'un nœud (sa distance, en nombre d'arêtes, à la racine)

Exemples d'opération sur un arbre

Calcul de la **profondeur** d'un nœud (sa distance, en nombre d'arêtes, à la racine)

Récurivement :

```
int btNodeDepth(BTree *tree, BTreeNode *n) {
```

Exemples d'opération sur un arbre

Calcul de la **profondeur** d'un nœud (sa distance, en nombre d'arêtes, à la racine)

Récurivement :

```
int btNodeDepth(BTree *tree, BTreeNode *n) {  
  
    if (btIsRoot(tree,n))  
        return 0;  
    else  
        return 1+btNodeDepth(tree, btParent(tree, n));  
}
```

Itérativement :

```
int btNodeDepth(BTree *tree, BTreeNode *n) {  
    int depth = 0;  
    while (!btIsRoot(tree,n)) {  
        n = btParent(tree, n);  
        depth++;  
    }  
    return depth;  
}
```

Complexité en temps : $O(n)$, où n est la taille de l'arbre (si les opérations de l'interface sont $O(1)$)

Exemples d'opération sur un arbre

Calcul de la **hauteur** de l'arbre (la plus longue distance, en nombre d'arêtes, d'un nœud à la racine)

```
int btHeight(BTree *tree) {
```

Exemples d'opération sur un arbre

Calcul de la **hauteur** de l'arbre (la plus longue distance, en nombre d'arêtes, d'un nœud à la racine)

```
int btHeight(BTree *tree) {
    return btHeightAux(tree, btRoot(tree));
}

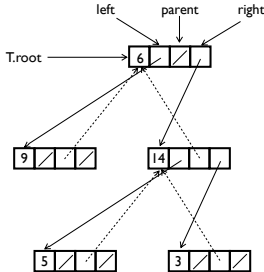
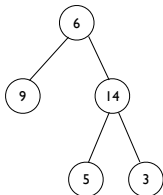
int btHeightAux(BTree *tree, BTreeNode *n) {
    if (btIsExternal(tree, n))
        return 0;
    else {
        int hl = 0;
        int hr = 0;
        if (btHasLeft(tree, n))
            hl = btHeightAux(tree, btLeft(tree, n));
        if (btHasRight(tree, n))
            hr = btHeightAux(tree, btRight(tree, n));
        return 1 + (hl > hr ? hl : hr);
    }
}
```

Complexité en temps : $O(n)$, où n est la taille de l'arbre (si les opérations de l'interface sont $O(1)$)

Implémentation en C (arbre binaire)

En utilisant une **structure liée** :

- **BTNode** : une structure contenant pour chaque nœud n :
 - ▶ Un champ de données (data)
 - ▶ Un pointeur vers son nœud parent (parent)
 - ▶ Un pointeur vers ses fils gauche et droit (left et right)
- **BTree** : une structure contenant un pointeur vers la racine (root)
- Complexité en temps des opérations : $O(1)$
- Complexité en espace : $O(n)$ pour n nœuds



(généralise la notion de liste liée)

Implémentation par structure liée : création et libération

```
#include <stdio.h>
#include <stdlib.h>
#include "BTree.h"

struct BTreeNode_t {
    BTreeNode_t *parent;
    BTreeNode_t *left;
    BTreeNode_t *right;
    int data;
};

struct BTree_t {
    BTreeNode_t *root;
    int size;
};

static void terminate(char *m) {
    printf("%s\n", m);
    exit(EXIT_FAILURE);
}
```

```
BTree *btCreate() {
    BTree *tree = malloc(sizeof(BTree));
    if (!tree)
        terminate("Tree can not be created");
    tree->root = NULL;
    tree->size = 0;
    return tree;
}

static void FreeNodesRec(BTreeNode_t *n) {
    if (!n)
        return;
    FreeNodesRec(n->left);
    FreeNodesRec(n->right);
    free(n);
}

void btFree(BTree *tree) {
    FreeNodesRec(tree->root);
    free(tree);
}
```

Implémentation par structure liée : création de nœuds

```
static BTreeNode *createNode(int data) {  
    BTreeNode *n = malloc(sizeof(BTreeNode));  
    if (!n)  
        terminate("Node can not be created");  
    n->data = data;  
    n->left = NULL;  
    n->right = NULL;  
    n->parent = NULL;  
    return n;  
}
```

```
BTreeNode *btCreateRoot(BTree *tree,  
                        int data) {  
    BTreeNode *root = createNode(data);  
    tree->root = root;  
    tree->size = 1;  
    return root;  
}
```

```
BTreeNode *btInsertLeft(BTree *tree,  
                        BTreeNode *n,  
                        int data) {  
    BTreeNode *nleft = createNode(data);  
    n->left = nleft;  
    nleft->parent = n;  
    tree->size++;  
    return nleft;  
}
```

```
BTreeNode *btInsertRight(BTree *tree,  
                         BTreeNode *n,  
                         int data) {  
    BTreeNode *nright = createNode(data);  
    n->right = nright;  
    nright->parent = n;  
    tree->size++;  
    return nright;  
}
```

Implémentation par structure liée : accesseurs

```
BTNode *btRoot(BTree *tree) {  
    return tree->root;  
}  
  
BTNode *btLeft(BTree *tree, BTNode *n) {  
    return n->left;  
}  
  
BTNode *btRight(BTree *tree, BTNode *n) {  
    return n->right;  
}  
  
BTNode *btParent(BTree *tree, BTNode *n) {  
    return n->parent;  
}  
  
int btGetData(BTree *tree, BTNode *n) {  
    return n->data;  
}  
  
int btSize(BTree *tree) {  
    return tree->size;  
}
```

```
int btIsRoot(BTree *tree, BTNode *n) {  
    return (n->parent==NULL);  
}  
  
int btIsInternal(BTree *tree, BTNode *n) {  
    return (n->left!=NULL || n->right!=NULL);  
}  
  
int btIsExternal(BTree *tree, BTNode *n) {  
    return (n->left==NULL && n->right==NULL);  
}  
  
int btHasLeft(BTree *tree, BTNode *n) {  
    return (n->left!=NULL);  
}  
  
int btHasRight(BTree *tree, BTNode *n) {  
    return (n->right!=NULL);  
}
```

Remarques :

- Pour plusieurs fonctions, l'argument `tree` ne sert à rien. Ca pourrait être le cas cependant pour une autre implémentation.
- On pourrait ajouter une gestion d'erreur (pour empêcher l'accès à un nœud inexistant par exemple)

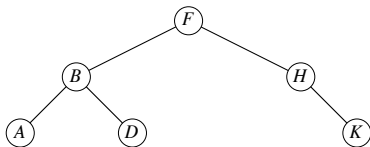
Parcours d'arbres (binaire)

Un parcours d'arbre est une façon d'**ordonner** les nœuds d'un arbre afin de les parcourir

Différents types de parcours :

- Parcours en profondeur :
 - ▶ Infixe (en ordre)
 - ▶ Préfixe (en préordre)
 - ▶ Suffixe (en postordre)
- Parcours en largeur

Parcours infixe



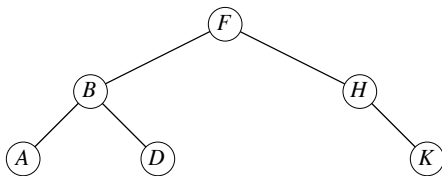
$\Rightarrow \langle A, B, D, F, H, K \rangle$

Parcours infixe (en ordre) : Chaque nœud est visité **après** son fils gauche et **avant** son fils droit

```
static void btInOrderTreeWalk(BTree *tree, BTNode *n) {  
    if (btHasLeft(tree, n))  
        btInOrderTreeWalk(tree, btLeft(tree, n));  
    printf(" %d", btGetData(tree, n));  
    if (btHasRight(tree, n))  
        btInOrderTreeWalk(tree, btRight(tree, n));  
}
```

Appel : `btInOrderTreeWalk(tree, btRoot(tree))`

Parcours préfixe



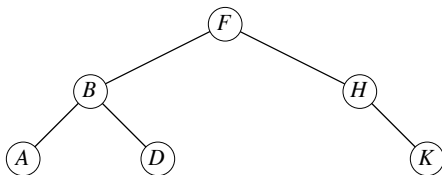
$\Rightarrow \langle F, B, A, D, H, K \rangle$

Parcours préfixe (en préordre) : chaque nœud est visité **avant** ses fils

```
static void btPreOrderTreeWalk(BTree *tree, BTNode *n) {  
    printf(" %d", btGetData(tree, n));  
    if (btHasLeft(tree, n))  
        btPreOrderTreeWalk(tree, btLeft(tree, n));  
    if (btHasRight(tree, n))  
        btPreOrderTreeWalk(tree, btRight(tree, n));  
}
```

Appel : `btPreOrderTreeWalk(tree, btRoot(tree))`

Parcours postfixe



$\Rightarrow \langle A, D, B, K, H, F \rangle$

Parcours postfixe (en postordre) : chaque nœud est visité **après** ses fils

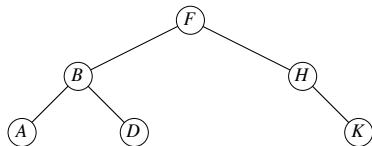
```
static void btPostOrderTreeWalk(BTree *tree, BTreeNode *n) {  
    if (btHasLeft(tree, n))  
        btPostOrderTreeWalk(tree, btLeft(tree, n));  
    if (btHasRight(tree, n))  
        btPostOrderTreeWalk(tree, btRight(tree, n));  
    printf(" %d", btGetData(tree, n));  
}
```

Appel : `btPostOrderTreeWalk(tree, btRoot(tree))`

Parcours en largeur

Parcours en largeur : on visite le nœud le plus proche de la racine qui n'a pas déjà été visité. Correspond à une visite des nœuds de profondeur 1, puis 2, ...

Implémentation à l'aide d'une file :



$\Rightarrow \langle F, B, H, A, D, K \rangle$

```
void btBreadthFirstTreeWalk(BTree *tree) {  
    Queue *q = queueCreate();  
    if (btSize(tree)>0)  
        queueEnqueue(q, btRoot(tree));  
    while (!queueIsEmpty(q)) {  
        BTreeNode *n = queueDequeue(q);  
        printf(" %d ", btGetData(tree, n));  
        if (btHasLeft(tree, n))  
            queueEnqueue(q, btLeft(tree, n));  
        if (btHasRight(tree, n))  
            queueEnqueue(q, btRight(tree, n));  
    }  
    queueFree(q);  
}
```

(Exercice : Implémenter les parcours en profondeur de manière non récursive)

Complexité des parcours

Tous les parcours sont :

- $O(n)$ en temps : dans tous les cas, on ne passe qu'une et une seule fois sur chaque nœud.
- $O(n)$ en mémoire dans le pire cas (*à quoi correspondent-ils ?*)

Plan

1. Introduction

2. Rappel : Pile et file

3. Arbre

4. Dictionnaire

- Principe

- Implémentation par tableau

- Implémentation par liste liée

- Implémentation par table de hachage

- Illustration

Plan

1. Introduction

2. Rappel : Pile et file

3. Arbre

4. Dictionnaire

Principe

Implémentation par tableau

Implémentation par liste liée

Implémentation par table de hachage

Illustration

Dictionnaire : principe

Un dictionnaire est une collection de paires (clé, valeur) où

- clé est une valeur permettant d'identifier de manière unique un élément du dictionnaire. Par exemple : un entier, une chaîne de caractères, etc.
- valeur est une valeur qu'on souhaite associer à cet élément.

Interface :

- `Insert(d, key, value)` : insère la paire (key,value) dans le dictionnaire d. Si la clé s'y trouve déjà, sa valeur est mise à jour.
- `Search(d, key)` : cherche la clé key dans le dictionnaire d. Si la valeur est trouvée, la valeur associée est renvoyée, sinon NULL.
- `Remove(d, key)` : supprime la clé key (et sa valeur) du dictionnaire.
- + création d'un dictionnaire vide et libération de la mémoire.
- + parcours de toutes les clés

Dictionnaire : principe

Un dictionnaire s'appelle aussi un **tableau associatif** ou une **table de symboles**.

Un dictionnaire peut être vu comme un **généralisation d'un tableau** dont les indices sont remplacés par n'importe quel type de clé.

$$\text{Insert}(d, \text{"Pierre"}, 4) \Leftrightarrow d[\text{"Pierre"}] = 4$$

Les clés sont souvent supposées pouvoir être ordonnées mais ce n'est pas toujours nécessaire.

Contraintes de **performance** implicites :

- Les opérations d'insertion et de recherche doivent être les plus efficaces possibles.
- L'espace mémoire consommé doit être minimal et idéalement s'adapter au nombre de clés.

Applications

Les applications sont très nombreuses :

- Liste de contacts : clé = nom, valeur = numéro de téléphone, adresse
- Dictionnaire : clé = mot, valeur = définition
- Recherche internet : clé = mot clé, valeur = liste de pages web
- domain name service (DNS) : clé = nom de domaine (`www.uliege.be`), valeur = adresse IP (`139.165.X.Y`)
- Compilateur : clé = nom de variable, valeur = valeur et type de la variable.
- Système de fichier : clé = nom de fichier, valeur = localisation du fichier sur le disque
- ...

Ensemble : un dictionnaire simplifié

Un **ensemble** (set) est une collection de clés uniques.

Équivalent à un dictionnaire, sans valeurs associées aux clés.

Interface :

- `Insert(s, key)` : ajoute la clé `key` dans l'ensemble `s` si elle ne s'y trouve pas déjà.
- `Contains(s, key)` : renvoie 1 si la clé `key` est contenue dans l'ensemble `s`, 0 sinon.
- `Remove(s, key)` : supprime la clé `key` de l'ensemble `s`.
- + création et libération de la structure.
- + parcours des clés de l'ensemble.

Les techniques d'implémentation d'un dictionnaire peuvent s'adapter trivialement à l'ensemble.

Exemple d'interface en c

En supposant des clés de type `int` et des valeurs de type `void *`.

dict.h

```
#ifndef _DICT_H
#define _DICT_H

typedef struct Dict_t Dict;

Dict *dictCreate();
void dictFree(Dict *);
void dictInsert(Dict *, int, void *);
void *dictSearch(Dict *, int);
int dictContains(Dict *, int);
void *dictRemove(Dict *, int);

#endif
```

set.h

```
#ifndef _SET_H
#define _SET_H

typedef struct Set_t Set;

Set *setCreate();
void setFree(Set *);
void setInsert(Set *, int);
int setContains(Set *, int);
void *setRemove(Set *, int);

#endif
```

Remarque : on ne peut pas remplacer le type de la clé par un type `void *`, sauf à rajouter des pointeurs de fonctions en argument aux fonctions `dicoCreate` et `setCreate`. Par exemple pour passer une fonction de comparaison des clés.

Application 1 : filtrage d'adresses emails

(voir Partie 3, slide 132)

```
...  
#include "set.h"  
  
int main() {  
    char buffer[10000];  
  
    // construction de l'ensemble  
    Set *s = setCreate(100);  
    while (fgets(buffer, 1000, stdin)) {  
        int lenstr = strlen(buffer) - 1;  
        buffer[lenstr]='\0';  
        setInsert(s, strdup(newstring));  
    }  
  
    // filtrage  
    if (setContains(s, "p.geurts@uliege.be")) {  
        ...  
    }  
    setFree(s);  
}
```

Utilisation `./filter < adresses.txt`

Application 2 : suppression des doublons dans un fichier

```
...  
#include "set.h"  
  
int main() {  
    char buffer[1000];  
    Set *s = setCreate(100);  
    while (fgets(buffer, 1000, stdin)) {  
        int lenstr = strlen(buffer)-1;  
        buffer[lenstr]='\0';  
  
        if (!setContains(s, buffer)) {  
            printf("%s\n", buffer);  
            setInsert(s, strdup(newstring));  
        }  
    }  
    setFree(s);  
}
```

Utilisation `./removeduplicates < list.txt > listunique.txt`

Application 3 : compter la fréquence des mots

```
...
#include "dict.h"

int main() {
    char buffer[1000];
    Dict *d = dictCreate(25000);

    while (fgets(buffer, 1000, stdin)) {
        char *string = buffer;
        char *token;
        while ((token = strtok(&string, " \t\n,;.:?\"\\r*-_() [] '")) != NULL) {
            if (strlen(token)>0) {
                int c = dictSearch(d, token);
                if (c == -1) {
                    dictInsert(d, strdup(token), 1);
                } else {
                    dictInsert(d, token, c+1);
                }
            }
        }
    }
    dictPrint(d); // affiche le contenu du dictionnaire
    dictFree(d);
}
```

Utilisation `./countwords < le_rouge_et_le_noir.txt > countings.csv`

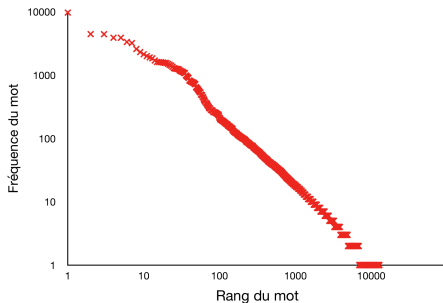
Application 3 : compter la fréquence des mots

Application en linguistique, compression de données, etc.

La loi de Zipf⁴ dit que la fréquence du n ème mot le plus fréquent dans un texte est $f(n) = \frac{K}{n}$ où K est une constante.

Exemple : “le rouge et le noir” de Stendhal⁵ (180000 mots dont 13000 uniques)

Mot	Rang	Freq.
de	1	9738
il	2	4454
la	3	4439
le	4	3902
à	5	3898
l	6	3344
et	7	3212
un	8	2587
que	9	2315
d	10	2140



4. https://fr.wikipedia.org/wiki/Loi_de_Zipf

5. Téléchargé de <https://www.gutenberg.org>

Plan

1. Introduction

2. Rappel : Pile et file

3. Arbre

4. Dictionnaire

Principe

Implémentation par tableau

Implémentation par liste liée

Implémentation par table de hachage

Illustration

Implémentation par tableau

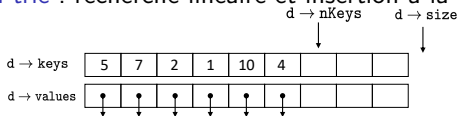
```
struct Dict_t {  
    int *keys;           // clés à valeurs entières  
    void **values;       // valeurs de type (void *)  
    unsigned int size;   // taille du tableau  
    unsigned int nKeys;  // nombre de clés  
};
```

Implémentation par tableau

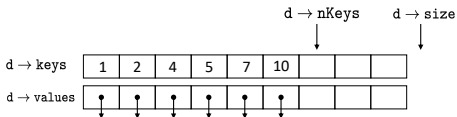
```
struct Dict_t {  
    int *keys;           // clés à valeurs entières  
    void **values;       // valeurs de type (void *)  
    unsigned int size;   // taille du tableau  
    unsigned int nKeys;  // nombre de clés  
};
```

Deux options :

- Tableau de clés **non trié** : recherche linéaire et insertion à la fin



- Tableau de clés **trié** : recherche dichotomique et insertion en décalant vers la droite.



(Implémentation laissée comme exercice)

Complexité

En temps : *(en fonction du nombre de clés n dans le dictionnaire)*

■ Tableau **non trié** :

- ▶ Recherche : $O(n)$
 - ▶ On parcourt le tableau jusqu'à trouver la clé recherchée ($O(n)$).
- ▶ Insertion : $O(n)$
 - ▶ On cherche la clé dans le tableau ($O(n)$) : si présente, on écrase sa valeur ($O(1)$), sinon on la place à la fin du tableau ($O(1)$).

■ Tableau **trié** :

- ▶ Recherche : $O(\log n)$
 - ▶ On utilise la recherche dichotomique.
- ▶ Insertion : $O(n)$
 - ▶ On insère la clé en fin de tableau et on la fait remonter vers la gauche jusqu'à sa position dans l'ordre (cf. tri par insertion)

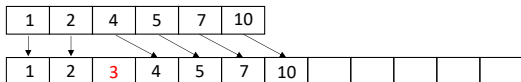
En espace : $O(1)$, car la taille du tableau ne dépend pas du nombre de paires (clé, valeur) stockées.

Tableau extensible

Problème de l'approche précédente : la **taille** du tableau est **fixée** et n'évolue pas en fonction des données (complexité en espace $O(1)$).

Solution :

- Initialisez les tableaux **keys** et **values** à une certaine taille.
- Quand les tableaux deviennent trop petits :
 - ▶ On alloue des nouveaux tableaux, **deux fois plus grands**.
 - ▶ On recopie les anciennes clés et valeurs dans ces nouveaux tableaux.
 - ▶ On libère les anciens tableaux.



Complexités :

- en temps : $O(n)$ (inchangée)
- en espace : $O(n)$ (au pire, on gaspille $O(n)$).

Plan

1. Introduction

2. Rappel : Pile et file

3. Arbre

4. Dictionnaire

Principe

Implémentation par tableau

Implémentation par liste liée

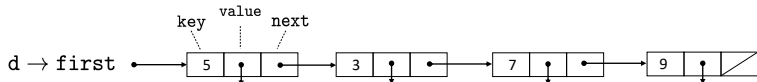
Implémentation par table de hachage

Illustration

Implémentation par liste liée

```
typedef struct Node_t {  
    int key;           // clés à valeurs entières  
    void *value;       // valeurs de type (void *)  
    struct Node_t *next;  
} Node;  
  
struct Dict_t {  
    Node *first;  
    unsigned int nKeys; // nombre de clés (optionnel)  
};
```

Recherche en parcourant la liste, insertion en début de liste.



(Implémentation laissée comme exercice)

Complexités

En temps : *(en fonction du nombre de clés n dans le dictionnaire)*

- Recherche : $O(n)$
 - ▶ On parcourt la liste depuis le début et on s'arrête quand on trouve la clé recherchée (ou quand la fin de liste est atteinte).
- Insertion : $O(n)$
 - ▶ On recherche la clé dans la liste ($O(n)$). Si présente, on écrase sa valeur ($O(1)$), sinon on l'ajoute en début de liste ($O(1)$).

En espace : $O(n)$

- L'espace mémoire nécessaire croît linéairement avec le nombre de paires stockées

Plan

1. Introduction

2. Rappel : Pile et file

3. Arbre

4. Dictionnaire

Principe

Implémentation par tableau

Implémentation par liste liée

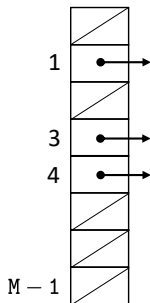
Implémentation par table de hachage

Illustration

Implémentation par tableau : deuxième version

Si on suppose que les clés prennent des valeurs entre 0 et $M - 1$ avec M petit, on peut obtenir une implémentation beaucoup plus efficace.

```
struct Dict_t {  
    void *values[M];  
}  
  
void dictInsert(Dict d, int key, void *value) {  
    d->values[key] = value;  
}  
  
void *dictSearch(Dict d, int key) {  
    return values[key];  
}
```



Complexités :

- en temps : $O(1)$ pour la recherche et l'insertion.
- en espace : $O(1)$, car l'espace mémoire ne dépend pas du nombre de clés.

Table de hachage : fonction de hachage

Problème de l'approche précédente : les clés doivent être **entières** et prendre des valeurs **pas trop grandes**.

Idée 1 : Soit U l'ensemble des valeurs possibles de clés, même non entières. On définit une **fonction de hachage** h :

$$h : U \rightarrow \{0, \dots, M - 1\}$$

envoyant chaque clé $k \in U$ vers une position $h(k)$ dans la table.

Insertion et recherche modifiées (toujours $O(1)$ si h est $O(1)$) :

```
void dictInsert(Dict d, int key, void *value) {
    d -> values[h(key)] = value;
}

void *dictSearch(Dict d, int key) {
    return values[h(key)];
}
```

Table de hachage : gestion des collisions

Problème : Sans connaître les clés, il est difficile de garantir que $h(k_1) \neq h(k_2)$ lorsque $k_1 \neq k_2$. Et c'est impossible dès que $|U| > M \Rightarrow$ le code précédent est incorrect.

Idée 2 : Chaque case i de la table contient une **liste liée** retenant toutes les clés k insérées telles que $h(k) = i$.

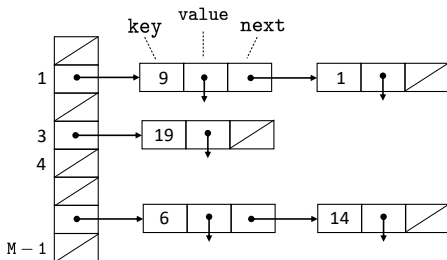


Table de hachage : implémentation (structure et création)

```
typedef struct Node_t {
    int key;
    void *value;
    struct Node_t *next;
} Node;

struct Dict_t {
    Node **array;
    unsigned int arraySize; // taille du tableau
    unsigned int nKeys;     // nombre de clés (optionnel)
};

Dict *dictCreate(int m) {
    Dict *d = malloc(sizeof(Dict));
    if (d == NULL) exit(-1);
    d -> array = calloc(m, sizeof(Node));
    if (d -> array == NULL) exit(-1);
    d -> arraySize = m;
    d -> nKeys = 0;
    return d;
}
```

Remarque : Contrairement à `malloc`, `calloc` initialise la mémoire à 0 (ou `NULL`) en plus de faire l'allocation. Nécessaire ici !

Table de hachage : implémentation (recherche)

```
void *dictSearch(Dict d, int key, void *value) {  
    Node *p = d -> array[h(d, key)];  
    while (p != NULL && p -> key != key)  
        p = p -> next;  
  
    if (p != NULL)  
        return p -> value;  
    else  
        return NULL;  
}
```

Remarque : La fonction de hachage `h` prend la table en argument (voir plus loin).

Table de hachage : implémentation (insertion)

```
void *dictInsert(Dict d, int key, void *value) {
    int hashval = h(d,key);
    Node *p = d -> array[hashval];
    while (p != NULL && p -> key != key)
        p = p -> next;
    if (p != NULL)
        p -> value = value;
    else {
        Node *newNode = malloc(sizeof(Node));
        newNode -> key = key;
        newNode -> value = value;
        newNode -> next = d -> array[hashval];
        d -> array[hashval] = newNode;
        d -> nKeys++;
    }
}
```


Complexité : éléments d'analyse

La complexité dépend de la taille M de la table et de la fonction de hachage (supposée $O(1)$).

Dans le **pire** cas :

- Toutes les clés sont envoyées dans la **même case**.
- $\Rightarrow$ Insertion et recherche en $O(n)$.

Dans le **meilleur** cas :

- Les clés sont réparties **uniformément** dans toutes les cases de la table.
- $\Rightarrow$ Insertion et recherche en $O(n/M)$.
- $\Rightarrow$ Proche de $O(1)$ si M est $O(n)$.

Complexité en espace : $O(M + n)$ (pour la table et pour les éléments de liste liée).

Fonction de hachage

Le choix de la fonction de hachage détermine l'efficacité des opérations :

- Elle doit être **facile à calculer** (c'est-à-dire $O(1)$).
- Elle doit répartir les clés aussi **uniformément** que possible dans la table (très difficile à assurer).

Lorsque les clés sont à valeurs entières, une approche simple est la **méthode de division** :

$$h(k) = k \mod M.$$

En C :

```
static unsigned int h(Dict *d, int key) {  
    return key % d->arraySize;  
}
```

Fonction de hachage : chaînes de caractères

Lorsque la clé n'est pas à valeur entière, il faut d'abord passer par une fonction d'encodage.

Exemples pour les chaînes de caractères :

- On additionne les caractères de la chaîne (naïf) :

```
static unsigned int h(Dict *d, char *key) {
    unsigned int hash = 0;
    while (*key != '\0') {
        hash += *key;
        key++;
    }
    return hash % d->arraySize;
}
```

- Une meilleure approche (djb2) :

```
static unsigned int h(Dict *d, char *key) {
    unsigned long hash = 5381;
    while (*key != '\0') {
        hash = hash * 33 + *key;
        key++;
    }
    return hash % d->arraySize;
}
```

Implémentation générique

On peut définir une table de hachage plus générique en stockant dans la structure une fonction de comparaison et une fonction d'encodage de clés (passées en arguments à dictCreate) :

```
typedef struct Node_t {
    void *key;
    void *value;
    struct Node_t *next;
} Node;

struct Dict_t {
    Node **array;
    unsigned int arraySize;
    unsigned int nKeys;

    int (*compareFunction)(const void *key1, const void *key2);
    unsigned long (*encodeKey)(const void *key);
};

static unsigned int h(Dict *d, void *key) {
    return d -> encodeKey(key) % d->arraySize;
}
```

Plan

1. Introduction

2. Rappel : Pile et file

3. Arbre

4. Dictionnaire

Principe

Implémentation par tableau

Implémentation par liste liée

Implémentation par table de hachage

Illustration

Application au filtrage d'adresses email

Quelle est la complexité du code du slide 319 en fonction de l'implémentation de l'ensemble ?

Création et remplissage de l'ensemble :

- $O(n^2)$ dans le pire cas pour toutes les implémentations
- $O(n \log n)$ avec l'implémentation tableau si on trie le tableau seulement quand toutes les adresses ont été lues (possible seulement si toutes les adresses sont connues à l'avance).
- $O(n)$ avec la table de hachage si fonction de hachage uniforme et table suffisamment grande.

Recherche d'une adresse :

- $O(n)$ dans le pire cas pour la liste liée et la table de hachage.
- $O(\log n)$ pour le tableau trié.
- $O(1)$ avec la table de hachage si fonction de hachage uniforme et table suffisamment grande.

(Complexité de la suppression de doublons et du comptage de mots ?)

Application au filtrage d'adresses email

Tests empiriques :

- Génération de données : n chaînes de caractères (a-z) aléatoires de longueur 10.
- Requêtes : $10n$ chaînes prises au hasard dans la liste (recherches positives) ou en dehors (recherches négatives).

Tableau trié (par fusion) et recherche dichotomique **versus** table de hachage ($M = 2n$, djb2) :

Création de la structure :			Recherches positives		
n	Tri (s)	Table (s)	n	Rech. dico. (s)	Table (s)
12500	0,003	0,002	500000	3,09	1,31
25000	0,005	0,002	1000000	7,95	2,89
50000	0,012	0,007			
100000	0,026	0,013			
200000	0,057	0,036			
400000	0,128	0,085			
			Recherches négatives		
			n	Rech. dico. (s)	Table (s)
			500000	3,76	1,23
			1000000	9,21	2,72

La table de hachage permet de traiter trois fois plus d'adresses à la seconde.

Conclusion

L'implémentation par **table de hachage** est en pratique **plus efficace** que les implémentations par liste liée et par tableau, malgré une complexité dans le pire cas identique, voire moins bonne.

Les performances de la recherche dans un **tableau trié** sont **plus stables** mais l'insertion est beaucoup plus coûteuse, à cause du décalage.

D'autres structures existent qui permettent de combiner la facilité d'insertion de la liste liée avec la rapidité et la stabilité de la recherche dichotomique. Par exemple les arbres binaires de recherche (INFO0902).

Il existe aussi des structures spécifiques pour les chaînes de caractères. Par exemple les *tries*.

Partie 6

Python et langages de programmation

14 septembre 2026

Objectifs

- Vous apprendre un peu plus sur le python, vu comme un langage de programmation plus que comme un outil de calcul numérique.
- Vous montrer l'intérêt d'avoir pratiqué un langage de bas niveau (le C) avant de s'attaquer à un langage de plus haut niveau.
- Vous ouvrir aux différents paradigmes de programmation.

Plan

1. Langages de programmation
2. Catégorisation des langages
3. Un tour du Python
 - Bases du langage
 - Classes
 - Structures de données
 - Gestion de la mémoire
4. Paradigmes de programmation
5. Comparaison des langages

Plan

1. Langages de programmation
2. Catégorisation des langages
3. Un tour du Python
 - Bases du langage
 - Classes
 - Structures de données
 - Gestion de la mémoire
4. Paradigmes de programmation
5. Comparaison des langages

Pourquoi autant de langages ?

- Beaucoup de langages sont spécifiques à une machine
- De nouveaux langages sont constamment introduits pour résoudre les limitations (supposées ou non) des langages existants.
- Les langages doivent suivre les évolutions des machines et les nouveaux développements en génie logiciel
- Pour des questions de hype et de marketing
- Le nombre de degrés de liberté pour le design d'un langage est très important.
- Beaucoup de langages sont spécialisés :
web (JS, PHP, HTML), bases de données (SQL), texte (PERL, TeX),
banques (COBOL), maths (Matlab, Mathematica), GPU (Cuda, OpenCL),
etc.

Pourquoi apprendre plusieurs langages ?

- Il n'existe pas de langage universel ultime qui conviendrait pour tout.
- Pour étoffer sa boîte à outils et aborder plus efficacement de nouveaux problèmes.
- Pour s'adapter à un domaine/environnement de travail, pour pouvoir réutiliser/améliorer les outils de ce domaine.
- Pour apprendre d'autres paradigmes de programmation.
 - ▶ Le langage utilisé façonne le processus de raisonnement
 - ▶ Différents problèmes sont résolus de manière efficace par différentes approches.
- Tant qu'on n'a pas testé un langage, on ne sait pas ce qu'on manque.
- Pour le challenge intellectuel.

Plan

1. Langages de programmation
2. Catégorisation des langages
3. Un tour du Python
 - Bases du langage
 - Classes
 - Structures de données
 - Gestion de la mémoire
4. Paradigmes de programmation
5. Comparaison des langages

Catégorisation des langages

Les langages sont généralement caractérisés selon un ensemble de propriétés, par exemple :

- Expressivité
- Niveau d'abstraction
- Paradigme de programmation
- Processus de compilation/exécution des programmes
- Approche de gestion de la mémoire
- Système de typage des variables/valeurs
- ...

Le **C** est un langage turing complet, de bas niveau, à typage statique faible, de type impératif procédural, compilé et dont la mémoire est gérée manuellement.

Le **Python** est un langage turing complet, de niveau moyen, à typage dynamique fort, multi-paradigme, interprété et dont la mémoire est gérée automatiquement.

Expressivité

Le C et le Python sont des langages **Turing-complet**.

La théorie de l'informatique étudie la classe des fonctions qui sont calculables par une **procédure effective**, c'est-à-dire une suite finie d'instructions qui renvoie toujours une réponse correcte.

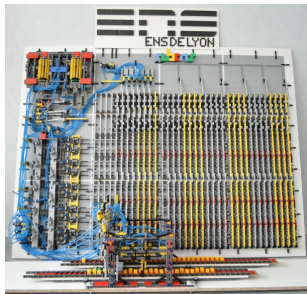
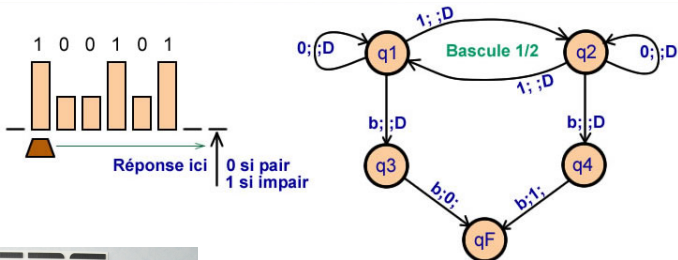
Thèse de Turing-Church : une fonction est calculable par une procédure effective si et seulement si elle est calculable par une **machine de Turing** (voir le slide suivant)

Un langage de programmation est **Turing-complet** s'il est au moins aussi expressif qu'une machine de Turing.

Pas déterminant dans le choix d'un langage car la plupart des langages pratiques sont Turing-complet.

Machine de Turing

Programme calculant la parité du nombre de 1 dans une séquence :



Source : [Le monde, 2017](#)

Source : Wikipedia

Niveau d'un langage

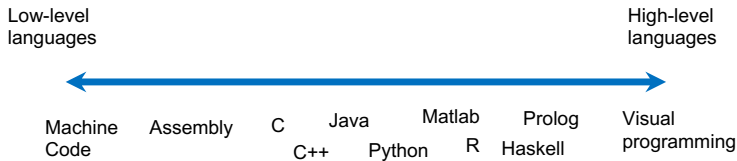
C est un langage de (relativement) **bas niveau**. Python est un langage d'un peu plus **haut niveau**.

Un langage est d'autant plus de **bas niveau** qu'il est proche du langage machine

- Contrôle fin de l'exécution et efficacité mais programmation plus laborieuse.

Un langage est d'autant plus de **haut niveau** qu'il est proche du langage humain

- Programmation plus facile mais contrôle et efficacité moindre.



Plan

1. Langages de programmation
2. Catégorisation des langages
3. Un tour du Python
 - Bases du langage
 - Classes
 - Structures de données
 - Gestion de la mémoire
4. Paradigmes de programmation
5. Comparaison des langages

Remarque : On ne considère ici que la librairie standard du Python, en tant que langage de programmation, pas les libraires supplémentaires telles que `numpy`, `scipy`, `matplotlib`, etc., que vous utiliserez dans d'autres cours.

Rappel : l'ensemble de Mandelbrot

```
#include <stdio.h>
#include <stdlib.h>

#define N 1000 // Maximum number of iterations
#define W 1000 // Width/height of plot (pixels)

static int mandelbrotSet(double xc, double yc);

int main(int argc, char *argv[]) {
    double x = atof(argv[2]);
    double y = atof(argv[3]);
    double size = atof(argv[4]);

    FILE *fp = fopen(argv[1], "w");

    for (int i = 0; i < W; i++) {
        for (int j = 0; j < W; j++) {
            double cr = x - size/2 + size*j/W;
            double ci = y + size/2 - size*i/W;
            fprintf(fp, "%d ", mandelbrotSet(cr, ci));
        }
        fprintf(fp, "\n");
    }

    fclose(fp);

    return 0;
}
```

```
static int mandelbrotSet(double cr, double ci) {
    double zr = 0;
    double zi = 0;
    int n = 0;

    while(n < N && ((zr*zr + zi*zi) <= 4.0)) {
        double temp;
        temp = zr*zr - zi*zi + cr;
        zi = 2*zr*zi + ci;
        zr = temp;
        n++;
    }
    return n;
}
```

Utilisation :

```
> gcc mandelbrot.c -o mandelbrot
> ./mandelbrot m.amat -0.5 0.0 2.0
```

Pour voir l'image avec matlab

```
> M = load('m.amat');
> imagesc(M);
```

Le même programme en python

mandelbrot.py

```
import sys

def mandelbrot(filename, x, y, size, W=1000):
    file = open(filename, "w")
    for i in range(W):
        for j in range(W):
            cr = x - size/2 + size*j/W
            ci = y + size/2 - size*i/W
            file.write(str(mandelbrotSet(cr, ci)))
            file.write(" ")
        file.write("\n")
    file.close()
```

```
def mandelbrotSet(cr, ci, N=1000):
    zr = 0.0
    zi = 0.0
    n = 0
    while n<N and (zr*zr+zi*zi) <= 4.0:
        temp = zr*zr - zi*zi + cr
        zi = 2*zr*zi + ci
        zr = temp
        n += 1
    return n

if __name__ == "__main__":
    filename = sys.argv[1]
    x = float(sys.argv[2])
    y = float(sys.argv[3])
    size = float(sys.argv[4])
    mandelbrot(filename, x, y, size)
```

Deux utilisations possibles :

- En ligne de commande :

```
> python mandelbrot.py m.amat -0.5 0.0 2.0
```

- Dans l'interpréteur :

```
>>> import mandelbrot
>>> mandelbrot.mandelbrot("m.amat", -0.5, 0.0, 2.0)
```

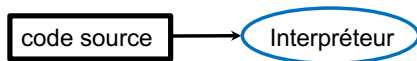
Une première différence : Python est interprété

Le C est un langage **compilé**, alors que le Python est un langage **interprété**.

Langage **compilé** : un compilateur transforme le code source en un code machine qui peut être exécuté efficacement. P.ex., C, C++, fortran.

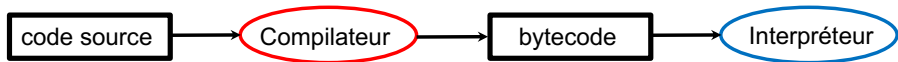


Langage **interprété** : un interpréteur lit et exécute (plus inefficacement) à la volée le code source. P.ex., Matlab, R, Python, Prolog.



Approche hybride (bytecode)

Pour certains langages, un compilateur transforme le code source en un code intermédiaire, le bytecode, qui peut être exécuté plus efficacement par un interpréteur (machine virtuelle).



Exemples : Java, Python (CPython).

Avantage principal : portabilité du bytecode, plus efficace que l'interpréteur direct.

Interprétation peut être interactive (Python) ou pas (Java).

Une deuxième différence : le typage

Le C est un langage à **typage statique faible** alors que le Python est un langage à **typage dynamique fort**.

Le typage d'un langage définit comment sont gérés les types des variables/objets :

- **Statique** (p.ex., C, Fortran, Java)
 - ▶ Toutes les variables ont un type.
 - ▶ Le système contrôle les erreurs de type lors de la compilation.
 - ▶ Plus contraignant mais plus efficace et prévient les erreurs.
- **Dynamique** (p.ex., Python)
 - ▶ Les valeurs, pas les variables, ont un type.
 - ▶ Le système contrôle les erreurs de types lors de l'exécution du code.
 - ▶ Plus souple mais moins efficace et plus sujet aux erreurs.

Le typage est **fort** si toutes les erreurs de type sont détectées (p.ex, Java, Python), **faible** si des opérations peuvent être réalisées sur des valeurs de mauvais types (p.ex, C).

Typage : C versus Python

En C, une variable est un emplacement mémoire de taille fixée et une assignation met une valeur dans l'emplacement.

```
char *a = "coucou";  
a = 4;           // Génère un warning à la compilation  
int c = b * a;    // Génère une erreur à la compilation  
int d = b * (int)a; // Pas de soucis  
short e = d;      // Pas de soucis
```

En Python, une variable est une référence ($\approx$ un pointeur) vers un objet/valeur de type quelconque.

```
a = 4.0  
type(a)           # renvoie le type de la valeur référencée par a (float)  
a = 4  
type(a)           # renvoie le type de la valeur référencée par a (int)  
b = 'CI'  
c = b * a          # c contient 'CICICICI'  
c = b * 4.0        # génère une erreur à l'exécution  
d = 4 + 'coucou'   # génère une erreur à l'exécution
```

Inférence de type

Certains langages à typage statique infèrent le type des variables lors de la compilation. Le type des variables/arguments ne doit pas être précisé explicitement dans le code.

Exemple en OCaml :

```
let rec length = function
[] -> 0
| h::t -> 1+(length t);;

let myf() = length 4;;
```

Malgré le fait que les arguments des fonctions ne soient pas typés, ce code génèrera une erreur à la *compilation*.

Types et opérations de base

- Types numériques : int, float, complex
- Constantes : True, False, None ($\approx$ NULL)
- Opérateurs logiques : and, or, not
- Opérateurs de comparaison : <, <=, >=, ==, !=, is (égalité d'objet), is not...
- Opérateurs mathématiques : +, -, *, /, // (division entière), % (modulo), ** (puissance), abs(x) (valeur absolue)...

```
z1, z2 = 1+5j, 1+5j           # Assignment simultané
print(z1 == z2)               # Renvoie True
print(z1 is z2)               # Renvoie False
print(True == 1 and False == 0) # Renvoie True
```

Contrôle de flux

```
while cond:  
    expr
```

```
for x in iterable:  
    expr
```

```
if cond1:  
    expr1  
elif cond2:  
    expr2  
else:  
    expr3
```

- Corps de la boucle/condition indenté à droite (4 espaces conseillés). Ne définit pas une portée locale.
- Condition ne nécessite pas de parenthèse et est suivie de `:`. `continue` et `break` comme en C
- `iterable` dans le cas du `for` est un objet de n'importe quel type sur lequel on peut itérer (liste, tuple, dictionnaire, set, string, etc.).
 - ▶ `range(start, stop[, step])` définit un itérateur allant de `start` à `stop` (exclu) par pas de `step`.

```
>>> for x in range(1,10,2):  
...     print(x, end=' ')  
13579
```

Fonctions

```
x0, x1, x2 = 0, 1, 2
def ftest(x3, x4 = 0):
    """ Documentation """
    global x0
    x0 = x0 + 1
    x1 = 1
    def ftest2():
        nonlocal x3
        x1 = -1
        x3 = x3 + 1
    ftest2()
    print(x0, x1, x2, x3, x4)
    return True
```

```
>>> ftest(2, 3)
1 1 2 3 3
>>> ftest(2)
2 1 2 3 0
>>> ftest(x4 = 1)
# Erreur (manque le premier argument)
>>> ftest(x4 = 2, x3 = 1)
3 1 2 2 2
```

- `return [value]` pour renvoyer une valeur (pas obligatoire)
- Les variables assignées dans la fonction ont une portée locale (sauf si déclarée `global`)
- Une fonction peut être définie dans une autre fonction (`nonlocal` pour accéder aux variables locales de la fonction appelante).

Un autre exemple

Qu'affichera le code suivant ?

```
def scope_test():
    def do_local():
        spam = "local spam"

    def do_nonlocal():
        nonlocal spam
        spam = "nonlocal spam"

    def do_global():
        global spam
        spam = "global spam"

    spam = "test spam"
    do_local()
    print("After local assignment:", spam)
    do_nonlocal()
    print("After nonlocal assignment:", spam)
    do_global()
    print("After global assignment:", spam)

scope_test()
print("In global scope:", spam)
```


Un autre exemple

Qu'affichera le code suivant ?

```
def scope_test():
    def do_local():
        spam = "local spam"

    def do_nonlocal():
        nonlocal spam
        spam = "nonlocal spam"

    def do_global():
        global spam
        spam = "global spam"

    spam = "test spam"
    do_local()
    print("After local assignment:", spam)
    do_nonlocal()
    print("After nonlocal assignment:", spam)
    do_global()
    print("After global assignment:", spam)

scope_test()
print("In global scope:", spam)
```

After local assignment: test spam
After nonlocal assignment: nonlocal spam
After global assignment: nonlocal spam
In global scope: global spam

Organisation de programmes en module

Des définitions de classes, fonctions ou variables peuvent être rassemblées dans un fichier `.py` appelé un module. E.g, `mandelbrot.py`.

Utilisation d'un module dans un autre module ou l'interpréteur :

- Importation de tout le module (sous un autre nom)

```
import module                # importe le module
import module as mname      # importe le module sous le nom mname
```

Fonction ou variable name accessible ensuite via `module.name`.

- Importation de certaines fonctions :

```
from module import f         # importe la fonction f seulement
from module import f as fname # (sous le nom fname)
from module import *         # importe tout (sauf noms commençant pas _)
```

Equivalent des fichiers `.h/.c` en C. Pas réellement de mécanisme de fonctions statiques ou de type opaques.

Exécution d'un module comme un script

Alternativement, le code d'un module peut être exécuté en ligne de commande :

```
> python module.py <arguments>
```

Un variable `__name__` reçoit la valeur `"__main__"` qui peut être utilisée pour insérer l'équivalent d'une fonction `main` :

```
if __name__ == "__main__":  
    filename = sys.argv[1]  
    x = float(sys.argv[2])  
    y = float(sys.argv[3])  
    size = float(sys.argv[4])  
    mandelbrot(filename, x, y, size)
```

Les arguments de la ligne de commande sont accessibles dans les variables `sys.argv` et `sys.argvc` (comme en C).

Efficacité du python

Calculons les temps de calcul de l'ensemble de Mandelbrot :

```
import time
...
start_time = time.time()
mandelbrot(filename, x, y, size)
end_time = time.time()
print(end_time-start_time)
...
```

Résultats :

- En C : 2.8s
- En Python : 102.8s

Le Python est un des langages les moins performants.

Accélération du code

Il existe des moyens d'accélérer le code, mais qui imposent des contraintes :

- Utilisation autant que possible des fonctions des librairies.
- Utilisation de sous-routines écrites en c.
- Compilation du code (par ex., numba)

La modification suivante fait descendre les temps de calcul à 2.9s :

```
from numba import jit
...
@jit
def mandelbrotSet(cr, ci, N=1000):
    zr = 0.0
    zi = 0.0
    n = 0
    while n < N and (zr*zr+zi*zi) <= 4.0:
        temp = zr*zr - zi*zi + cr
        zi = 2*zr*zi + ci
        zr = temp
        n += 1
    return n
```

Plan

1. Langages de programmation
2. Catégorisation des langages
3. Un tour du Python
 - Bases du langage
 - Classes**
 - Structures de données
 - Gestion de la mémoire
4. Paradigmes de programmation
5. Comparaison des langages

Définition de nouveau type d'objets (= classe)

complex.py

```
from math import sqrt

class Complex:
    """ Complex numbers """
    def __init__(self, realpart, imagpart):
        self.r = realpart
        self.i = imagpart

    def abs(self):
        return sqrt(self.r*self.r + self.i * self.i)

    def add(self, other):
        return Complex(self.r+other.r, self.i+other.i)

    def mul(self, other):
        return Complex(self.r*other.r - self.i*other.i,
                        self.r*other.i - self.i*other.r)
```

Utilisation :

```
from complex import *

c = Complex(-1.001,-1.0)
z = Complex(0,0)
n = 0
while n<1000 and z.abs() <= 2.0:
    z = z.mul(z).add(c)
    n += 1

print(n)  # 19
```

Classe : principe

- Une classe regroupe des champs de données, globaux ou initialisés par la fonction `__init__`, et des méthodes.
- Une instance (objet) de la classe est créée par `nomclasse(args)` qui appelle `__init__(self,args)`, où `self` est le nouvel objet créé.
- On accède aux méthodes en utilisant `object.method(...)`. Le premier argument de la définition d'une méthode est toujours l'objet lui-même qui ne doit pas être donné en argument :

`object.method(args) ⇔ class.method(object,args)`

- La classe peut aussi contenir des variables globales à la classe

```
class MyClass:
    i = 10120
    def test(self):
        print(self.i)
        self.x = 10
```

```
o = MyClass()
o.x = 4
o.i=-4
print(o.x)           # 4
MyClass.test(o)      # -4
print(o.x)           # 10
```


Classes : surcharge des opérateurs

Une version qui s'intègre mieux au langage :

complex.py

```
from math import sqrt

class Complex:
    """ Complex numbers """
    def __init__(self, realpart, imagpart):
        self.r = realpart
        self.i = imagpart

    def __abs__(self):
        return sqrt(self.r*self.r + self.i * self.i)

    def __add__(self, other):
        return Complex(self.r+other.r, self.i+other.i)

    def __mul__(self, other):
        return Complex(self.r*other.r - self.i*other.i,
                        self.r*other.i - self.i*other.r)

    def __repr__(self):
        return "{:.2f}+j*{:.2f}".format(self.r,self.i)
```

Utilisation :

```
from complex import *

c = Complex(-1.001,-1.0)
z = Complex(0,0)
n = 0
while n<1000 and abs(z) <= 2.0:
    z = z * z + c
    n += 1
print(z)  # -1.88+j*-1.00
print(n)  # 19
```

Des fonctions spécifiques de la classe (nommées `__X__`), si elles existent, sont appelées pour appliquer les opérateurs/fonctions standards (+, -, print, abs) aux objets de cette classe.

On parle de **surchage d'opérateurs** (*overloading*).

Classe : polymorphisme

Python permet de définir des fonctions qui peuvent fonctionner pour différents types.

On appelle des fonctions de ce type des fonctions **polymorphiques**.

Exemple :

```
def multadd(x, y, z):  
    return x * y + z  
  
print(multadd(4,2,3))           # 11  
z = Complex(1.88,-1.0)  
c = Complex(-1.001,-1.0)  
print(multadd(z,z,c))           # 1.53+j*-1.00  
print(multadd(1,[1,2],[3,4]))  # [1, 2, 1, 2, 3, 4]
```

Classe : héritage

Une classe peut dériver d'une (ou plusieurs) autre et alors **hériter** de ses méthodes et attributs.

```
class Polygon:
    def __init__(self, no_of_sides):
        self.n = no_of_sides
        self.sides = [0 for i in range(no_of_sides)]

    def inputSides(self):
        for i in range(self.n):
            self.sides[i] = float(input("Enter side "+str(i+1)+" : "))

    def dispSides(self):
        for i in range(self.n):
            print("Side",i+1,"is",self.sides[i])

class Triangle(Polygon):
    def __init__(self):
        Polygon.__init__(self,3)

    def findArea(self):
        a, b, c = self.sides
        # calculate the semi-perimeter
        s = (a + b + c) / 2
        area = (s*(s-a)*(s-b)*(s-c)) ** 0.5
        print('The area of the triangle is %0.2f' %area)
```

```
>>> t = Triangle()

>>> t.inputSides()
Enter side 1 : 3
Enter side 2 : 5
Enter side 3 : 4

>>> t.dispSides()
Side 1 is 3.0
Side 2 is 5.0
Side 3 is 4.0

>>> t.findArea()
The area of the triangle is 6.00

>>> print(isinstance(t, Triangle))
True

>> print(isinstance(t, Polygon))
True

>> issubclass(Triangle, Polygon)
True

>> issubclass(bool, int)
True
```

[source](#)

Plan

1. Langages de programmation
2. Catégorisation des langages
3. Un tour du Python
 - Bases du langage
 - Classes
 - Structures de données**
 - Gestion de la mémoire
4. Paradigmes de programmation
5. Comparaison des langages

Structures de données natives

Le python offre plusieurs structures de données de base natives :

- **str** : chaîne de caractères `'e', 'abcbehd', "apoiazeo"`
- **list et tuple** : séquences d'objets ordonnés `[1,3], ('c', 4), []`
- **range** : séquences ordonnées d'entiers `range(10), range(1,10,2)`
- **set** : ensemble d'objets uniques non ordonnés `{1,2,5}, {}`
- **dict** : dictionnaire `{'key1': 'val1', 'key2': 'val2'}`

Pour chaque type, des méthodes sont accessibles par `objet.method`.

Des opérateurs et constructeurs spécifiques sont également fournis.

Opérations sur les séquences

Les opérations suivantes sont valables pour toutes les types séquentiels : liste, tuple, range (excepté +,*), et chaîne de caractères.

`x in s` : True si un élément de s est égal (==) à x, False sinon.

`s + t` : concaténation de s et t.

`s*n` ou `n*s` : concatène s n fois à lui-même.

`s[i]` : ième élément de s (indexation commence à 0).

`s[i:j]` : tranche (slice) de s de i à j-1. Si $i < 0$ ($j < 0$), l'indice est $\text{len}(s)+i$ ($\text{len}(s)+j$).

`s[i:j:k]` : tranche (slice) de s de i à j-1 avec un pas de k.

`len(s)` : longueur de s.

`min(s)`, `max(s)` : le plus petit/grand élément de s.

Chaînes de caractères (classe str)

Délimitées par des guillemets simples, doubles ou triples :

```
'ceci est une chaîne', "ceci aussi", """ triples guillemets """,  
''' ici aussi '''
```

Pas de type spécifique pour les caractères unique (par ex., 'a', "b", "'")

Une chaîne de caractère est **immuable** (*immutable*). On ne peut pas la modifier une fois créée :

```
s = "immutable"  
s[4] = "p"           # génère une erreur
```

Une chaîne de caractère a souvent une représentation unique en mémoire :

```
s = "immutable"  
s[4] = "p"           # génère une erreur  
t = "im" + "mutable"  
print(s is t)        # True
```

Chaîne de caractères : quelques méthodes

`s.find(sub[, start[, end]])` : l'indice dans `s` de la première occurrence de la sous-chaîne `sub`, -1 si `sub` n'apparaît pas dans `s` (entre `start` et `end`).

`s.join(iterable)` : concatène les chaînes dans la séquence `iterable` séparée par `s`.

`s.split(sep = None, maxsplit=-1)` : renvoie une liste des mots de `s`, en utilisant `sep` comme séparateur.

`s.format(...)` : permet d'insérer des valeurs dans une chaîne pour le formatage de sortie (équivalent de `printf` en C). Voir la documentation.

```
s = "ceci est un court texte"
print(s.find('w'))           # -1
print(s.find("un"))          # 9
print(s.split())              # ['ceci', 'est', 'un', 'court', 'texte']
print(" ".join(s.split()) == s) # True
print("Ceci est un court {}".format("texte"))      # Ceci est un court texte
print("La valeur de pi est {:.3f}".format(3.1416)) # La valeur de pi est 3.142
```


Listes (classe list)

Une liste permet de représenter une séquence de valeurs quelconques, pouvant être indexée par leur position (à partir de 0) et modifiée (*mutable*).

Méthodes spécifiques : `l.extend(iterable)`, `l.insert(i,x)`, `l.remove(x)`, `l.pop(i)`, `l.sort(key=None, reverse=False)`, `l.reverse()`, `l.copy()`

```
l = [1,2,3,4]
l.append(5)
l.append([1,2,3])      # liste imbriquée
l.append("abcd")        # on peut mélanger des types
print(l)               #
print(l[2])            #
print(l[2:3])          #
l[1:3] = []            # supprime les éléments des positions 1 et 2
l[:] = []              # vide la liste (différent de l = [])
```

Implémentation d'une pile et d'une file

```
class Stack:
    def __init__(self):
        self.items = []
    def push(self, val):
        self.items.append(val)
    def pop(self):
        return self.items.pop()
    def isempty(self):
        return self.items == []
```

```
s = Stack()
s.push(1)
s.push(2)
print(s.pop())      # 2
print(s.pop())      # 1
print(s.isempty())  # True
```

```
class Queue:
    def __init__(self):
        self.items = []
    def enqueue(self, val):
        self.items.insert(0, val)
    def dequeue(self):
        return self.items.pop()
    def isempty(self):
        return self.items == []
```

```
q = Queue()
q.enqueue(1)
q.enqueue(2)
print(q.dequeue())  # 1
print(q.dequeue())  # 2
print(q.isempty())  # True
```

Complexité ?

Complexité des opérations

Les listes sont implémentées par des tableaux C (de pointeurs vers des objets).
Les opération de slicing créent des nouvelles listes.

Complexité des opérations ($\text{len}(l)=n$ et $\text{len}(s)=m$) :

- $l[i] : O(1)$
- $l.append(1) : O(1)$ ($O(n)$ si besoin d'étendre le tableau)
- $l.extend(s) : O(m)$ ($O(n + m)$ si besoin d'étendre le tableau)
- $l + s : O(n + m)$ (crée une nouvelle liste)
- $l.pop() : O(1)$
- $l.pop(0) : O(n)$
- $l.insert(0,1) : O(n)$
- $l[i:i+k] : O(k)$
- $x \text{ in } l : o(n)$
- $\text{len}(l) : o(1)$
- $\text{min}(l), \text{max}(l) : o(n)$

[source](#)

⇒ L'opération enqueue du slide précédent est $o(n)$!

Tuple (classe tuple)

Les tuples sont similaires aux listes, si ce n'est qu'ils sont **immutables**. Seules les opérations du slide 381 sont accessibles.

Principalement utilisés pour contenir des séquences d'objets hétérogènes ne devant pas être modifiées (p.ex., listes d'arguments ou de valeurs de retour des fonctions).

Exemples :

```
t = 12345, 54321, 'hello!'
print(t)                # (12345, 54321, 'hello!')
print(t[0])              # 12345
#t[1] = 12345            # génère une erreur
empty = () # tuple vide
singleton = ('hello!')  # tuple singleton
x, y, z = t              # Sequence unpacking
print(x, y, z)           # 12345 54321 hello!
x, y, z = [1, 2, 3]      # works with any sequence
print(x, y, z)           # 1 2 3
```

Range

Une classe pour représenter des séquences d'entiers croissants ou décroissants :

- `range(stop)` : entiers allant de 0 à `stop-1`.
- `range(start, stop)` : entiers allant de `start` à `stop-1`
- `range(start, stop, step)` : entiers allant de `start` à `stop-1` par pas de `step` (peut être négatif)

Exemples :

```
print(list(range(10)))      # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(list(range(1, 11)))   # [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(list(range(0, 10, 3))) # [0, 3, 6, 9]
print(list(range(0, -6, -1))) # [0, -1, -2, -3, -4, -5]
print(list(range(1, 0)))     # []
print(range(10))             # range(10)
```

Ne renvoie pas une liste mais un objet de type **iterable**. Permet de limiter l'espace mémoire en ne générant pas explicitement la liste de valeurs.

Compréhension de listes

Une manière compacte de créer des nouvelles listes à partir d'autres (sans for).

Syntaxe générale :

```
newlist = [expression for item in iterable if condition]
```

équivalent à :

```
newlist = []  
for item in iterable:  
    if condition:  
        newlist.append(expression)
```

Peut contenir plusieurs for l'un à la suite de l'autre qui sont alors **imbriqués**.

```
newlist = [expr for item1 in iter1 if cond1 for item2 in iter2 if cond2]
```

L'expression peut être une compréhension de liste également

```
newlist = [[expr for item1 in iter1 if cond1] for item2 in iter2 if cond2]
```

NB : On peut créer des tuples au lieu de listes en remplaçant les crochets par des parenthèses.

Compréhension de listes : exemples

```
vec1 = [-4, -2, 0, 2, 4]
print([x**2 for x in vec1]) # [ 16, 4, 0, 4, 16]
print([x for x in vec1 if x>=0]) # [0, 2, 4]
print([(x,y) for x in [0,1,2,3] for y in [0,1,2,3] if x!=y])
# [(0,1),(0,2),...,(3,1),(3,2)]
vec2 = [[1,2,3], [4,5,6], [7,8,9]]
print([num for elem in vec2 for num in elem]) # [1, 2, 3, 4, 5, 6, 7, 8, 9]
print([a+b+c for (a,b,c) in vec2]) # [6, 15, 24]
vec3 = [1, 2, 3, -1, 2]
print([x+y for (x,y) in zip(vec1,vec3)]) # [-3, 0, 3, 1, 6]
```

Compréhension de listes : exemples

```
vec1 = [-4, -2, 0, 2, 4]
print([x**2 for x in vec1]) # [ 16, 4, 0, 4, 16]
print([x for x in vec1 if x>=0]) # [0, 2, 4]
print([(x,y) for x in [0,1,2,3] for y in [0,1,2,3] if x!=y])
# [(0,1),(0,2),...,(3,1),(3,2)]
vec2 = [[1,2,3], [4,5,6], [7,8,9]]
print([num for elem in vec2 for num in elem]) # [1, 2, 3, 4, 5, 6, 7, 8, 9]
print([a+b+c for (a,b,c) in vec2]) # [6, 15, 24]
vec3 = [1, 2, 3, -1, 2]
print([x+y for (x,y) in zip(vec1,vec3)]) # [-3, 0, 3, 1, 6]
```

Que fait ce code ?

```
def mystere(matrix):
    return [[row[i] for row in matrix] for i in range(len(matrix))]

matrix = [[1,2,3],[4,5,6],[7,8,9]]
print(mystere(matrix))
```


Compréhension de listes : exemples

```
vec1 = [-4, -2, 0, 2, 4]
print([x**2 for x in vec1]) # [ 16, 4, 0, 4, 16]
print([x for x in vec1 if x>=0]) # [0, 2, 4]
print([(x,y) for x in [0,1,2,3] for y in [0,1,2,3] if x!=y])
# [(0,1),(0,2),...,(3,1),(3,2)]
vec2 = [[1,2,3], [4,5,6], [7,8,9]]
print([num for elem in vec2 for num in elem]) # [1, 2, 3, 4, 5, 6, 7, 8, 9]
print([a+b+c for (a,b,c) in vec2]) # [6, 15, 24]
vec3 = [1, 2, 3, -1, 2]
print([x+y for (x,y) in zip(vec1,vec3)]) # [-3, 0, 3, 1, 6]
```

Que fait ce code ?

```
def mystere(matrix):
    return [[row[i] for row in matrix] for i in range(len(matrix))]

matrix = [[1,2,3],[4,5,6],[7,8,9]]
print(mystere(matrix))
```

Version plus compacte basée sur les fonctions map et zip :

```
def mystere2(matrix):
    return list(map(list,zip(*matrix)))
```

Set

Une classe permettant de représenter une collection non ordonnée d'objets distincts. Créé via `set(iterable)` ou en utilisant les accolades :

```
s = set(['pomme', 'poire', 'abricot', 'banane', 'pomme'])
s = {'pomme', 'poire', 'abricot', 'banane', 'pomme'}
```

Opérations : `x in s`, `len(s)`, `s.add(object)`, `s.remove(object)`,
`s.intersection/union/difference(set)`, `s&t/s|t/s-t...`

Exemples :

```
s = set(['pomme', 'poire', 'abricot', 'banane', 'pomme'])
print('banane' in s)                # True
print(s.intersection({'fraise', 'poire'})) # {'poire'}
print(set('agcgctcgcgagagct'))      # {'c', 't', 'a', 'g'}
s.remove('pomme')
s.add('fraise')
print(s)                            # {'abricot', 'banane', 'fraise', 'poire'}
print(s-{'poire', 'kiwi'})          # {'abricot', 'banane', 'fraise'}
```

Dictionnaire (class dict)

Une classe implémentant un dictionnaire tel que vu dans la partie 5 du cours, i.e., un ensemble de paires (clé,valeur) :

```
a = dict(one=1, two=2, three=3)
b = {'one': 1, 'two': 2, 'three': 3}
c = {}
(c['one'], c['two'], c['three']) = (1,2,3)
print(a==b==c) # True
```

Opérations : key in d, d[key], d[key]=val, d.get(key[,default]), d.pop(key), d.keys(), d.items()...

Exemples :

```
tel = {'jack': 4098, 'sape': 4139}
tel['guido'] = 4127
print(tel) # {'jack': 4098, 'sape': 4139, 'guido': 4127}
print('jack' in tel, tel['jack']) # True 4098
tel.pop('sape')
tel['irv'] = 4127
print(tel) # {'jack': 4098, 'guido': 4127, 'irv': 4127}
print(list(tel)) # ['jack', 'guido', 'irv']
```

Implémentation

Les types `set` et `dict` sont implémentés par des **tables de hachages**.

⇒ les valeurs des clés doivent être **hachables**.

Les valeurs hachables sont tous les types natifs (`int`, `float`, `str`...) et les conteneurs immuables (tuples) s'ils ne contiennent que des éléments hachables :

- Valeurs hachables : `2`, `2.345`, `"jack"`, `(1,2)`, `("val1", (1,2))`...
- Valeurs non hachables : `[1]`, `([1,2],[2,3])`, `{1,2}`...

La fonction `hash` permet d'obtenir la valeur de hachage d'un objet (ou une erreur si l'objet n'est pas hachable)

```
print(hash(2))           # 2
print(hash(2.345))       # 795515838178724866
print(hash((1,2)))       # 3713081631934410656
print(hash("jack"))      # 2252698075858441600
print(hash([1,2],[2,3])) # Erreur
```

Complexité

L'insertion et la recherche d'une valeur dans un `set/dict` contenant n clés est :

- $O(1)$ en moyenne (clés bien réparties dans la table de hachage),
- $O(n)$ dans le pire cas (toutes les clés dans la même case).

Les opérations sur les ensembles en moyenne (en supposant qu'un nouvel ensemble est créé) :

- Union ($s|t$) : $O(\text{len}(s)+\text{len}(t))$
- Intersection ($s\&t$) : $O(\min(\text{len}(s), \text{len}(t)))$
- Différence ($s-t$) : $O(\text{len}(s))$

Pire cas ?

Application : compter l'occurrence des mots (1/3)

Renvoyez une liste triée de paires (mot, nombre d'occurrences) dans un texte (voir slide 321 pour une version C).

Une première version :

```
def countwords1(text):  
    """ return a sorted list of (word,nb occurrences) """  
    word_dict = {}  
    unwanted_characters = ",;.:?\"\\r*-_() [] '"  
  
    filtered_text = ""  
    for c in text:  
        if c in unwanted_characters:  
            filtered_text += " "  
        else:  
            filtered_text += c  
  
    for word in filtered_text.lower().split():  
        if word in word_dict:  
            word_dict[word] += 1  
        else:  
            word_dict[word] = 1  
  
    return sorted(word_dict.items(), reverse = True,  
                  key = lambda x: x[1])
```

Complexité ?

Complexité

En principe, la séquence suivante est $O(l^2)$ si $l = \text{len}(\text{text})$:

```
filtered_text = ""
for c in text:
    if c in unwanted_characters:
        filtered_text += " "
    else:
        filtered_text += c
```

car en général $s+=t$ est $O(\text{len}(s)+\text{len}(t))$ (une nouvelle chaîne est créée).

En pratique, cependant, le Python peut optimiser $s+=t$ en $O(\text{len}(t))$.

```
newtext = ""
for c in text:
    newtext = newtext + c
```

l	time	
10^4	0.001	$\Rightarrow O(l)$
10^5	0.012	
10^6	0.127	

```
newtext = ""
for c in text:
    bidon = newtext
    newtext = newtext + c
```

l	time	
10^4	0.002	$\Rightarrow O(l^2)$
10^5	0.241	
10^6	24.178	

Application : compter l'occurrence des mots (2/3)

Une deuxième version en $O(l)$ garanti dans tous les cas :

```
def countwords2(text):
    """ return a sorted list of (word,nb occurrences) """
    word_dict = {}
    unwanted_characters = ",;.:?\"\\r*_ _() []'"

    filtered_text = []
    for c in text:
        if c in unwanted_characters:
            filtered_text.append(" ")
        else:
            filtered_text.append(c)
    filtered_text = ''.join(filtered_text)

    for word in filtered_text.lower().split():
        if word in word_dict:
            word_dict[word] += 1
        else:
            word_dict[word] = 1

    return sorted(word_dict.items(), reverse = True,
                  key = lambda x: x[1])
```


Application : compter l'occurrence des mots (3/3)

Une troisième version utilisant le module `re` (*regular expressions*) :

```
def countwords3(text):  
    """ return a list of all words appearing in text """  
  
    word_dict = {}  
  
    for word in re.sub(r'[^\\w\\s]', ' ', text).lower().split():  
        if word in word_dict:  
            word_dict[word] += 1  
        else:  
            word_dict[word] = 1  
  
    return sorted(word_dict.items(), key = lambda x: x[1],  
                  reverse = True)
```

- `re.sub(pat, repl, str)` : génère une nouvelle chaîne correspondant à `str` où les occurrences du pattern `pat` sont remplacés par `repl`.
- `r'[^\\w\\s]` : pattern qui match tout sauf des lettres (`\\w`) et des espaces (`\\s`)

Comparaison des temps de calcul

Temps de calcul sur *le rouge et le noir* ($\approx 10^6$ caractères) :

- countwords1 : 0.25s
- countwords2 : 0.20s
- countwords3 : 0.10s

Version C du slide 321 : 0.04s

Application : matrices creuses (1/3)

Un utilisation du dictionnaire pour représenter des matrices creuses, i.e., avec peu d'éléments non nuls.

```
class Sparsematrix():
    """ a class to represent sparse matrices """
    def __init__(self, m, n, mat = None):
        if not (type(m) == int and type(n) == int and m>0 and n>0):
            raise ValueError("init: wrong matrix dimension")
        self.m = m
        self.n = n
        self.dict = {}
        if mat:
            for i in range(m):
                for j in range(n):
                    if mat[i][j] != 0:
                        self.dict[(i,j)] = mat[i][j]

    def __str__(self):
        repr = ""
        for i in range(self.m):
            repr += str(self.get(i,0))
            for j in range(1,self.n):
                repr += " "+str(self.get(i,j))
            repr += '\n'
        return repr

    def getnonzero(self):
        return len(self.dict)
```

Application : matrices creuses (2/3)

```
def get(self, i, j):
    return self.dict.get((i,j),0)

def set(self, i, j, val):
    if val!=0:
        self.dict[(i,j)] = val
    elif (i,j) in self.dict:
        self.dict.pop((i,j))

def __add__(self, other):
    if self.m != other.m or self.n != other.n:
        raise ValueError("addition: wrong matrices dimensions")
    newsm = Sparsematrix(self.m, self.n)
    newsm.dict = self.dict.copy()
    for (key,val) in other.dict.items():
        oldval = newsm.dict.get(key, 0)
        newval = oldval + val
        if newval != 0:
            newsm.dict[key] = newval
        elif oldval != 0:
            newsm.dict.pop(key)
    return newsm
```

Application : matrices creuses (3/3)

Exemple d'utilisation :

```
>> tab1 = [[1,0,0],[1,0,1],[0,0,1],[1,0,0]]
>> tab2 = [[0,1,0],[1,0,2],[0,0,0],[1,0,0]]
>> M1 = Sparsematrix(4, 3, tab1)
>> M2 = Sparsematrix(4, 3, tab2)
>> print(M1)
1 0 0
1 0 1
0 0 1
1 0 0
>> M3 = M1 + M2
>> print(M3)
1 1 0
2 0 3
0 0 1
2 0 0
>> M3.set(0,0,0)
>> print(M3)
0 1 0
2 0 3
0 0 1
2 0 0
```

Plan

1. Langages de programmation
2. Catégorisation des langages
3. Un tour du Python
 - Bases du langage
 - Classes
 - Structures de données
 - Gestion de la mémoire
4. Paradigmes de programmation
5. Comparaison des langages

Gestion de la mémoire dans les langages

Deux approches :

- **Manuelle** : l'utilisateur a la responsabilité d'allouer et de désallouer la mémoire
 - ▶ P.ex., malloc et free en C
 - ▶ Laborieux pour le programmeur mais il a un contrôle total
- **Automatique** : la mémoire doit toujours être allouée mais la désallocation est automatique.
 - ▶ L'algorithme qui s'occupe de désallouer la mémoire s'appelle le ramasse-miettes ("garbage collector").
 - ▶ Pas de fuite mémoire possible mais moins efficace et pas de contrôle de la mémoire par le programmeur

Exemple : réutilisation d'un nom de tableau, C versus Java :

```
double arr[] = malloc(5, sizeof(double));  
...  
free(arr);  
arr = malloc(10, sizeof(double));
```

```
double[] arr = new double[5];  
...  
arr = new double[10];
```

Gestion de la mémoire dans Python

La mémoire est gérée automatiquement dans le Python.

Le Python utilise un ramasse-miette par **comptage de références** : chaque objet inclut un compteur des références à lui-même et lorsque ce dernier tombe à zéro, il est libéré de la mémoire.

```
import sys
a = 'my-string'
print(sys.getrefcount(a)) # 2
b = [a]
c = { 'key': a } # Create a dictionary with a as one of the values.
print(sys.getrefcount(a)) # 4
```

La gestion de cycle requière des algorithmes particuliers :

```
class MyClass:
    pass
a = MyClass()
a.obj = a
del a
```


Conclusions

Le **Python** est un langage turing complet, de niveau moyen, à typage dynamique fort, multi-paradigme, interprété et dont la mémoire est gérée automatiquement.

Une grande force du langage est sa très grande souplesse : il existe énormément de manière de faire la même chose, même dans la librairie standard, et de nombreux modules sont disponibles qui étendent encore les capacités du langage.

C'est également une faiblesse du langage : pas toujours facile de choisir entre différentes alternatives et les compromis en terme de performance ne sont pas toujours clairs.

Plan

1. Langages de programmation
2. Catégorisation des langages
3. Un tour du Python
 - Bases du langage
 - Classes
 - Structures de données
 - Gestion de la mémoire
4. Paradigmes de programmation
5. Comparaison des langages

Paradigmes de programmation

Le C est un langage **impératif procédural**, alors que le Python est **multi-paradigme**.

Il existe de multiple styles (= paradigmes) de programmation

Principaux paradigmes :

- Impératif :

- ▶ Procédural : C, fortran...
- ▶ Orienté objet : C++, Java...

- Déclaratif :

- ▶ Fonctionnel : Lisp, Scheme, OCaml, Haskell...
- ▶ Logique : Prolog, Oz, Coq...

Historiquement, les langages étaient mono-paradigmes. De nos jours, la plupart des langages sont multi-paradigmes, même si tous les paradigmes ne sont pas implémentés avec la même efficacité.

La programmation **impérative** consiste à décrire les calculs comme une série de commandes/instructions modifiant directement l'état du programme (les valeurs stockées dans la mémoire).

Procédural signifie que les séquences d'instructions sont organisées en procédures, fonctions ou sous-routines qui peuvent s'appeler les unes les autres.

Paradigme très populaire, généralement des langages simples de bas niveau. Pas adapté pour des projets de grandes envergures.

Impérative : le programme modifie directement l'état du programme (les champs associés aux objets).

Orientée objet : la programmation est centrée sur les (types de) données.

- On identifie les “choses” qui font partie du problème ou de la solution.
- On décrit ces choses par des variables (champs).
- Ces choses interagissent avec le monde via des méthodes.

Programmation orientée “nom”, plutôt que “verbe”, contrairement au procédural.

Basé sur l'**encapsulation** : les données (objets) et les méthodes associées sont définies conjointement (=classes) et l'accès direct aux données est interdit (=types opaques).

Très bien adaptée à de gros projets collaboratifs. Très utilisée dans l'industrie.

Exemple en Java

Complex.java

```
public class Complex
{
    private final double re; // type opaque
    private final double im;
    public Complex(double real, double imag)
    { re = real; im = imag; }
    public Complex plus(Complex b)
    { // Return the sum of this number and b.
        double real = re + b.re;
        double imag = im + b.im;
        return new Complex(real, imag);
    }
    public Complex times(Complex b)
    { // Return the product of this number and b.
        double real = re * b.re - im * b.im;
        double imag = re * b.im + im * b.re;
        return new Complex(real, imag);
    }
    public double abs()
    { return Math.sqrt(re*re + im*im); }
    public double re() { return re; }
    public double im() { return im; }
    public String toString()
    { return re + " + " + im + "i"; }
}
```

Exemple d'utilisation

```
Complex z0 = new Complex(1.0, 1.0);
Complex z = z0;
z = z.times(z).plus(z0);
z = z.times(z).plus(z0);
StdOut.println(z);
```

Remarques :

- new replace le malloc. Pas besoin de free
- Méthodes directement associées aux objets ($\neq$ procédures appliquées aux objets).
- En Java, pas de pointeurs.

Programmation fonctionnelle *Lisp, Scheme, OCaml, Haskell...*

Approche plus **abstraite** de la programmation. On décrit **quoi** calculer plutôt que **comment** le calculer (= déclaratif).

Un programme calcule en évaluant des **fonctions**, au sens mathématique du terme.

Caractéristiques principales :

- **Fonctions d'ordre supérieur** : fonctions pouvant prendre comme arguments des fonctions et/ou en renvoyer en sortie.
- La **réursion** est utilisée pour la construction de boucles.
- Les **listes** sont la structures de base (**map/reduce/filter**)
- **Pas d'effets de bord** et d'état partagé ($\neq$ impératif)

Avantages : idempotence des fonctions, optimisation, test et débogage plus aisés, permet le calcul concurrent, concision, etc.

Pas “mainstream” mais des touches de fonctionnelles sont présentes dans beaucoup de langages multi-paradigmes (Python, Rust, Swift, Javascript...).

Fonctions d'ordre supérieur en Python

Python n'est pas un langage purement fonctionnel mais il en a quelques caractéristiques.

Fonctions d'ordre supérieur :

```
def maxVal(f, g, x):  
    return max(f(x), g(x))
```

```
def f(x):  
    return x+2
```

```
g = lambda x: 6
```

```
maxVal(f,g,3)
```

```
def maxFun(f, g):  
    def maximumFunction(x):  
        return max(f(x), g(x))  
    return maximumFunction
```

```
maxFun(f,g)(3)  
mv = maxFun(f,g)  
mv(3)
```

- `lambda` permet de définir une fonction non nommée.
- `MaxFun` renvoie une fermeture (*closure*), une fonction avec des variables liées (ici, `f` et `g`).

Listes, map, filter et reduce

```
l = [2, 18, 9, 22, 17, 24, 8, 12, 27]
print(list(filter(lambda x: x % 3 == 0, l)))
# [18, 9, 24, 12, 27]
print(list(map(lambda x: x * 2 + 10, l)))
# [14, 46, 28, 54, 44, 58, 26, 34, 64]
print(reduce(lambda x,y: x + y, l))
# 139
```

- `map(f,l)` applique `f` à tous les éléments de `l`, `filter(f,l)` renvoie la liste des éléments tels que `f(x)==True`.
- `reduce(f, [x1,x2,...,xN])` renvoie

$$f(f(\dots f(f(f(x1, x2), x3), x4), \dots), xN)$$

- `map` et `filter` sont évalués [paresseusement](#), c'est-à-dire seulement quand on y accède (via `list` par exemple).

Test de primalité en Python : approche impérative

Un nombre naturel n est premier si et seulement si :

$$\nexists k \in [2, n]: n \bmod k = 0.$$

Solution impérative procédurale en python :

```
def is_prime(n):  
    k = 2  
    while k < n:  
        if n % k == 0:  
            return False  
        k += 1  
    return True
```

Liste d'instructions à exécuter pas à pas, effets de bords locaux (incrémentations de k).

Test de primalité en Python : approche fonctionnelle

```
def is_prime(n):  
    return not any(filter(lambda k: n%k==0, range(2,n)))  
  
def primes(m):  
    return list(filter(is_prime, range(1,m)))  
  
print(primes(20))  
#[1, 2, 3, 5, 7, 11, 13, 17, 19]
```

Remarques :

- `range(n,m)` renvoie la liste `[n,n+1,...,m-1]`.
- `any` renvoie `True` si au moins une valeur (non `False`) dans la liste, `False` si la liste est vide

Solution particulièrement compacte et lisible (car proche de la définition).

Une autre solution possible en Python, encore plus lisible :

```
def is_prime(n):  
    return not any(n%k==0 for k in range(2,n))
```

Programmation logique

Encore plus déclaratif que le fonctionnel (Rappel : on décrit le *quoi* plutôt que le *comment*).

Inclut plusieurs paradigmes :

- Programmation logique : principalement logique booléenne *Prolog*
- Programmation sous contraintes : contraintes numériques *Oz, AMPL*
- Assistants de preuve (“theorem provers”) *Coq, Agda, Isabelle*

Langages généralement assez inefficaces pour des tâches classiques, basés sur un algorithme de recherche/optimisation.

Plus spécialisés que les autres paradigmes.

Exemples en Prolog : 3SUM et test de primalité

3SUM problem :

```
triplet0([A,B,C],L) :-  
    member(A,L),  
    member(B,L),  
    member(C,L),  
    all_distinct([A,B,C]),  
    A+B+C == 0.  
  
?- triplet0([A,B,C],[1,2,3,-2,-3,5,6,-1]).
```

Test de primalité :

```
is_prime(P) :- P < 4.  
is_prime(P) :- integer(P), P > 3, not(has_factor(P,3)).  
has_factor(N,L) :- N mod L == 0.  
has_factor(N,L) :- L < N-1, L2 is L + 1, has_factor(N,L2).
```

Exemples en Prolog : puzzle

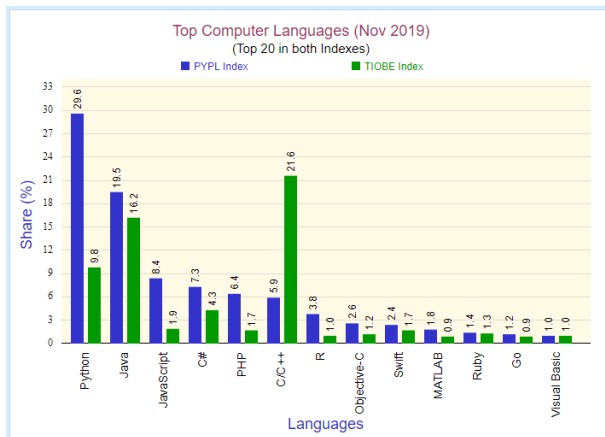
```
/* Houses logical puzzle: who owns the zebra and who drinks water?
1) Five colored houses in a row, each with an owner, a pet,
   cigarettes, and a drink.
2) The English lives in the red house.
3) The Spanish has a dog.
4) They drink coffee in the green house.
...
15) The Norwegian lives near the blue house.
Who owns the zebra and who drinks water?*/
houses(Hs) :- length(Hs, 5),                                     % 1
               member(h(english,_,_,_,red), Hs),                % 2
               member(h(spanish,dog,_,_,_), Hs),                % 3
               member(h(_,_,_,coffee,green), Hs),              % 4
               member(h(ukrainian,_,_,tea,_), Hs),              % 5
               next(h(_,_,_,_,green), h(_,_,_,_,white), Hs),    % 6
               ...
               next(h(norwegian,_,_,_,_), h(_,_,_,_,blue), Hs), % 15
               member(h(_,_,_,water,_), Hs),                    % one of them drinks water
               member(h(_,zebra,_,_,_), Hs).                     % one of them owns a zebra

?- zebra_owner(Owner).
?- water_drinker(Drinker).
```

Plan

1. Langages de programmation
2. Catégorisation des langages
3. Un tour du Python
 - Bases du langage
 - Classes
 - Structures de données
 - Gestion de la mémoire
4. Paradigmes de programmation
5. Comparaison des langages

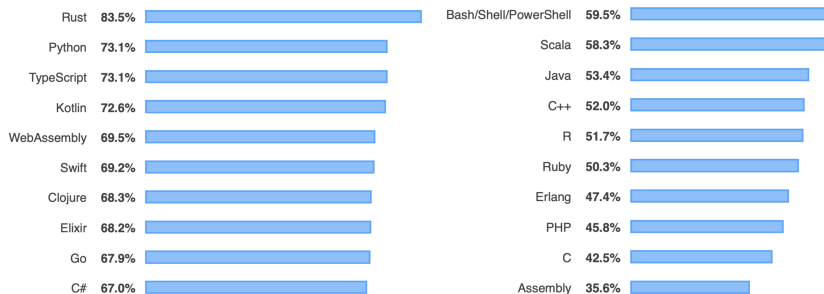
Les langages les plus populaires



Source

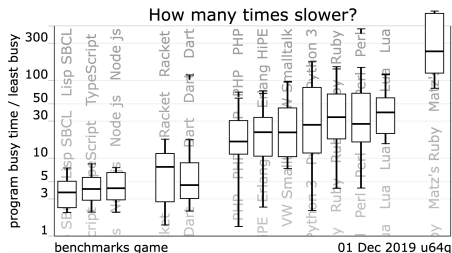
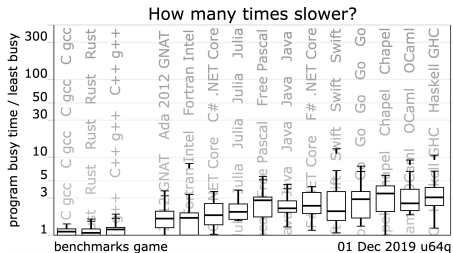
Deux indices de popularité : [PYPL](#) (recherche d'aide sur google) et [TIOBE](#) (basé sur plus de sources).

Les langages les plus aimés



Sondage sur [stackoverflow](https://stackoverflow.com) (en 2019) : pourcentage de développeurs utilisant le langage et ayant exprimé avoir l'intention de continuer de développer dans ce langage. (Seulement le top 10 et le bottom 10 du classement).

Les langages les plus performants



Source : [The computer language benchmark games](#)

- L'ordonnée montre combien de fois plus lente est la meilleure solution à chaque tâche en moyenne par rapport à la meilleur solution parmi tous les langages.
- Pas toujours le critère le plus pertinent.

Quel langage et pour quoi ?

Parmi les langages généralistes, orientés applications scientifiques :

- **C/C++** : Valeurs sûres. Efficace. C++ mieux adapté pour des gros projets.
- **Java** : Efficace. Très utilisé dans les entreprises de développement logiciel.
- **Python** : Très utilisé en recherche. Prototypage très facile, multi-paradigmes, interprété, énormément de bibliothèques externes...
- **Matlab** : Utilisé dans le monde académique et dans les sciences de l'ingénieur. Beaucoup de bibliothèques dédiées à l'ingénierie. Gros défaut : payant (le seul de cette liste).
- **R** : Orienté sciences des données et statistiques. Interprété, beaucoup de bibliothèques et outils de visualisation.
- **Julia** : Orienté calcul scientifique (comme Matlab). Très efficace, gratuit. Manque de bibliothèques, en comparaison avec Python ou Matlab.

Pour les plus aventureux :

- **Rust** : Langage qui monte. Multi-paradigme, sécurisé, concurrent, très efficace et peu gourmand en mémoire. Manque pour l'instant de bibliothèques.
- **Haskell, OCaml** : Pour ceux qui veulent tester la programmation fonctionnelle.

Programmations et langages dans les cours d'informatique

- INFO0902 - Structures de données et algorithmes : Impérative procédurale en C.
- INFO0062 - Object-oriented programming : Orientée objet (en Java).
- INFO0012 - Computation structures : langage d'assemblage.
- INFO9012 - Parallel programming : Programmation parallèle (C).
- INFO0004 - Projet de programmation orientée-objet : Orientée objet (en C++).
- INFO0054 - Programmation fonctionnelle : Fonctionnelle (en Scala).
- INFO8006 - Introduction to artificial intelligence : Logique (un peu), projets en Python.
- INFO0009 - Base de données : Langage d'interrogation de base de données (SQL).

Références

Sources d'inspiration : Cyril Soldani (présentation aux geeks anonymes), John R. Woodward, Adam Byrtek, Robert Sedgewick/Kevin Wayne.

Références sur le python :

- La documentation officielle : <https://docs.python.org/3/>
- Wenworth et al., *how to think like a computer scientists* (plusieurs éditions disponibles gratuitement).